2306 项目火车购票系统









一、项目简介

课程特色：

- 千万级流量的大型分布式系统架构
- 高性能、高并发、高可用场景
- 前后端使用最新的技术栈

技术栈

- 前后端分离搭建：springcloud Alibaba+vue3
- 高并发技术，实现高性能抢票
- 高并发：缓存、分布式锁、限流、令牌、削峰、排队、等
- 解决的问题：缓存击穿、缓存穿透、缓存雪崩

开发版本

- Springboot3+jdk17
  - jdk17
  - SpringBoot3.0.0
  - SpringCloud Alibaba 2022.0.0.0
- Mybatis
- MybatisPlus

项目前后端使用的技术



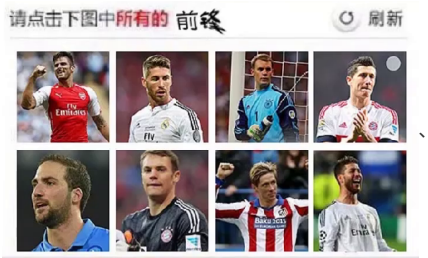
- *CDN*的全称是Content Delivery Network，即内容分发网络

二、12306 系统架构

2.1 架构解决的问题

- 动态库存
- 选座功能
- 线上线下
- 持续高并发业务，需要更综合的高并发设计
- 不停的刷票
- 绝不能超卖

2.2 解决方案

- 提高处理能力：QPS和TPS
  - 堆积硬件：就是不断增加服务器（不推荐 ）
  - 软件：Gemfire（商业软件，分布式内存数据库；查询时间从15秒提高到0.2秒）
  - 算法：模型、逻辑
- 削峰：实现削峰填谷
  - 业务：验证码 

、分时段（每天可以分为21个时间段）、排队
- 技术：限流、异步（消息队列MQ）

2.3 系统如何保证其正常及稳定性

2.3.1 持续秒杀高并发技术

- 前端
  - 针对静态资源做CDN（内容分发网络）
  - 页面静态化
  - 倒计时&Loading（前端优化）
  - 使用验证码削峰
- 后端
  - 微服务-服务拆分
- 负载均衡
  - 限流降级
- 缓存（缓存击穿、缓存穿透、缓存雪崩）
  - 本地缓存
  - 分布式缓存
  - 令牌（token）
- 异步处理（请求和处理分开）

2.3.2 数据库

- 分库：
  - 业务分库
  - 读写分离
- 分表：
  - 横向分表：按地区或时间

    - 纵向分表：一张表纵向分为多张表
        - 例如文章表：标题、作者、时间可以拆成一张表
        - 例如文章表：内容(大字段)拆成一张表
- 冗余设计，反范式（冗余设计），空间换时间
- 分布式数据库

2.3.3 其它

- 分时段秒杀
- 弹性扩容
- 候补+排队

2.4 如何保证不超卖、不少卖，还要能承受极高的并发

- **模型设计&逻辑实现**
-

ABCDE	5站：4+3+2+1
A	B
A	C
A	D
A	E
B	C
B	D
B	E
C	D
C	E
D	E

    - 余票查询：记录站站余票
        - 一列火车有5个站，可拆分成4+3+2+1=10条站站记录
    - 座位购买：记录座位销售详情
        - 一列火车有5个站AE，1号座位：0111，代表只剩AB可买

2.5 系统核心功能

2.5.1 核心功能：

- 会员：
    - 登录/注册：使用手机+验证码
    - 乘客管理：
    - 余票查询：查询某日起点到终点的所有车次及余票
    - 车票购买：选择车次、座位购买
    - 我的车票：查看购买的车票
- 管理：
    - 基础车次维护：车站、车次、车厢、座位、到站
    - 每日车次维护：通过车次基础数据生成每日车次数据
    - 会员管理：注册会员
    - 车票管理：查看已售出车票

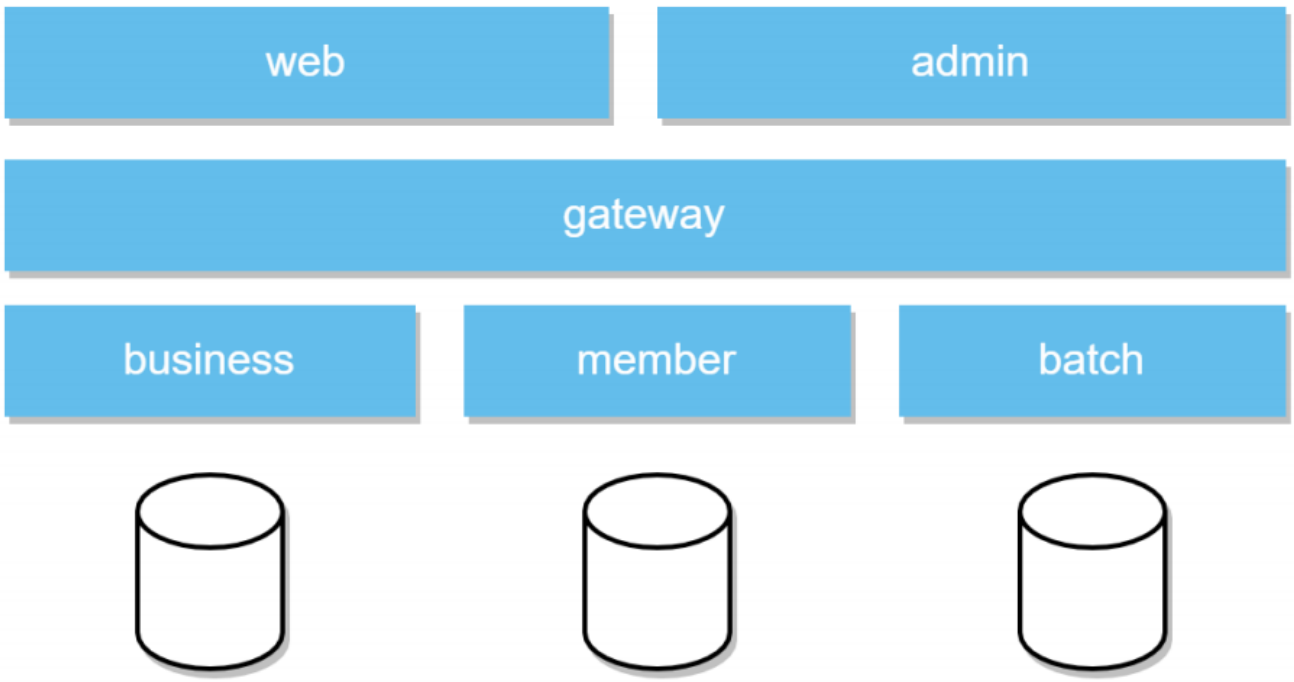
2.6 系统功能模块

- Gateway 网关模块：
    - 路由转发
    - 登录校验
- Member 会员模块：
    - 会员
    - 乘客
    - 已购买的车票
- Business 业务模块：

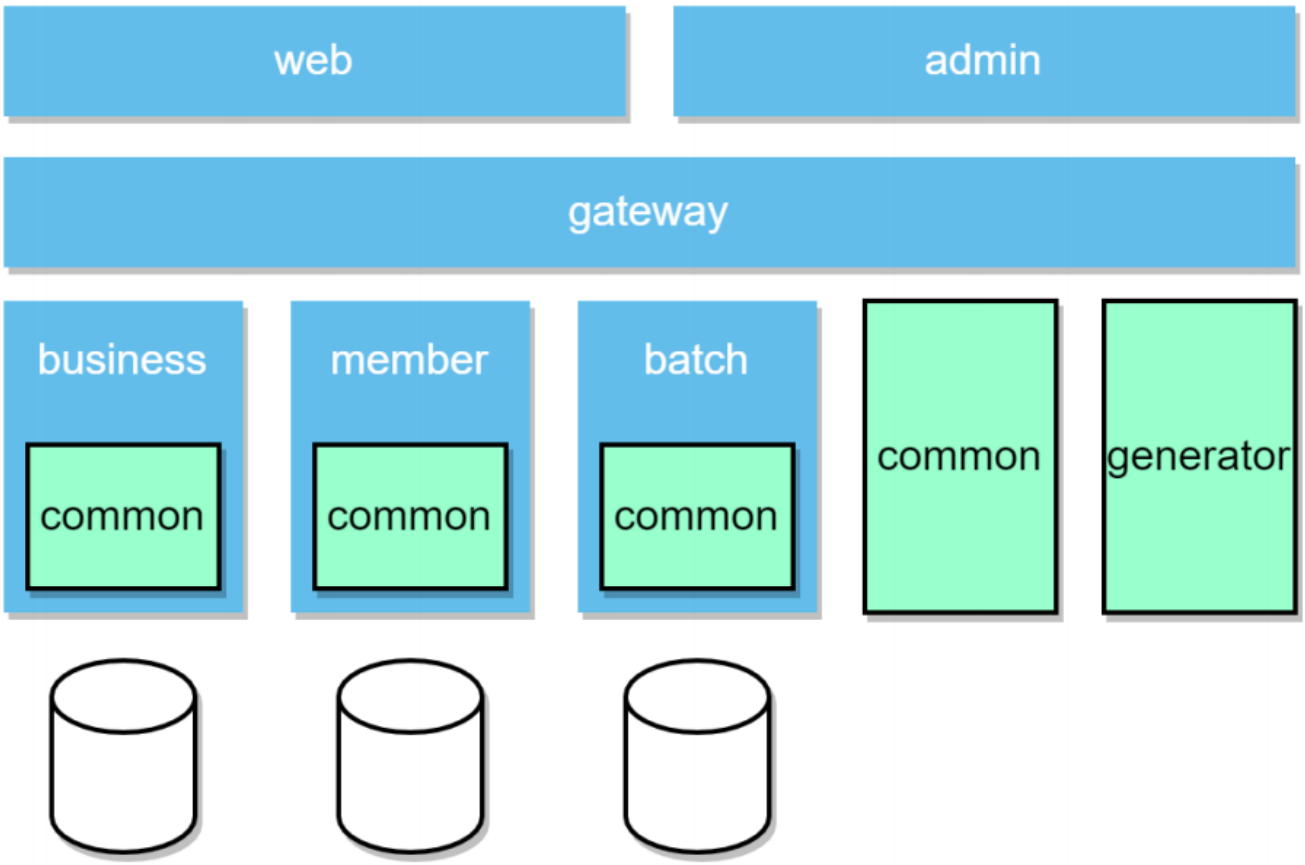
    - 所有的车次数据
    - 余票信息
- Batch 跑批模块：
    - 所有的定时任务（spring 定时任务）
    - 可通过界面启停
- Web 模块：
    - 会员相关界面
- Admin 模块：
    - 管理员相关界面

2.7 整体系统架构设计

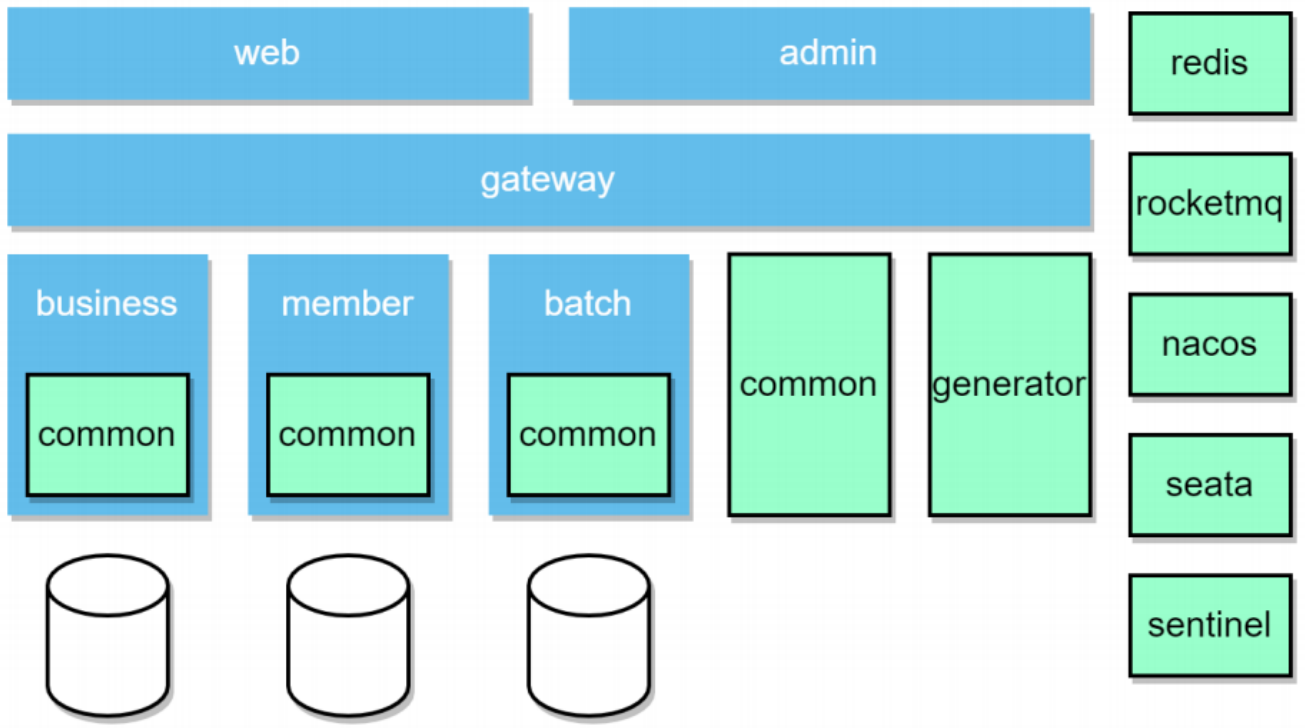
2.7.1 简单的模块划分

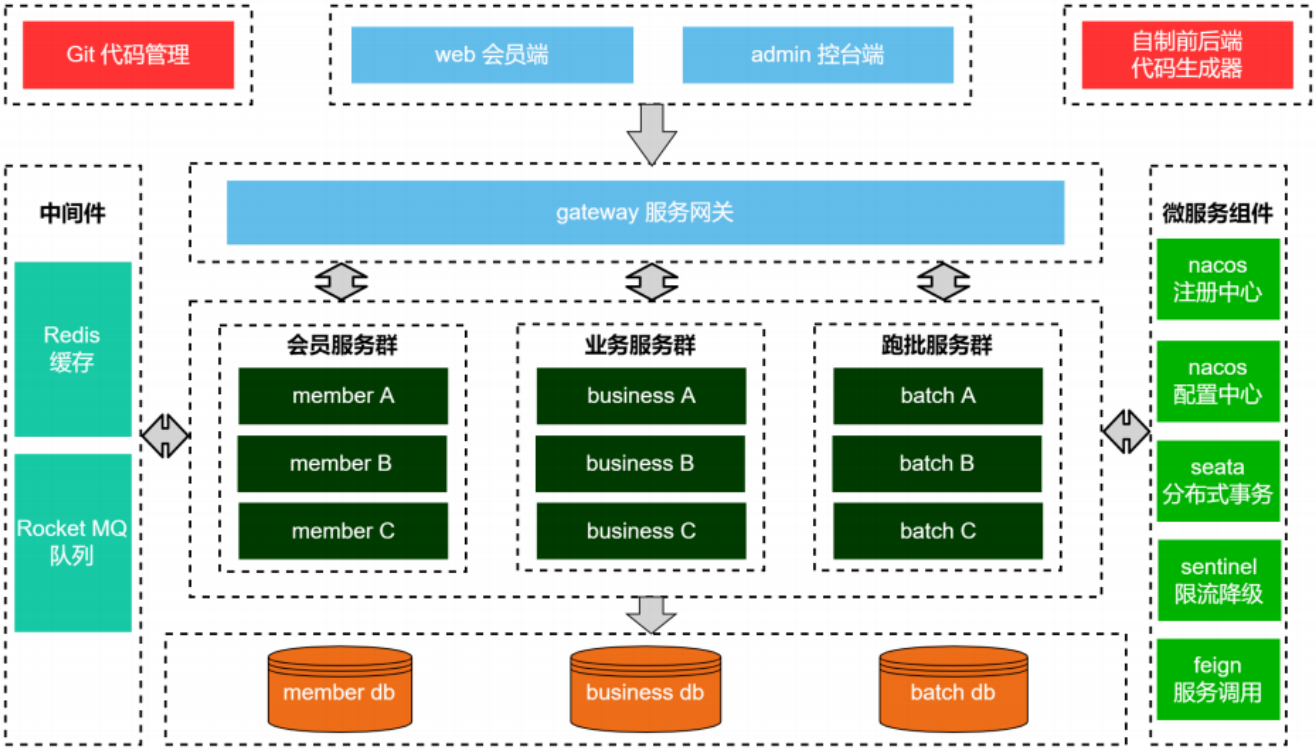


2.7.2 加入common模块，放置公共代码；



2.7.3 加入中间件及微服务组件





2.8 系统数据库表

2.8.1 会员模块：

- 会员表：手机号
- 乘客表：会员ID，姓名，身份证，旅客类型（成人、儿童、学生、军人等）
- 车票表：会员ID，乘客ID，乘客姓名，日期，车次信息，座位信息

2.8.2 业务模块：

- 车站表：站名，站名拼音
- 车次表：车次编号，车次类型，始发站，出发时间，终点站，到站时间
- 到站表：车次编号，站名，进站时间，出站时间，停站时长，里程
- 车箱表：车次编号，箱号，座位类型，座位数，排数，列数
- 座位表：车次编号，箱号，排号，列号
- 每日车次表：日期，基础车次信息
- 每日到站表：日期，基础到站信息
- 每日车箱表：日期，基础车箱信息
- 每日座位表：日期，基础座位信息，销售详情
- 每日余票表：日期，车次编号，出发站，出发时间，到达站，到站时间，各种座位的余票信息

2.8.3 其它：

- quartz相关表：定时任务
- seata相关表：分布式事务管理

名	自动递...	修改日期	数据长度	表类型	行	注释
confirm_order	0		16 KB	InnoDB	0	确认订单
daily_train	0		16 KB	InnoDB	0	每日车次
daily_train_carriage	0		16 KB	InnoDB	0	每日车厢
daily_train_seat	0		16 KB	InnoDB	0	每日座位
daily_train_station	0		16 KB	InnoDB	0	每日车站
daily_train_ticket	0		16 KB	InnoDB	0	余票信息
member	0		16 KB	InnoDB	0	会员
passenger	0		16 KB	InnoDB	0	乘车人
qrtz_blob_triggers	0		16 KB	InnoDB	0	
qrtz_calendars	0		16 KB	InnoDB	0	
qrtz_cron_triggers	0		16 KB	InnoDB	0	
qrtz_fired_triggers	0		16 KB	InnoDB	0	
qrtz_job_details	0		16 KB	InnoDB	0	
qrtz_locks	0		16 KB	InnoDB	0	
qrtz_paused_trigger_grps	0		16 KB	InnoDB	0	
qrtz_scheduler_state	0		16 KB	InnoDB	0	
qrtz_simple_triggers	0		16 KB	InnoDB	0	
qrtz_simprop_triggers	0		16 KB	InnoDB	0	
qrtz_triggers	0		16 KB	InnoDB	0	
sk_token	0		16 KB	InnoDB	0	秒杀令牌
station	0		16 KB	InnoDB	0	车站
ticket	0		16 KB	InnoDB	0	车票
train	0		16 KB	InnoDB	0	车次
train_carriage	0		16 KB	InnoDB	0	火车车厢
train_seat	0		16 KB	InnoDB	0	座位
train_station	0		16 KB	InnoDB	0	火车车站
undo_log	1		16 KB	InnoDB	0	

定时任务表

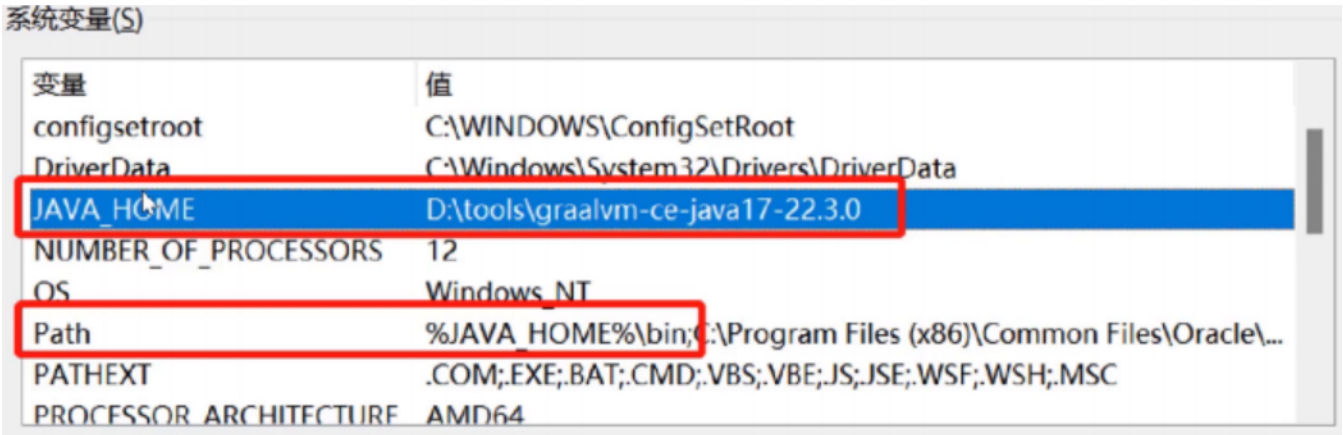
三、Springboot3

3.1 AOT与JIT

- JIT（Just-in-Time，实时编译）一直是Java语言的灵魂特性之一，与之相对的AOT（Ahead-of-Time，预编译或提前编译）方式;
- AOT的优点（云原生，为云而生）
  - 在程序运行前编译，可以避免在运行时的编译性能消耗和内存消耗
  - 可以在程序运行初期就达到最高性能，程序启动速度快(传统**springboot**启动**2**秒，**AOT**需要**100**毫秒)
  - 运行产物只有机器码，打包体积小
- AOT的缺点
  - 由于是静态提前编译，不能根据硬件情况或程序运行情况择优选择机器指令序列，理论峰值性能不如JIT
  - 没有动态能力：没有AOP
  - 编译后的程序不能跨平台运行

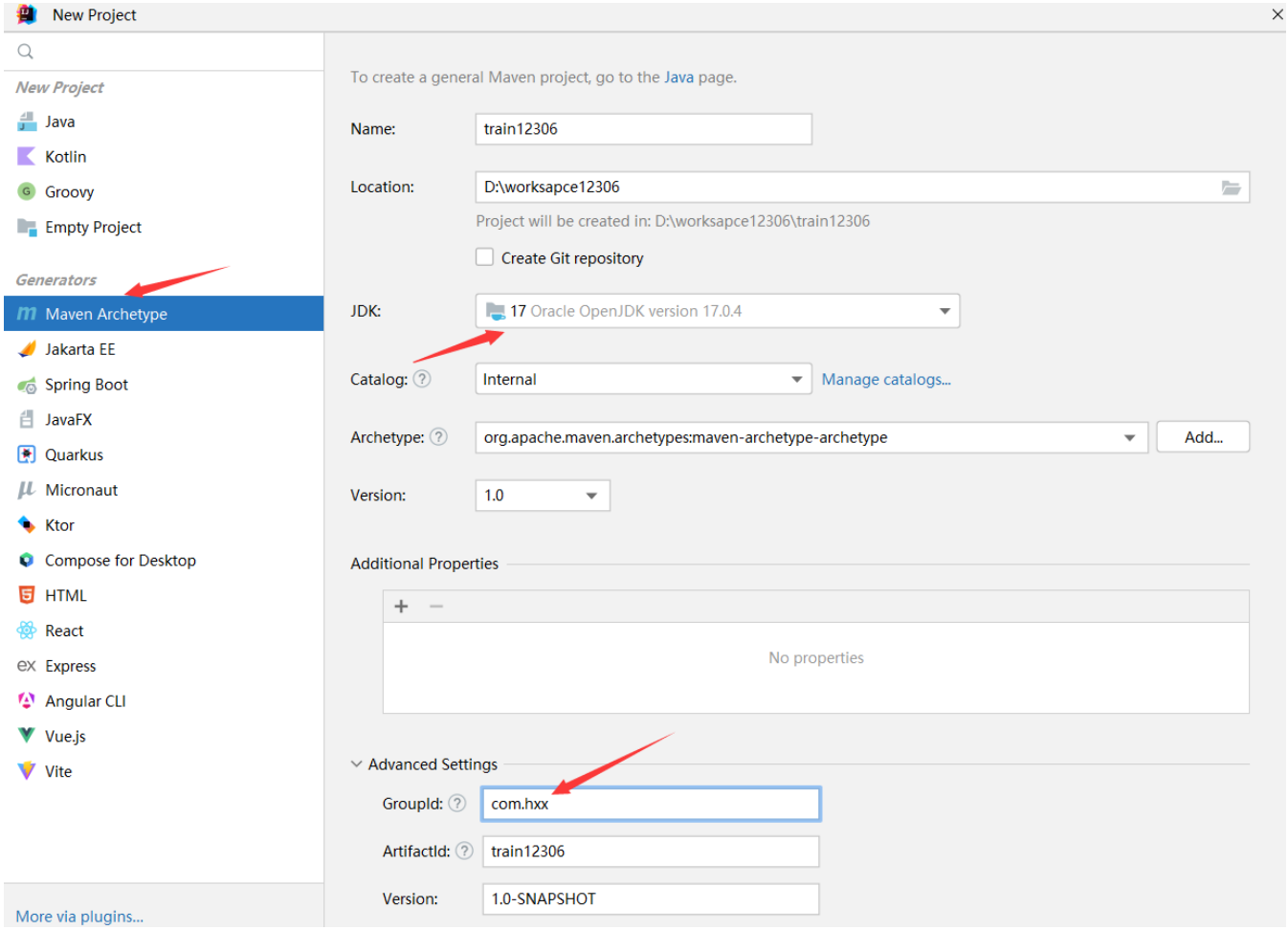
3.2 GraalVM代替JDK实现AOT

- GraalVM 是一个跨语言的通用虚拟机，不仅支持了 Java、Scala、Groovy、Kotlin 等基于 JVM 的语言，以及 C、C++ 等基于 LLVM 的语言，还支持其它像 JavaScript、Ruby、Rust、Python 和 R 语言等。
- GraalVM下载：https://github.com/graalvm/graalvm-ce-builds/releases/tag/vm-22.3.0

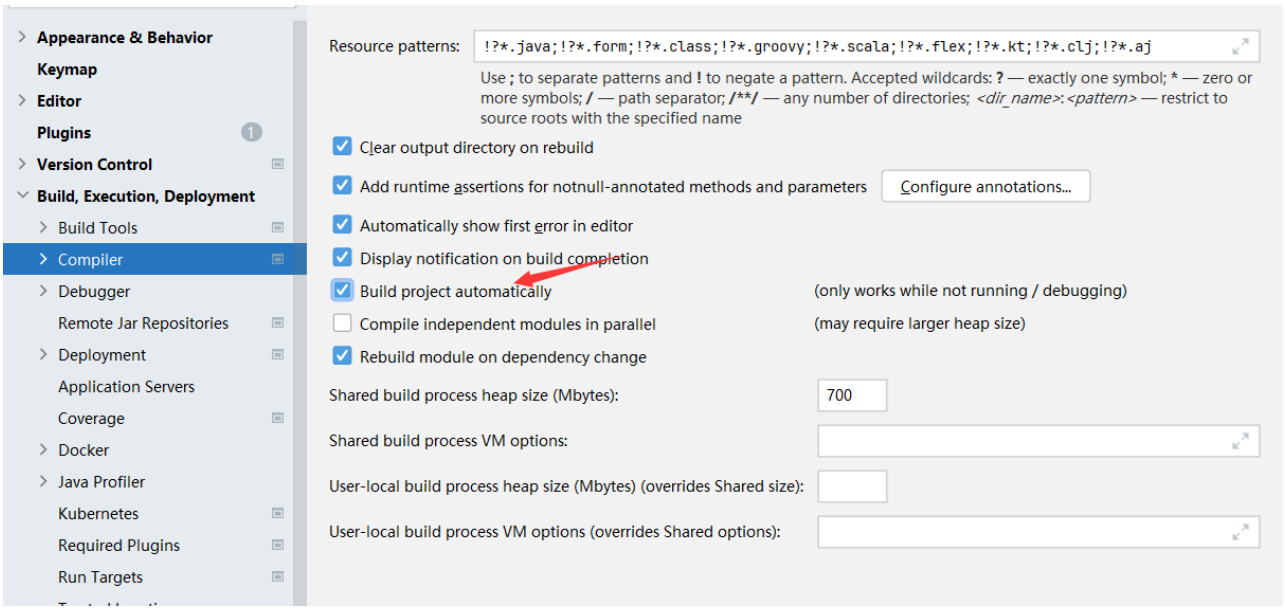


3.3 搭建Springboot3开发环境

3.3.1 环境搭建



- 配置热部署



3.4 搭建公共模块common

3.4.1 编写工具类

- 增加AOP打印请求参数和返回结果：完成对日志的输出
- pom

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
```

```
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.hxx</groupId>
        <artifactId>train12306</artifactId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <artifactId>train-common</artifactId>
    <name>Archetype - train-common</name>
    <url>http://maven.apache.org</url>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter</artifactId>
        </dependency>
        <dependency>
            <groupId>com.baomidou</groupId>
            <artifactId>mybatis-plus-boot-starter</artifactId>
        </dependency>
        <!-- 集成mysql连接 -->
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-aop</artifactId>
        </dependency>

        <dependency>
            <groupId>com.alibaba</groupId>
            <artifactId>fastjson</artifactId>
        </dependency>

        <dependency>
            <groupId>cn.hutool</groupId>
            <artifactId>hutool-all</artifactId>
        </dependency>

        <!-- 集成mysql连接 -->
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>

        <!--SpringBoot 2.4版本之后 SpringCloud 2020，需要引入该依赖，才能读取bootstrap文件-->
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-bootstrap</artifactId>
        </dependency>
```

```
        <!-- 配置中心 -->
        <dependency>
            <groupId>com.alibaba.cloud</groupId>
            <artifactId>spring-cloud-starter-alibaba-nacos-config</artifactId>
        </dependency>
        <!-- 注册中心 -->
        <dependency>
            <groupId>com.alibaba.cloud</groupId>
            <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
        </dependency>
    </dependencies>
</project>
```

- 工具类

```
package com.hxx.aspect;

import com.alibaba.fastjson.JSONObject;
import com.alibaba.fastjson.support.spring.PropertyPreFilters;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.http.HttpServletRequest;
import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.Signature;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.aspectj.lang.annotation.Pointcut;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.context.request.RequestContextHolder;
import org.springframework.web.context.request.ServletRequestAttributes;
import org.springframework.web.multipart.MultipartFile;

@Aspect
@Component
public class LogAspect {
    /**
     * 构造函数
     */
    public LogAspect() {
        System.out.println("Common LogAspect");
    }

    /**
     * 定义一个日志记录器
     */
    private final static Logger LOG = LoggerFactory.getLogger(LogAspect.class);

    /**
     * 定义一个切点
     */
    @Pointcut("execution(public * com.hxx..*Controller.*(..))")
    public void controllerPointcut() {
    }

    /**
     * 前置通知 用于拦截Controller层记录用户的操作的开始时间
     *
     * @param joinPoint 切点
     */
    @Before("controllerPointcut()")
    public void doBefore(JoinPoint joinPoint) {

        // 开始打印请求日志
        ServletRequestAttributes attributes = (ServletRequestAttributes)
RequestContextHolder.getRequestAttributes();
        HttpServletRequest request = attributes.getRequest();
```

```
        Signature signature = joinPoint.getSignature();
        String name = signature.getName();

        // 打印请求信息
        LOG.info("-------------- 开始 --------------");
        LOG.info("请求地址: {} {}", request.getRequestURL().toString(), request.getMethod());
        LOG.info("类名方法: {}.{}", signature.getDeclaringTypeName(), name);
        LOG.info("远程地址: {}", request.getRemoteAddr());

        // 打印请求参数
        Object[] args = joinPoint.getArgs();
        // LOG.info("请求参数: {}", JSONObject.toJSONString(args));

        // 排除特殊类型的参数，如文件类型
        Object[] arguments = new Object[args.length];
        for (int i = 0; i < args.length; i++) {
            if (args[i] instanceof ServletRequest
                    || args[i] instanceof ServletResponse
                    || args[i] instanceof MultipartFile) {
                continue;
            }
            arguments[i] = args[i];
        }
        // 排除字段，敏感字段或太长的字段不显示：身份证、手机号、邮箱、密码等
        String[] excludeProperties = {};
        // 用户可以在资源管理器中选择要筛选的属性，
        PropertyPreFilters filters = new PropertyPreFilters();
        //添加过滤器
        PropertyPreFilters.MySimplePropertyPreFilter excludefilter = filters.addFilter();
        excludefilter.addExcludes(excludeProperties);
        LOG.info("请求参数: {}", JSONObject.toJSONString(arguments, excludefilter));
    }

    /**
     * 环绕通知
     *
     * @param proceedingJoinPoint 切点
     */
    @Around("controllerPointcut()")
    public Object doAround(ProceedingJoinPoint proceedingJoinPoint) throws Throwable {
        long startTime = System.currentTimeMillis();
        //连接点
        Object result = proceedingJoinPoint.proceed();
        // 排除字段，敏感字段或太长的字段不显示：身份证、手机号、邮箱、密码等
        String[] excludeProperties = {};
        PropertyPreFilters filters = new PropertyPreFilters();
        PropertyPreFilters.MySimplePropertyPreFilter excludefilter = filters.addFilter();
        excludefilter.addExcludes(excludeProperties);
        LOG.info("返回结果: {}", JSONObject.toJSONString(result, excludefilter));
        LOG.info("-------------- 结束 耗时：{} ms --------------", System.currentTimeMillis() - startTime);
        return result;
    }
}
```

3.4.2 封装雪花算法

```
package com.hxx.util;

import cn.hutool.core.util.IdUtil;

/**
 * 工具类
 */
public class SnowUtil {
    // 数据中心、数据种子
    private static long dataCenterId = 1;
    // 机器id
    private static long machineId = 1;
    // 雪花算法
    public static long getSnowflakeId() {
```

```
        return IdUtil.getSnowflake().nextId();
    }
    // 雪花算法
    public static String getSnowflakeIdStr() {
        return IdUtil.getSnowflake().nextIdStr();
    }
}
```

3.4.3 前后端数据统一格式R

- 接口

```
public interface ResultCode {
    int SUCCESS = 200;
    int ERROR = 201;
}
```

- 实现类

```
package com.hxx.util;

import lombok.Data;

import java.util.HashMap;
import java.util.Map;

@Data
public class R {
    private Integer code;
    private String msg;
    private Map<String,Object> data = new HashMap<>();
    private Boolean SUCCESS;

    private R(){}

    public static R ok(){
        R r = new R();
        r.setSUCCESS(true);
        r.setCode(ResultCode.SUCCESS);
        r.setMsg("成功");
        return r;
    }

    public static R error(){
        R r = new R();
        r.setSUCCESS(false);
        r.setCode(ResultCode.ERROR);
        r.setMsg("失败");
        return r;
    }

    public R data(String key,Object value){
        this.data.put(key,value);
        return this;
    }

    public R data(Map<String,Object> map){
        this.setData(map);
        return this;
    }

    public R msg(String msg){
        this.setMsg(msg);
        return this;
    }

    public R code(Integer code){
        this.setCode(code);
        return this;
```

```
    }
}
```

3.4.4 分页拦截器

```
package com.hxx.util;

import com.baomidou.mybatisplus.annotation.DbType;
import com.baomidou.mybatisplus.extension.plugins.MybatisPlusInterceptor;
import com.baomidou.mybatisplus.extension.plugins.inner.PaginationInnerInterceptor;
import org.mybatis.spring.annotation.MapperScan;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class MybatisPlusConfig {

    /**
     * 添加分页插件
     */
    @Bean
    public MybatisPlusInterceptor mybatisPlusInterceptor() {
        MybatisPlusInterceptor interceptor = new MybatisPlusInterceptor();
        interceptor.addInnerInterceptor(new PaginationInnerInterceptor(DbType.MYSQL)); // 如果配置多个插件, 切
记分页最后添加
        return interceptor;
    }
}
```

3.4.5 日期自动填充

```
package com.hxx.util;

import com.baomidou.mybatisplus.core.handlers.MetaObjectHandler;
import org.apache.ibatis.reflection.MetaObject;
import org.springframework.stereotype.Component;

import java.util.Date;

@Component
public class MetaDataHandler implements MetaObjectHandler {
    @Override
    public void insertFill(MetaObject metaObject) {
        this.strictInsertFill(metaObject,"createTime", Date.class,new Date());
        this.strictInsertFill(metaObject,"updateTime", Date.class,new Date());
    }

    @Override
    public void updateFill(MetaObject metaObject) {
        this.strictUpdateFill(metaObject,"updateTime",Date.class,new Date());
    }
}
```

3.5 创建网关模块

- 模块：train-gateway
- pom依赖：bootstrap.yml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.hxx</groupId>
        <artifactId>train12306</artifactId>
```

```
        <version>1.0-SNAPSHOT</version>
    </parent>
    <artifactId>train-gateway</artifactId>
    <name>Archetype - train-gateway</name>
    <url>http://maven.apache.org</url>

    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-gateway</artifactId>
        </dependency>
        <dependency>
            <groupId>cn.hutool</groupId>
            <artifactId>hutool-all</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-bootstrap</artifactId>
        </dependency>
        <!-- 配置中心 -->
        <dependency>
            <groupId>com.alibaba.cloud</groupId>
            <artifactId>spring-cloud-starter-alibaba-nacos-config</artifactId>
        </dependency>
        <!-- 注册中心 -->
        <dependency>
            <groupId>com.alibaba.cloud</groupId>
            <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
        </dependency>
        <!--负载均衡-->
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-loadbalancer</artifactId>
        </dependency>
    </dependencies>
</project>
```

- 配置文件

```
server:
  port: 9999
spring:
  application:
    name: train-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
      globalcors:
        cors-configurations:
          '[/**]':
            allowedOriginPatterns: "*"
            allowedMethods: "*"
            allowedHeaders: "*"
            allowCredentials: true
            max-age: 3600
    nacos:
      config:
        server-addr: 127.0.0.1:8848
```

```
        username: nacos
        password: nacos
```

- 主配置类

```
package com.hxx;

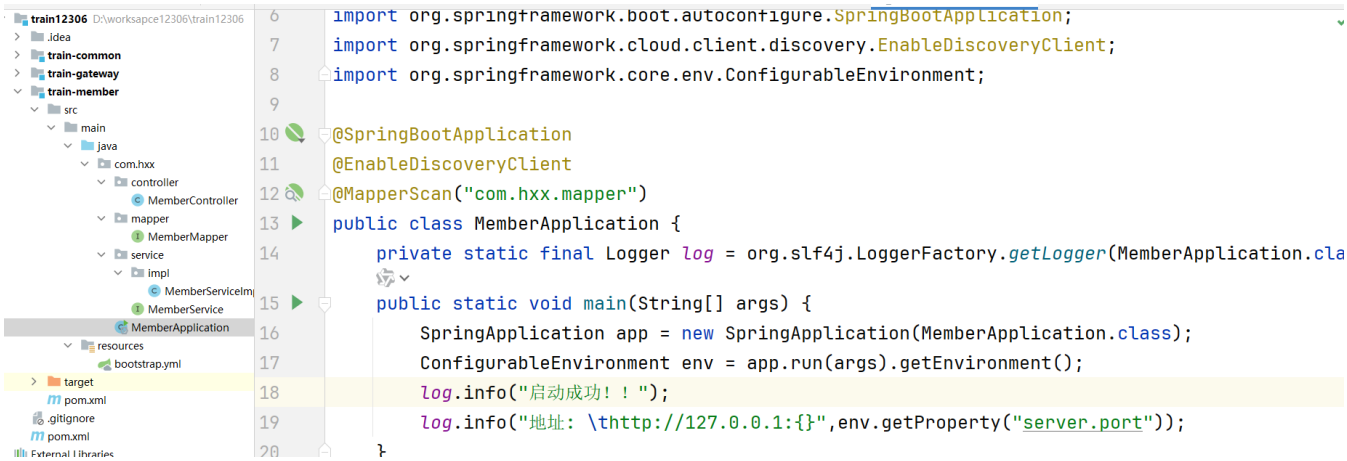
import org.slf4j.Logger;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
import org.springframework.core.env.ConfigurableEnvironment;

@SpringBootApplication
@EnableDiscoveryClient
public class GateWayApplication {

    private static final Logger LOGGER = org.slf4j.LoggerFactory.getLogger(GateWayApplication.class);
    public static void main(String[] args) {
        SpringApplication springApplication = new SpringApplication(GateWayApplication.class);
        ConfigurableEnvironment environment = springApplication.run(args).getEnvironment();
        LOGGER.info("网关启动成功！");
        LOGGER.info("网关地址: \t http://localhost:{}"+environment.getProperty("server.port"));
    }
}
```

四、搭建会员模块

4.1 环境搭建



- pom

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.hxx</groupId>
        <artifactId>train12306</artifactId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <artifactId>train-member</artifactId>
    <name>Archetype - train-member</name>
    <url>http://maven.apache.org</url>

    <dependencies>
        <dependency>
            <groupId>com.hxx</groupId>
            <artifactId>train-common</artifactId>
            <version>1.0-SNAPSHOT</version>
```

```
        </dependency>
    </dependencies>
</project>
```

- 配置文件：bootstrap.yml

```
server:
  port: 8001
spring:
  application:
    name: train-member
  cloud:
    nacos:
      config:
        server-addr: 127.0.0.1:8848
        username: nacos
        password: nacos
  datasource:
    url: jdbc:mysql://127.0.0.1:3306/train?useUnicode=true&characterEncoding=utf-
8&useSSL=false&serverTimezone=Asia/Shanghai
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: root
    password: root
```

4.2 功能编写

4.2.1 三层

- mapper

```
@Mapper
public interface MemberMapper extends BaseMapper<MemberEntity> {}
```

- service

```
public interface MemberService extends IService<MemberEntity> {

    Boolean login(String mobile);
}

-------------------------------

@Service
public class MemberServiceImpl extends ServiceImpl<MemberMapper, MemberEntity> implements MemberService
{

    @Autowired
    private  MemberMapper memberMapper;

    @Override
    public Boolean login(String mobile) {
        return memberMapper.selectOne(new LambdaQueryWrapper<MemberEntity>()
                .eq(MemberEntity::getMobile,mobile))!=null;
    }
}
```

- controller

```
@RestController
public class MemberController {
    @Autowired
    private MemberService memberService;

    private String code;

    //登录
```

```
    @GetMapping("/login")
    public R login(MemberInfo info){
        Boolean login = memberService.login(info.getMobile());
        if (login && code.equals(info.getCode())){
            return R.ok();
        }
        return R.error();
    }

    //随机生成验证码
    private String code(){
        String code = "";
        Random random = new Random();
        for (int i = 0; i < 4; i++) {
            code +=random.nextInt(10);
        }
        return code;
    }

    //发送验证码
    @GetMapping("/sendCode")
    public R sendCode(String mobile){
        code = code();
        return R.ok().data("code", code);
    }
}
```

五、前端vue3

5.1 环境搭建

5.1.1 nodejs

- nodejs>=18的版本

```
C:\>node -v
v18.16.0
```

5.1.2 搭建

```
pnpm create vite train12306Front
```

```
 added 1, done
? Select a framework: » - Use arrow-keys. Return to submit.
    Vanilla
>   Vue
    React
 added 1, done
√ Select a framework: » Vue
? Select a variant: » - Use arrow-keys. Return to submit.
>   TypeScript
    JavaScript
    Customize with create-vue ↗
    Nuxt ↗
```

- 安装依赖

```
pnpm install
```

5.2 安装AntDesign组件库

5.2.1 简介



5.2.2 安装组件



5.2.3 项目中main.ts引入



- 引入icon图标组件



- main.ts引入

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import Antd from 'ant-design-vue'
import 'ant-design-vue/dist/antd.css';
import * as Icons from '@ant-design/icons-vue'
const app = createApp(App)
//加载
app.use(Antd).use(router).mount('#app')
//图标
const icons:any = Icons
for(const i in icons){
    app.component(i,icons[i])
}
```

- 安装axios

```
pnpm install axios
```

- 编写main.ts拦截

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index'
import store from './store/index'
import Antd from 'ant-design-vue'
import 'ant-design-vue/dist/reset.css';
import * as Icons from '@ant-design/icons-vue'
import axios from 'axios'


const app = createApp(App)
//加载
app.use(Antd).use(router).mount('#app')
//图标
const icons:any = Icons
for(const i in icons){
    app.component(i,icons[i])
}

//请求拦截
axios.interceptors.request.use(config=>{
    return config;
},err=>{
    return Promise.reject(err);
})
//响应拦截
axios.interceptors.response.use(res=>{
    return res;
},err=>{
    return Promise.reject(err);
})
```

- 安装路由

```
pnpm install vue-router
```

- 安装vuex

```
pnpm install vuex
```

- 配置开发环境:根目录下



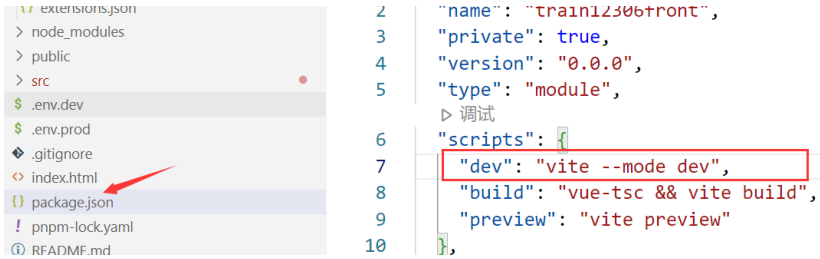
- .env.dev

```
VITE_ENV='dev'
VITE_APP_BASE_API='http://localhost:7000'
```

- .env.prod

```
VITE_ENV='prod'
VITE_APP_BASE_API='http://localhost:7000'
```

- package.json配置开发模式



- main.ts配置开发访问地址

```
//配置访问地址
axios.defaults.baseURL = import.meta.env.VITE_APP_BASE_API
console.log('环境: ',import.meta.env.VITE_ENV);
console.log('服务端: ',import.meta.env.VITE_APP_BASE_API);
```

5.2.4 优化配置

- 配置@别名

```
pnpm install path
pnpm install -D @types/node
```

- 配置vite.config.ts

```
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import path from 'path'

// https://vitejs.dev/config/
export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname,'src'),
    },
  },
})
```

- tsconfig.ts

```
"compilerOptions": {
    ...
    "baseUrl":".",
    "paths":{
        "@/*":["src/*"]
    }
}
```

5.3 编写页面

5.3.1 登录Login.vue




```
<template>
    <a-row class="login">
      <a-col :span="8" :offset="8" class="login-main">
        <h1 style="text-align: center"><rocket-two-tone /> 12306售票系统</h1>
        <a-form
            :model="loginForm"
            name="basic"
            autocomplete="off"
        >
          <a-form-item
              label=""
              name="mobile"
              :rules="[{ required: true, message: '请输入手机号!' }]"
          >
            <a-input v-model:value="loginForm.mobile" placeholder="手机号"/>
          </a-form-item>

          <a-form-item
              label=""
              name="code"
              :rules="[{ required: true, message: '请输入验证码!' }]"
          >
            <a-input v-model:value="loginForm.code">
              <template #addonAfter>
                <a @click="sendCode">获取验证码</a>
              </template>
            </a-input>
            <!--<a-input v-model:value="loginForm.code" placeholder="验证码"/>-->
          </a-form-item>

          <a-form-item>
            <a-button type="primary" block @click="login">登录</a-button>
          </a-form-item>

        </a-form>
```

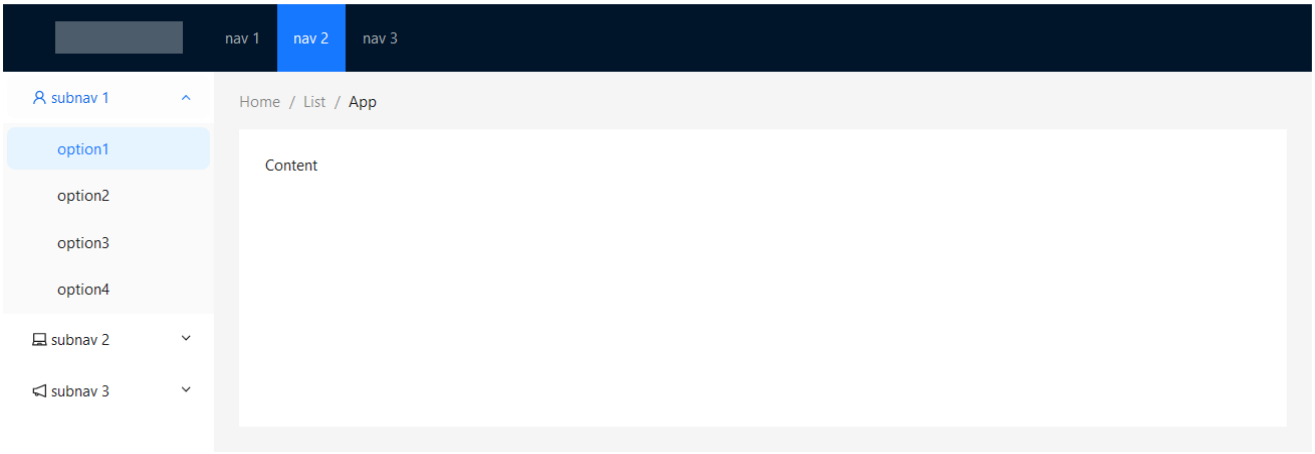
```
    </a-col>
  </a-row>
</template>
<script setup lang='ts'>
import { reactive } from 'vue';
import axios from 'axios';
import { notification } from 'ant-design-vue';
import { useRouter } from 'vue-router'
import store from "@/store";

//创建路由
const router = useRouter();
//参数
const loginForm = reactive({
  mobile: '13936439780',
  code: '',
});
//获取验证码
const sendCode = () => {
  axios({
    url: "/trainMember/sendCode",
    method: 'get',
    params: {mobile: loginForm.mobile}
  }).then(response => {
    let data = response.data;
    if (data) {
      notification.success({ message: '发送验证码成功！' });
      loginForm.code = data.data.code;
    } else {
      notification.error({ message: data.message });
    }
  });
};

//登录
const login = () => {
  //请求后端
  axios.post("/trainMember/login", loginForm).then(response => {
    //获取数据信息
    let data = response.data;
    console.log('------------>>>',data);
    if (data.success) {
        //保存会员id
        sessionStorage.setItem('memberId', data.member.id);
        notification.success({ message: '登录成功！' });
        //跳转主页面
        router.push('/main');
    } else {
      notification.error({ message: data.message });
    }
  })
}
</script>

<style>
.login-main h1 {
  font-size: 25px;
  font-weight: bold;
}
.login-main {
  margin-top: 100px;
  padding: 30px 30px 20px;
  border: 2px solid grey;
  border-radius: 10px;
  background-color: #fcfcfc;
}
</style>
```

5.3.2 主页面TrainMain.vue



5.3.3 编写路由文件和store文件

- store/index.ts

```
import { createStore } from 'vuex'
const MEMBER = 'MEMBER'
//vuex存储状态数据
export default createStore({
    state: {
        member: window.sessionStorage.getItem(MEMBER) || null,
    },
    getters: {},
    mutations: {//提交修改状态数据的方法
        setMember(state: any, member: any) {
            state.member = member;
            window.sessionStorage.setItem(MEMBER, member);
        }
    },
    actions: {}
})
```

- 路由文件：router/index.ts

```
import {createRouter,createWebHistory,RouteRecordRaw} from 'vue-router'
import { notification } from 'ant-design-vue'
import Login from '@/views/Login.vue'
import store from '@/store'

const routes:Array<RouteRecordRaw> = [
    {
        path:'/',
        component: Login
    },
    {
        path:'/main',
        component: () => import('@/views/TrainMain.vue'),//懒加载方式
        children:[
            {
                path:'/login',
                redirect:'/'
            },
        ]
    },

]

const router = createRouter({
    history: createWebHistory(import.meta.env.BASE_URL),
    routes,
})
```

```
//路由守卫
router.beforeEach((to,from,next)=>{
    if(to.matched.some(item=>{
      return item.meta.loginRequire
    })){
      let _member = store.state.member
      if(!_member.token){
        notification.error({message:'没有登录或者超时'})
        next('/')
      }else{
        next() //通过
      }
    }else{
      next() //通过
    }
  })

  export default router
```

- App.vue添加路由显示标签

```
<template>
  <div>
    <router-view />
  </div>
</template>
```

5.4 编写公共组件

- 主页面：拆分为header、silder、main三个部分



5.4.1 编写TrainHeader.vue

```
<template>
  <a-layout-header class="header">
    <div class="logo">
      <RouterLink to="/main/hxx">
        <img :src="hxx" style="width: 20px;height: 20px">12306火车售票
      </RouterLink>
    </div>
    <div style="float: right; color: white;">
```

```
      欢迎使用管理控台
    </div>
    <a-menu
      v-model:selectedKeys="selectedKeys"
      theme="dark"
      mode="horizontal"
      :style="{ lineHeight: '64px' }"
    >
      <a-menu-item key="/main/welcome">
        <RouterLink to="/main/welcome">
           <coffee-outlined />   欢迎
        </RouterLink>
      </a-menu-item>
      <a-menu-item key="/main/about">
        <RouterLink to="/main/about">
           <user-outlined />   关于
        </RouterLink>
      </a-menu-item>
    </a-menu>
  </a-layout-header>
</template>
<script lang="ts" setup>
import router from '@/router';
import { ref,watch } from 'vue';

//引入图标
const hxx = ref('/hxx.jpg')
//被选中的菜单项
const selectedKeys = ref<string[]>([]);

//监听路由变化，改变菜单项
watch(()=>router.currentRoute.value.path,(newValue:any,oldalue:any) => {
    console.log('---------->>',newValue,oldalue)
    selectedKeys.value = []
    selectedKeys.value.push(newValue)
}, { immediate: true })
</script>
<style scoped>
.logo {
  float: left;
  height: 31px;
  width: 150px;
  color: white;
  font-size: 20px;
}
</style>
```

5.4.2 编写TrainSilder.vue

```
<template>
  <a-layout-sider width="200" style="background: #fff">
    <a-menu v-model:selectedKeys="selectedKeys" mode="inline" :style="{ height: '100%', borderRight: 0 }">
      <a-menu-item key="/main/welcome">
        <router-link to="/main/welcome">
          <coffee-outlined />   欢迎
        </router-link>
      </a-menu-item>
      <a-menu-item key="/main/about">
        <router-link to="/main/about">
          <TeamOutlined />   关于
        </router-link>
      </a-menu-item>
      <a-sub-menu key="member">
        <template #title>
          <span>
            <UnorderedListOutlined />
            会员管理
          </span>
        </template>
        <a-menu-item key="/main/passenger">
```

```
        <router-link to="/main/passenger">
            <UserDeleteOutlined />乘车人管理
        </router-link>
      </a-menu-item>
      <a-menu-item key="/main/trainDicker">
        <router-link to="/main/trainDicker">
          <UserDeleteOutlined />余票查询
        </router-link>
      </a-menu-item>
      <a-menu-item key="/main/myConfirmOrder">
        <router-link to="/main/myConfirmOrder">
          <UserDeleteOutlined />我的车票
        </router-link>
      </a-menu-item>
      <a-menu-item key="3"><UserDeleteOutlined />控台管理</a-menu-item>
    </a-sub-menu>
    <a-sub-menu key="business">
      <template #title>
        <span>
          <UnorderedListOutlined />
          业务管理
        </span>
      </template>
      <a-menu-item key="/daily/confirmOrder">
        <router-link to="/daily/confirmOrder">
            <UserDeleteOutlined />订单信息
        </router-link>
      </a-menu-item>
      <a-menu-item key="/daily/dailyTrainDicket">
          <router-link to="/daily/dailyTrainDicket">
              <UserDeleteOutlined />余票查询
          </router-link>
      </a-menu-item>
      <a-menu-item key="/daily/dailyTrain">
          <router-link to="/daily/dailyTrain">
              <UserDeleteOutlined />每日车次
          </router-link>
      </a-menu-item>
      <a-menu-item key="/daily/dailyTrainStation">
        <router-link to="/daily/dailyTrainStation">
          <UserDeleteOutlined />每日车站
        </router-link>
      </a-menu-item>
      <a-menu-item key="/daily/dailyTrainCarriage">
        <router-link to="/daily/dailyTrainCarriage">
          <UserDeleteOutlined />每日车厢
        </router-link>
      </a-menu-item>
      <a-menu-item key="/daily/dailyTrainSeat">
        <router-link to="/daily/dailyTrainSeat">
          <UserDeleteOutlined />每日座位
        </router-link>
      </a-menu-item>
    </a-sub-menu>
    <a-sub-menu key="base">
      <template #title>
        <span>
          <UnorderedListOutlined />
          基础数据
        </span>
      </template>
      <a-menu-item key="/main/station">
        <router-link to="/main/station">
          <UserDeleteOutlined />车站管理
        </router-link>
      </a-menu-item>
      <a-menu-item key="/main/train">
        <router-link to="/main/train">
          <UserDeleteOutlined />火车管理
        </router-link>
      </a-menu-item>
      <a-menu-item key="/main/trainStation">
```

```
          <router-link to="/main/trainStation">
            <UserDeleteOutlined />火车车站
          </router-link>
        </a-menu-item>
        <a-menu-item key="/main/trainCarriage">
          <router-link to="/main/trainCarriage">
            <UserDeleteOutlined />火车车厢
          </router-link>
        </a-menu-item>
        <a-menu-item key="/main/trainSeat">
          <router-link to="/main/trainSeat">
            <UserDeleteOutlined />火车座位
          </router-link>
        </a-menu-item>
      </a-sub-menu>
      <a-sub-menu key="batch">
        <template #title>
          <span>
            <UnorderedListOutlined />
            跑批管理
          </span>
        </template>
        <a-menu-item key="/batch/job">
          <router-link to="/batch/job">
            <user-outlined />   任务管理
          </router-link>
        </a-menu-item>
      </a-sub-menu>
    </a-menu>
  </a-layout-sider>
</template>

<script setup lang="ts">
import {ref, watch} from 'vue';
import router from "@/router";

  const selectedKeys = ref<string[]>([]);

  watch(() => router.currentRoute.value.path, (newValue) => {
    selectedKeys.value = [];
    selectedKeys.value.push(newValue);
  }, {immediate: true});
</script>

<!-- Add "scoped" attribute to limit CSS to this component only -->
<style scoped>

</style>
```

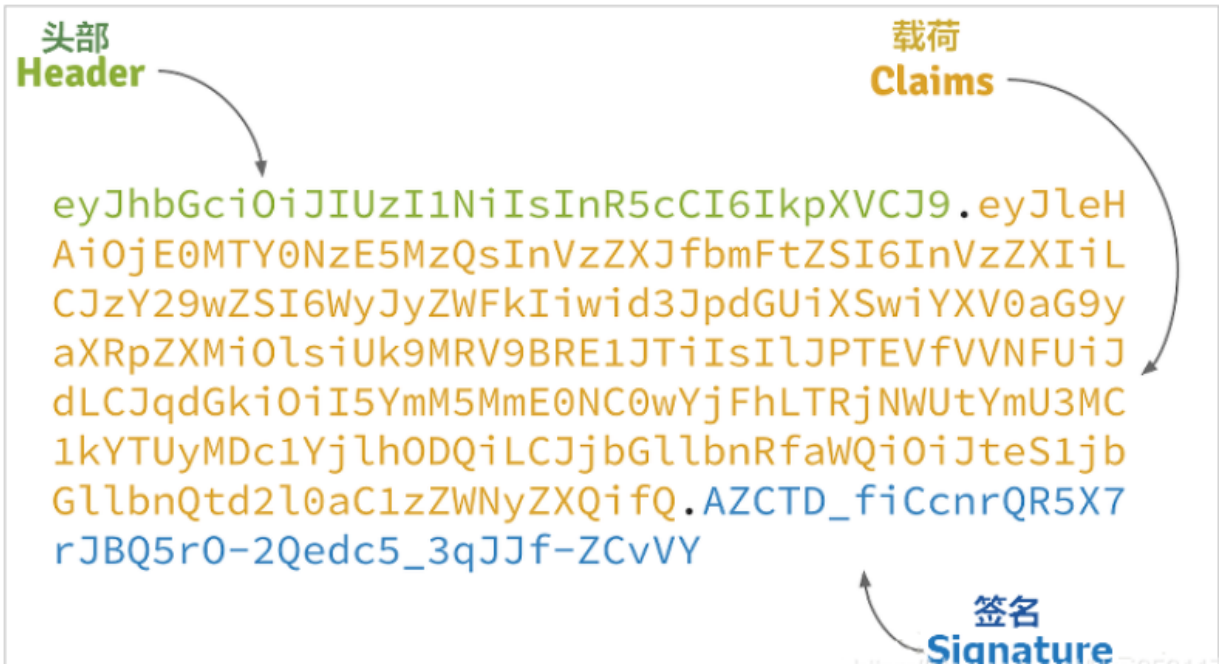
5.4.3 编写TrainMain.vue

```
<template>
    <a-layout>
      <train-header />
      <a-layout>
        <train-silder />
          <a-layout-content :style="{ background: '#fff', padding: '24px', margin: 0, minHeight: '280px' }">
            <router-view />
          </a-layout-content>
      </a-layout>
    </a-layout>
  </template>
  <script lang="ts" setup>
  import TrainHeader from '@/components/TrainHeader.vue';
  import TrainSilder from '@/components/TrainSilder.vue';
  </script>
  <style scoped>
  </style>
```

六、JWT登录验证

6.1 简介

- JWT，全称是Json Web Token， 是JSON风格轻量级的授权和身份认证规范，可实现无状态、分布式的Web应用授权；
- 数据格式：
  - JWT包含三部分数据：
    - Header：头部，通常头部有两部分内容：
      - 声明类型，这里是JWT
      - 签名算法，自定义
      - 我们会对头部进行base64加密（可解密），得到第一部分数据
    - Payload：载荷，就是有效数据，一般包含下面信息：
      - 用户身份信息（注意，这里因为采用base64加密，可解密，因此不要存放敏感信息）
      - tokenID：当前这个JWT的唯一标示
      - 注册声明：如token的签发时间，过期时间，签发人等
      - 这部分也会采用base64加密，得到第二部分数据
    - Signature：签名，是整个数据的认证信息。一般根据前两步的数据，再加上服务的的密钥（secret）（不要泄漏，最好周期性更换），通过加密算法生成。用于验证整个数据完整和可靠性



- 加密方式：
  - 对称加密：AES
    - 基本原理：将明文分成N个组，然后使用密钥对各个组进行加密，形成各自的密文，最后把所有的分组密文进行合并，形成最终的密文。
    - 优势：算法公开、计算量小、加密速度快、加密效率高
    - 缺陷：**双方都使用同样密钥**，安全性得不到保证
  - 非对称加密：RSA
    - 基本原理：同时生成两把密钥：私钥和公钥，钥隐秘保存，公钥可以下发给信任客户端
      - 私钥加密，持有私钥或公钥才可以解密
      - 公钥加密，持有私钥才可解密
    - 优点：安全，难以破解
    - 缺点：算法比较耗时
  - 不可逆加密，如MD5，SHA
    - 基本原理：加密过程中不需要使用密钥，输入明文后由系统直接经过加密算法处理成密文，这种加密后的数据是无法被解密的，无法根据密文推算出明文

6.2 在common模块中编写jwt工具类

```
package com.hxx.util;
```

```java
import cn.hutool.core.date.DateField;
import cn.hutool.core.date.DateTime;
import cn.hutool.json.JSONObject;
import cn.hutool.jwt.JWT;
import cn.hutool.jwt.JWTPayload;
import cn.hutool.jwt.JWTUtil;

import java.time.LocalDateTime;
import java.util.HashMap;

/**
 * jwt工具类
 */
public class JwtUtils {

    //秘钥
    private static final String JWT_SECRET = "hxx";

    /**
     * 创建JWT令牌
     *
     * @param id 用户ID，用于在令牌中标识用户
     * @param mobile 用户手机号，可用于额外的身份验证
     * @return 生成的JWT令牌字符串
     *
     * 本函数用于根据给定的用户ID和手机号创建一个JWT令牌。
     * 令牌中包含发行时间、过期时间、生效时间和用户ID、手机号等信息。
     * 使用预定义的JWT密钥对令牌进行签名，确保其安全性。
     */
    //创建token
    public static String createToken(Long id,String mobile){
        // 获取当前时间
        DateTime now = new DateTime();
        // 计算令牌的过期时间，为当前时间后24小时
        DateTime expTime = now.offsetNew(DateField.HOUR, 24);
        // 初始化令牌的载荷信息
        HashMap<String, Object> payload = new HashMap<>();
        // 设置令牌的发行时间
        payload.put(JWTPayload.ISSUED_AT,now);
        // 设置令牌的过期时间
        payload.put(JWTPayload.EXPIRES_AT,expTime);
        // 设置令牌的生效时间，为当前时间
        payload.put(JWTPayload.NOT_BEFORE,now);
        // 设置用户ID和手机号作为自定义载荷信息
        payload.put("id",id);
        payload.put("mobile",mobile);
        // 使用载荷信息和密钥生成JWT令牌
        return JWTUtil.createToken(payload,JWT_SECRET.getBytes());
    }

    /**
     * 验证JWT令牌的有效性。
     *
     * 本函数用于解析并验证一个JWT令牌的合法性。它首先使用JWTUtil解析令牌，然后设置秘钥，
     * 最后通过调用JWT的validate方法，检查令牌是否在有效期内。
     * 注意，这里将验证过期时间设为0，意味着只检查签名有效性，不检查实际过期时间。
     *
     * @param token 待验证的JWT令牌字符串。
     * @return 如果令牌有效（签名验证通过），返回true；否则返回false。
     */
    public static boolean verify(String token){
        JWT jwt = JWTUtil.parseToken(token).setKey(JWT_SECRET.getBytes());
        return jwt.validate(0);
    }

    /**
     * 从JWT令牌中提取声明信息。
     *
     * 该方法用于解析JWT令牌，并从中移除与时间相关的声明，返回剩余的声明信息。
     * 时间相关的声明包括：发行时间、过期时间和生效时间。
     * 这样做的目的是为了保护敏感信息的隐私或减少信息泄露的风险。
```

```
     *
     * @param token 待解析的JWT令牌字符串。
     * @return 包含JWT令牌中除时间声明外的所有声明的JSONObject。
     */
    public static JSONObject getClaims(String token){
        // 解析JWT令牌并设置解密密钥
        JWT jwt = JWTUtil.parseToken(token).setKey(JWT_SECRET.getBytes());
        // 获取JWT令牌的载荷部分
        JSONObject payloads = jwt.getPayloads();
        // 移除时间相关的声明
        payloads.remove(JWTPayload.ISSUED_AT);
        payloads.remove(JWTPayload.EXPIRES_AT);
        payloads.remove(JWTPayload.NOT_BEFORE);
        // 返回处理后的载荷信息
        return payloads;
    }
}
```

6.2 在会员中登录使用jwt

```
//登录
@PostMapping("/login")
public R login(@RequestBody MemberInfo info){
    MemberEntity login = memberService.login(info.getMobile());
    if (login!=null && code.equals(info.getCode())){
        return R.ok().data("member",login)//添加令牌
                .data("token", JwtUtils.createToken(info.getId(),info.getMobile()));
    }
    return R.error();
}
```

6.3 前端登录获取jwt

6.3.1 Login.vue登录

```
//登录
const login = () => {
    //请求后端
    axios.post("/trainMember/login", loginForm).then(({data}) => {
        if (data.success) {
            //保存会员id
            sessionStorage.setItem('memberId', data.data.member.id);
            notification.success({ message: '登录成功！' });
            //jwt
            store.commit('setMember', data.data.token);
            //跳转主页面
            router.push('/main');
        } else {
            notification.error({ message: data.message });
        }
    })
}
```

6.3.2 main.ts编写拦截器

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index'
import store from './store/index'
import Antd, { message } from 'ant-design-vue'
import 'ant-design-vue/dist/reset.css';
import * as Icons from '@ant-design/icons-vue'
import axios from 'axios'


const app = createApp(App)
//加载
```

```
app.use(Antd).use(store).use(router).mount('#app')
//图标
const icons:any = Icons
for(const i in icons){
    app.component(i,icons[i])
}

//请求拦截
axios.interceptors.request.use(config=>{
    //请求头携带token
    if(store.state.member!=null){
        const _token = store.state.member;
        if(_token){
            config.headers.token = _token;
        }
    }
    return config;
},err=>{
    return Promise.reject(err);
})
//响应拦截
axios.interceptors.response.use(res=>{
    return res;
},err=>{
    const _response = err.response;
    const status = _response.status;
    if(status==401){
        //未登录
        store.commit('setMember',null);
        message.error('未登录,请重新登录');
        router.push('/login');
    }
    return Promise.reject(err);
})

//配置访问地址
axios.defaults.baseURL = import.meta.env.VITE_APP_BASE_API
console.log('环境: ',import.meta.env.VITE_ENV);
console.log('服务端: ',import.meta.env.VITE_APP_BASE_API);
```

6.4 后端编写拦截器：gateway模块

```
@Component
public class LoginFilter implements Ordered, GlobalFilter {
    /**
     * 自定义网关过滤器，用于权限验证。
     * 在请求通过网关路由到具体服务之前，此过滤器会检查请求是否包含有效的token。
     * 指定路径的请求将绕过token验证。
     *
     * @param exchange 服务器web交换对象，包含请求和响应信息。
     * @param chain 过滤器链，用于继续或停止过滤器链的执行。
     * @return Mono<Void> 表示异步操作的结果。
     */
    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        // 获取请求路径
        String path = exchange.getRequest().getURI().getPath();

        // 指定路径的请求不需要token验证
        // 放行
        if (path.contains("/admin") ||
                path.contains("/trainMember/login") ||
                path.contains("/trainMember/sendCode") ){
            return chain.filter(exchange);
        }

        // 从请求头中获取token
        String token = exchange.getRequest().getHeaders().getFirst("token");

        // 如果token不存在或为空，则返回未授权状态
```

```
        if (token==null || token.isEmpty()) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        // 验证token的有效性
        if (JwtUtils.verify(token)){
            return chain.filter(exchange);
        }else {
            // 如果token无效，则返回未授权状态
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
    }
    /**
     * 获取当前对象的顺序。
     *
     * 本方法重写了父类中的getOrder方法，返回值为0，表示当前对象的执行顺序。
     * 在使用注解处理器时，顺序通常很重要，因为它决定了处理程序的执行序列。
     * 返回0意味着这个处理程序应该在所有其他处理程序之前执行。
     *
     * @return 返回0，表示当前对象的执行顺序。
     */
    @Override
    public int getOrder() {
        return 0;
    }
}
```

七、乘车人管理（member模块）

7.1 功能编写

7.1.1 数据表passenger

```
CREATE TABLE `passenger` (
  `id` bigint NOT NULL COMMENT 'id',
  `member_id` bigint NOT NULL COMMENT '会员id',
  `name` varchar(20) NOT NULL COMMENT '姓名',
  `id_card` varchar(18) NOT NULL COMMENT '身份证',
  `type` char(1) NOT NULL COMMENT '旅客类型|枚举[PassengerTypeEnum]',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  KEY `member_id_index` (`member_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='乘车人';
```

7.1.2 编写枚举类

```
package com.hxx.enums;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

public enum PassengerTypeEnum {

    ADULT(1,"成人"),
    CHILD(2,"儿童"),
    STUDENT(3,"学生");

    private Integer code;
    private String desc;

    PassengerTypeEnum(Integer code, String desc) {
        this.code = code;
        this.desc = desc;
```

```
    }

    public Integer getCode() {
        return code;
    }

    public void setCode(Integer code) {
        this.code = code;
    }

    public String getDesc() {
        return desc;
    }

    public void setDesc(String desc) {
        this.desc = desc;
    }
    /**
     * 获取乘客类型的枚举列表。
     * 该方法通过遍历PassengerTypeEnum枚举类的所有值，将每个值的code和desc存入HashMap中，
     * 然后将这些HashMap添加到一个List中，最后返回这个List。这样做的目的是为了以一种更方便的数据结构
     * （List<HashMap<String,Object>>）来存储和访问乘客类型的代码和描述。
     *
     * @return List<HashMap<String,Object>> 包含乘客类型代码和描述的列表。
     */
    public static List<HashMap<String,Object>> getEnumList(){
        // 获取PassengerTypeEnum枚举类的所有值
        PassengerTypeEnum[] values = PassengerTypeEnum.values();
        // 初始化一个List来存储HashMap
        List<HashMap<String,Object>> list = new ArrayList<>();
        // 遍历枚举值
        for (PassengerTypeEnum passengerTypeEnum : values) {
            // 初始化一个HashMap来存储每个枚举值的code和desc
            HashMap<String,Object> map = new HashMap<>();
            // 将枚举值的code和desc存入HashMap
            map.put("code",passengerTypeEnum.getCode());
            map.put("desc",passengerTypeEnum.getDesc());
            // 将HashMap添加到List中
            list.add(map);
        }
        // 返回包含所有枚举值的HashMap的List
        return list;
    }
}
```

7.1.3 三层编写

- mapper

```
@Mapper
public interface PassengerMapper extends BaseMapper<PassengerEntity> {}
```

- service

```
public interface PassengerService extends IService<PassengerEntity> {
    Page<PassengerEntity> getPassengerList(Page<PassengerEntity> page,
                                           PassengerEntity passengerEntity);
}
```

- serviceImpl

```
@Service
public class PassengerServiceImpl extends ServiceImpl<PassengerMapper, PassengerEntity>
        implements PassengerService {
    @Autowired
    private PassengerMapper passengerMapper;
    @Override
    public Page<PassengerEntity> getPassengerList(Page<PassengerEntity> page,
```

```
                                                PassengerEntity passengerEntity) {
        return passengerMapper.selectPage(page,
                                          new LambdaQueryWrapper<PassengerEntity>()
                .eq(passengerEntity.getName()!=null,
                    PassengerEntity::getName,passengerEntity.getName())
                .eq(passengerEntity.getType()!=null,
                    PassengerEntity::getType,passengerEntity.getType())
        );
    }
}
```

- controller

```
@RestController
@RequestMapping("/passenger")
public class PassengerController {
    @Autowired
    private PassengerService passengerService;

    @PostMapping("/add")
    public R add(@RequestBody PassengerEntity passengerEntity){
        passengerService.save(passengerEntity);
        return R.ok();
    }

    @PutMapping("/update")
    public R update(@RequestBody PassengerEntity passengerEntity){
        passengerService.updateById(passengerEntity);
        return R.ok();
    }
    @DeleteMapping("/delete")
    public R delete(Long id){
        passengerService.removeById(id);
        return R.ok();
    }
    @GetMapping("/page")
    public R getPassengerList(Page page, PassengerEntity passengerEntity){
        return R.ok().data("list",passengerService.getPassengerList(page,passengerEntity));
    }

    @GetMapping("/show")
    public R show(){
        return R.ok().data("list",passengerService.list());
    }
}
```

7.2 common模块编写线程工具类

7.2.1 线程类

```
package com.hxx.context;

import com.hxx.dto.MemberInfo;

//会员信息存储线程中，从线程中获取会员信息
public class LoginMemberContext {

    private static final ThreadLocal<MemberInfo> MEMBER_THREAD_LOCAL = new ThreadLocal<>();

    public static void setMember(MemberInfo memberEntity) {
        MEMBER_THREAD_LOCAL.set(memberEntity);
    }

    public static MemberInfo getMember() {
        return MEMBER_THREAD_LOCAL.get();
    }

    public static void removeMember() {
        MEMBER_THREAD_LOCAL.remove();
```

```
    }
    //获取会员id
    public static Long getMemberId() {
        return MEMBER_THREAD_LOCAL.get().getId();
    }
}
```

7.2.2 编写拦截器获取会员信息

```
@Component
public class MemberInterceptor implements HandlerInterceptor {
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler)
        throws Exception {
        String token = request.getHeader("token");
        if (StrUtil.isNotBlank(token) && !"null".equals(token)) {
            JSONObject claims = JwtUtils.getClaims(token);
            MemberInfo memberInfo = JSONUtil.toBean(claims, MemberInfo.class);
            LoginMemberContext.setMember(memberInfo);
        }
        return true;
    }
}
```

7.3 在member模块使用拦截器

```
@Configuration
public class SpringMvcConfig implements WebMvcConfigurer {
    // 注入拦截器
    @Resource
    private MemberInterceptor memberInterceptor;

    /**
     * 添加拦截器到拦截器注册表中。
     *
     * 此方法用于配置拦截器，使其拦截所有请求，除了特定的登录和发送验证码接口。
     * 通过拦截器，可以实现一些全局的功能或逻辑，比如权限验证、日志记录等。
     *
     * @param registry 拦截器注册表，用于添加和管理拦截器。
     */
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        // 添加memberInterceptor拦截器，并配置拦截所有请求
        registry.addInterceptor(memberInterceptor)
                // 指定拦截器应该拦截的路径模式，这里设置为拦截所有路径
                .addPathPatterns("/**")
                // 排除登录和发送验证码接口，这些接口不需要被拦截器拦截
                .excludePathPatterns("/trainMember/login","/trainMember/sendCode");
    }
}
```

7.4 乘车人：前端管理

- 编写分页组件，在main.ts中作为全局组件

index.ts 1 | TrainPassenger.vue | main.ts | TrainHeader.vue

src > main.ts > ...

```ts
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index'
import store from './store/index'
import Antd, { message } from 'ant-design-vue'
import 'ant-design-vue/dist/reset.css';
import * as Icons from '@ant-design/icons-vue'
import axios from 'axios'
import TrainPagination from '@/components/TrainPagination.vue';

const app = createApp(App)
//全局挂载
app.component('TrainPagination', TrainPagination)
//加载
app.use(Antd).use(store).use(router).mount('#app')
//图标
```

7.4.1 显示：TrainPassenger.vue




```vue
<template>
    <p>
      <a-space>
        <a-button type="primary" @click="refreshData">刷新</a-button>
        <a-button type="primary" @click="add">新增</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{column,record}">
         <template v-if="column.dataIndex==='type'">
            <span v-for="item in PASSENGER_TYPE_ARRAY" :key="item.code">
                <span v-if="item.code===record.type">{{ item.desc }}</span>
            </span>
         </template>
         <template v-if="column.dataIndex==='operation'">
          <a-space>
            <a-popconfirm
              v-if="dataSource.length"
              title="是否删除?"
              cancelText="取消"
              okText="确定"
              @confirm="onDelete(record.id)"
              >
              <a-button type="primary" danger>删除</a-button>
            </a-popconfirm>
```

```
          <a-button type="primary" @click="edit(record)">编辑</a-button>
        </a-space>
       </template>
    </template>
  </a-table>
  <br>
  <TrainPagination :total="total" :pageSizeOptions="['5','7']" @change="onChange"></TrainPagination>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import {PASSENGER_TYPE_ARRAY} from '@/utils/enum.ts';
import TrainPagination from '@/components/TrainPagination.vue';

//路由对象
const router = useRouter();

//接口
interface Passenger {
  id: number,
  memberId: number,
  name: string,
  idCard: string,
  type: string,
  createTime: string,
  updateTime: string,
}

//数据列
const columns = [
{
  title: '姓名',
  dataIndex: 'name',
  key: 'name',
},
{
  title: '身份证',
  dataIndex: 'idCard',
  key: 'idCard',
},
{
  title: '旅客类型',
  dataIndex: 'type',
  key: 'type',
},
{
  title: '操作',
  dataIndex: 'operation'
}
];
//数据源
const dataSource = ref<Passenger[]>([]);

//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/trainMember/passenger/del/${id}`,
    }).then(()=>{
      showData();
      message.success('删除成功');
    })
}
//刷新
const refreshData = () => {
    showData();
}
//新增
const add = () => {
```

```
    router.push('/main/passengerSave');
  }
  //编辑
  const edit = (record:Passenger) => {
    router.push({
      path: '/main/passengerSave',
      state: {
        params:JSON.stringify(record)
      }
    })
  }

  //--------------分页----------------
  const total = ref(0);
  //分页参数
  const queryParams=ref({
    current:1,
    size:5,
    type:null,
    name:null
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainMember/passenger/page',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })

</script>
```

7.4.2 保存更新: TrainPassengerSave.vue




```
<template>
  <a-form
    :model="formState"
```

```
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="会员id"
      name="memberId"
    >
      <a-input v-model:value="formState.memberId" />
    </a-form-item>
    <a-form-item
      label="乘车人"
      name="name"
    >
      <a-input v-model:value="formState.name" />
    </a-form-item>
    <a-form-item
      label="身份证"
      name="idCard"
    >
      <a-input v-model:value="formState.idCard" />
    </a-form-item>
    <a-form-item
      label="旅客类型"
      name="type"
    >
     <a-select v-model:value="formState.type" placeholder="请选择">
      <a-select-option value="1">成人</a-select-option>
      <a-select-option value="2">儿童</a-select-option>
      <a-select-option value="3">学生</a-select-option>
     </a-select>
    </a-form-item>

    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.memberId ? '修改' : '新增' }}
      </a-button>
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';

//路由对象
const router = useRouter();
//接口类型
interface FormState {
  id: string;
  memberId: string;
  name: string;
  idCard: string;
  type: string;
}
//表单数据
const formState = reactive<FormState>({
  id: '',
  memberId: '',
  name: '',
  idCard: '',
  type: '',
});

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
```

```
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainMember/passenger/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainMember/passenger/update', values).then(() => {
     message.success('更新成功');
  })
}

const onFinish = () => {
     if(formState.id){
        updateData(formState);
     }else{
        saveData(formState);
     }
     // 刷新列表
     router.push('/main/passenger');
}
const onFinishFailed = (errorInfo: any) => {
  console.log('Failed:', errorInfo);
};
</script>
```

八、基础数据维护

8.1 车站管理

8.1.1 数据表

```
CREATE TABLE `station` (
  `id` bigint NOT NULL COMMENT 'id',
  `name` varchar(20) NOT NULL COMMENT '站名',
  `name_pinyin` varchar(50) NOT NULL COMMENT '站名拼音',
  `name_py` varchar(50) NOT NULL COMMENT '站名拼音首字母',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `name_unique` (`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='车站';
```

8.1.2 后端三层+分页

8.1.3 前端：车站

- 安装拼音

  ```
  官网：
      https://github.com/zh-1x/pinyin-pro
  拼音：
      pnpm install pinyin-pro
  ```

- 引入tool.js工具到public目录

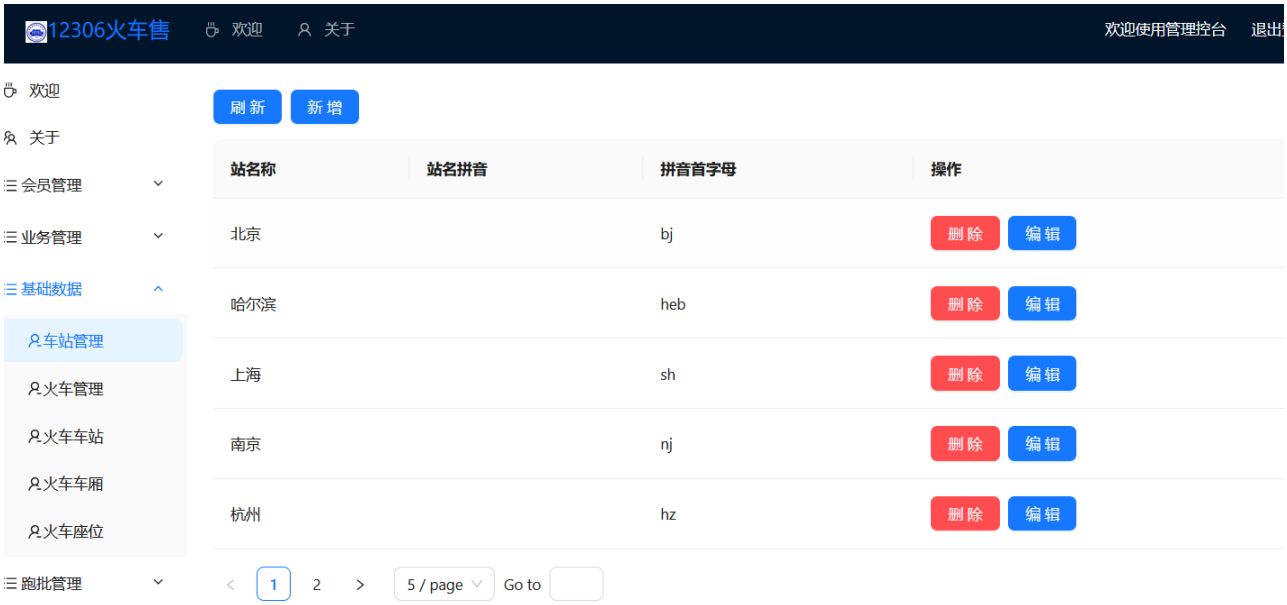
```
index.html页面引入：
    <script src="/public/tool.js"></script>
```

- 安装日期工具

```
官网：https://dayjs.fenxianglu.cn/

安装：pnpm add dayjs
```

- TrainStation.vue




```
<template>
    <p>
      <a-space>
        <a-button type="primary" @click="refreshData">刷新</a-button>
        <a-button type="primary" @click="add">新增</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{column,record}">
         <template v-if="column.dataIndex==='operation'">
          <a-space>
            <a-popconfirm
              v-if="dataSource.length"
              title="是否删除?"
              cancelText="取消"
              okText="确定"
              @confirm="onDelete(record.id)"
              >
              <a-button type="primary" danger>删除</a-button>
            </a-popconfirm>
            <a-button type="primary" @click="edit(record)">编辑</a-button>
          </a-space>
         </template>
      </template>
    </a-table>
    <br>
   <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
  </template>

  <script setup lang="ts">
  import { ref, onMounted } from 'vue';
  import axios from "axios";
  import { message } from 'ant-design-vue';
  import { useRouter } from 'vue-router';
```

```
//路由对象
const router = useRouter();

//接口
interface Station {
  id: number,
  name: string,
  namePinyin: string,
  namePy: string,
}

//数据列
const columns = [
{
  title: '站名称',
  dataIndex: 'name',
  key: 'name',
},
{
  title: '站名拼音',
  dataIndex: 'namepinyin',
  key: 'namePinyin',
},
{
  title: '拼音首字母',
  dataIndex: 'namePy',
  key: 'namePy',
},
{
  title: '操作',
  dataIndex: 'operation'
}
];
//数据源
const dataSource = ref<Station[]>([]);

//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/trainBase/station/delete/${id}`,
    }).then(()=>{
      showData();
      message.success('删除成功');
    })
}
//刷新
const refreshData = () => {
    showData();
}
//新增
const add = () => {
  router.push('/main/base/stationSave');
}
//编辑
const edit = (record:Station) => {
  router.push({
    path: '/main/base/stationSave',
    state: {
      params:JSON.stringify(record)
    }
  })
}
//--------------分页----------------
const total = ref(0);
//分页参数
const queryParams=ref({
  current:1,
  size:5,
  type:null,
  name:''
```

```
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainBase/station/page',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })

</script>
```

- StationSave.vue




```
<template>
  <a-form
    :model="formState"
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="id"
      name="id"
      hidden="true"
    >
      <a-input v-model:value="formState.id" />
    </a-form-item>
    <a-form-item
      label="车站名称"
      name="name"
    >
      <a-input v-model:value="formState.name" />
    </a-form-item>
```

```
    <a-form-item
      label="车站拼音"
      name="namePinyin"
    >
      <a-input v-model:value="formState.namePinyin" />
    </a-form-item>
    <a-form-item
      label="拼音首字母"
      name="namePy"
    >
      <a-input v-model:value="formState.namePy" />
    </a-form-item>
    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.id ? '修改' : '新增' }}
      </a-button>
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted,watch } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';
import {pinyin} from 'pinyin-pro';

//路由对象
const router = useRouter();
 //接口
 interface FormState {
    id: string,
    name: string,
    namePinyin: string,
    namePy: string,
  }
//表单数据
const formState = reactive<FormState>({
  id: '',
  name: '',
  namePinyin: '',
  namePy: '',
});

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainBase/station/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainBase/station/update', values).then(() => {
     message.success('更新成功');
  })
}

const onFinish = (values: any) => {
     if(formState.id){
      console.log(values);
      updateData(values);
     }else{
      saveData(values);
```

```
        }
        // 刷新列表
        router.push('/main/base/station');
    }
    const onFinishFailed = (errorInfo: any) => {
      console.log('Failed:', errorInfo);
    };
    //添加监听拼音
    watch(()=>formState.name,()=>{
        formState.namePinyin=pinyin(formState.name,{toneType:'none'}).replaceAll(" ","")
        formState.namePy=pinyin(formState.name,{toneType:'none',pattern:'first'}).replaceAll(" ","")
    })
    </script>
```

8.2 火车车次管理

8.2.1 数据表

```
CREATE TABLE `train` (
  `id` bigint NOT NULL COMMENT 'id',
  `code` varchar(20) NOT NULL COMMENT '车次编号',
  `type` char(1) NOT NULL COMMENT '车次类型|枚举[TrainTypeEnum]',
  `start` varchar(20) NOT NULL COMMENT '始发站',
  `start_pinyin` varchar(50) NOT NULL COMMENT '始发站拼音',
  `start_time` time NOT NULL COMMENT '出发时间',
  `end` varchar(20) NOT NULL COMMENT '终点站',
  `end_pinyin` varchar(50) NOT NULL COMMENT '终点站拼音',
  `end_time` time NOT NULL COMMENT '到站时间',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `code_unique` (`code`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='车次';
```

8.2.2 Train.vue




```
<template>
    <p>
      <a-space>
        <a-button type="primary" @click="refreshData">刷新</a-button>
        <a-button type="primary" @click="add">新增</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{column,record}">
         <template v-if="column.dataIndex==='operation'">
          <a-space>
```

```
          <a-popconfirm
            v-if="dataSource.length"
            title="是否删除?"
            cancelText="取消"
            okText="确定"
            @confirm="onDelete(record.id)"
            >
            <a-button type="primary" danger>删除</a-button>
          </a-popconfirm>
          <a-button type="primary" @click="edit(record)">编辑</a-button>
          <a-button type="primary" @click="genData(record.code)">生成座位</a-button>
        </a-space>
      </template>
    </template>
  </a-table>
  <br>
  <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';


//路由对象
const router = useRouter();

//接口
interface Train {
  id: number,
  code: string,
  type: string,
  start: string,
  startPinyin: string,
  startTime: string,
  end: string,
  endPinyin: string,
  endTime: string,
}

//数据列
const columns = [
{
  title: '车次编号',
  dataIndex: 'code',
  key: 'code',
},
{
  title: '车次类型',
  dataIndex: 'type',
  key: 'type',
},
{
  title: '始发站',
  dataIndex: 'start',
  key: 'start',
},
{
  title: '始发站拼音',
  dataIndex: 'startPinyin',
  key: 'start',
},
{
  title: '出发时间',
  dataIndex: 'startTime',
  key: 'startTime',
},
{
  title: '终点站',
  dataIndex: 'end',
```

```
    key: 'end',
  },
  {
    title: '终点站拼音',
    dataIndex: 'endPinyin',
    key: 'endPinyin',
  },
  {
    title: '到站时间',
    dataIndex: 'endTime',
    key: 'endTime',
  },
  {
    title: '操作',
    dataIndex: 'operation'
  }
];
//数据源
const dataSource = ref<Train[]>([]);

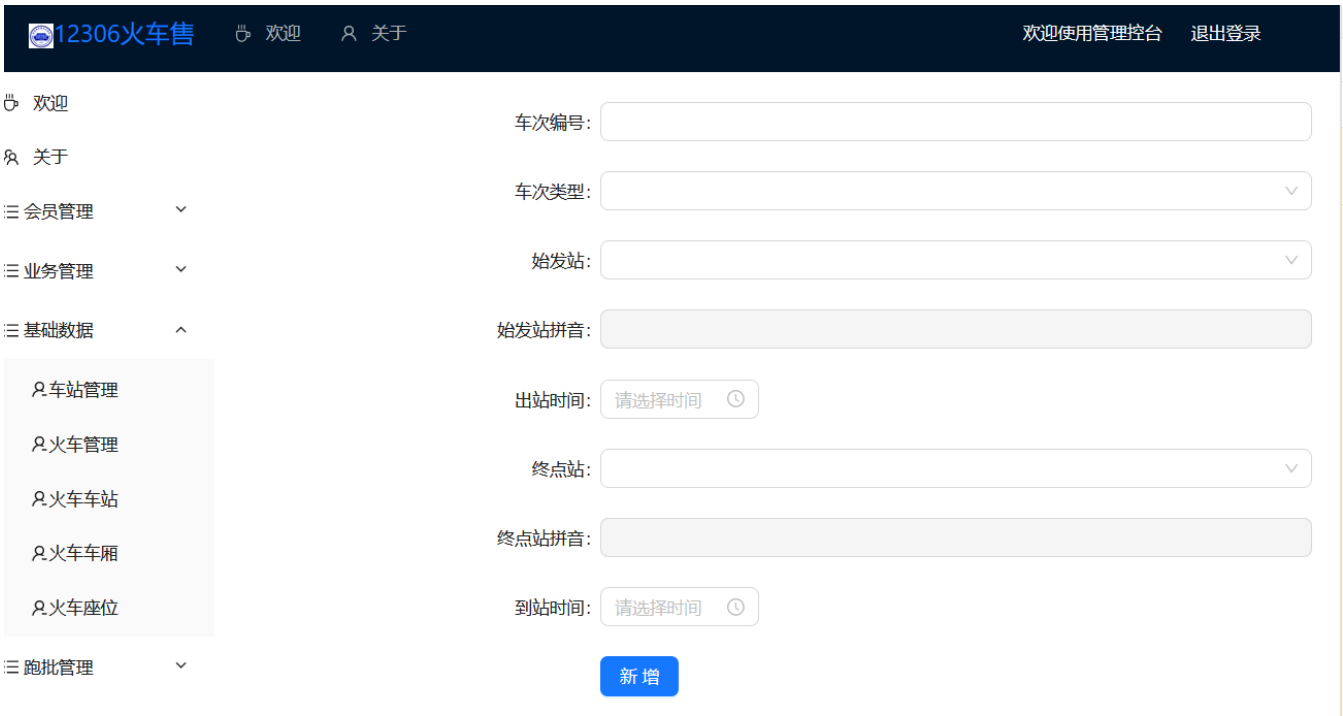
//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/train/delete/${id}`,
    }).then(()=>{
      showData();
      message.success('删除成功');
    })
}
//刷新
const refreshData = () => {
    showData();
}
//新增
const add = () => {
  router.push('/main/base/trainSave');
}
//编辑
const edit = (record:Train) => {
  router.push({
    path: '/main/base/trainSave',
    state: {
      params:JSON.stringify(record)
    }
  })
}
//生成座位
const genData = (code:string) => {
  axios({
    method: 'get',
    url: `/trainBase/train/gen/${code}`,
  }).then(()=>{
    message.success('生成座位成功');
  })
}
//--------------分页----------------
const total = ref(0);
//分页参数
const queryParams=ref({
  current:1,
  size:5,
  type:null,
  name:''
})

//当前页码变化
const onChange = (page:number,pageSize:number) => {
  queryParams.value.current = page
  queryParams.value.size = pageSize
  showData()
};
```

```
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainBase/train/page',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })
</script>
```

8.2.3 TrainSave.vue




```
<template>
  <a-form
    :model="formState"
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="id"
      name="id"
      hidden="true"
    >
      <a-input v-model:value="formState.id" />
    </a-form-item>
    <a-form-item
      label="车次编号"
      name="code"
    >
      <a-input v-model:value="formState.code" />
    </a-form-item>
    <a-form-item
```

```
      label="车次类型"
      name="type"
    >
    <a-select v-model:value="formState.type">
      <a-select-option v-for="item in TRAIN_TYPE_ARRAY" :key="item.code" :value="item.code">
      {{item.desc}}
      </a-select-option>
    </a-select>
    </a-form-item>
    <a-form-item
      label="始发站"
      name="start"
    >
      <StationSelect v-model="formState.start"></StationSelect>
    </a-form-item>
    <a-form-item
      label="始发站拼音"
      name="startPinyin"
      >
      <a-input v-model:value="formState.startPinyin" disabled/>
    </a-form-item>
    <a-form-item
      label="出站时间"
      name="startTime"
    >
    <a-time-picker v-model:value="formState.startTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
    </a-form-item>
    <a-form-item
      label="终点站"
      name="end"
    >
    <StationSelect v-model="formState.end"></StationSelect>
    </a-form-item>
    <a-form-item
    label="终点站拼音"
    name="endPinyin"
  >
    <a-input v-model:value="formState.endPinyin" disabled/>
  </a-form-item>
  <a-form-item
      label="到站时间"
      name="endTime"
    >
    <a-time-picker v-model:value="formState.endTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
    </a-form-item>


    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.id ? '修改' : '新增' }}
      </a-button>
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted,watch } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';
import {pinyin} from 'pinyin-pro';
import StationSelect from '@/components/StationSelect.vue';
import {TRAIN_TYPE_ARRAY} from '@/utils/enum.ts'

//路由对象
const router = useRouter();
 //接口
 interface FormState {
    id: string,
    code: string,
    type: string,
    start: string,
    startPinyin: string,
```

```
    startTime: string,
    end: string,
    endPinyin: string,
    endTime: string,
  }
//表单数据
const formState = reactive<FormState>({
  id: '',
  code: '',
  type: '',
  start: '',
  startPinyin: '',
  startTime: '',
  end: '',
  endPinyin: '',
  endTime: '',
});

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainBase/train/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainBase/train/update', values).then(() => {
     message.success('更新成功');
  })
}

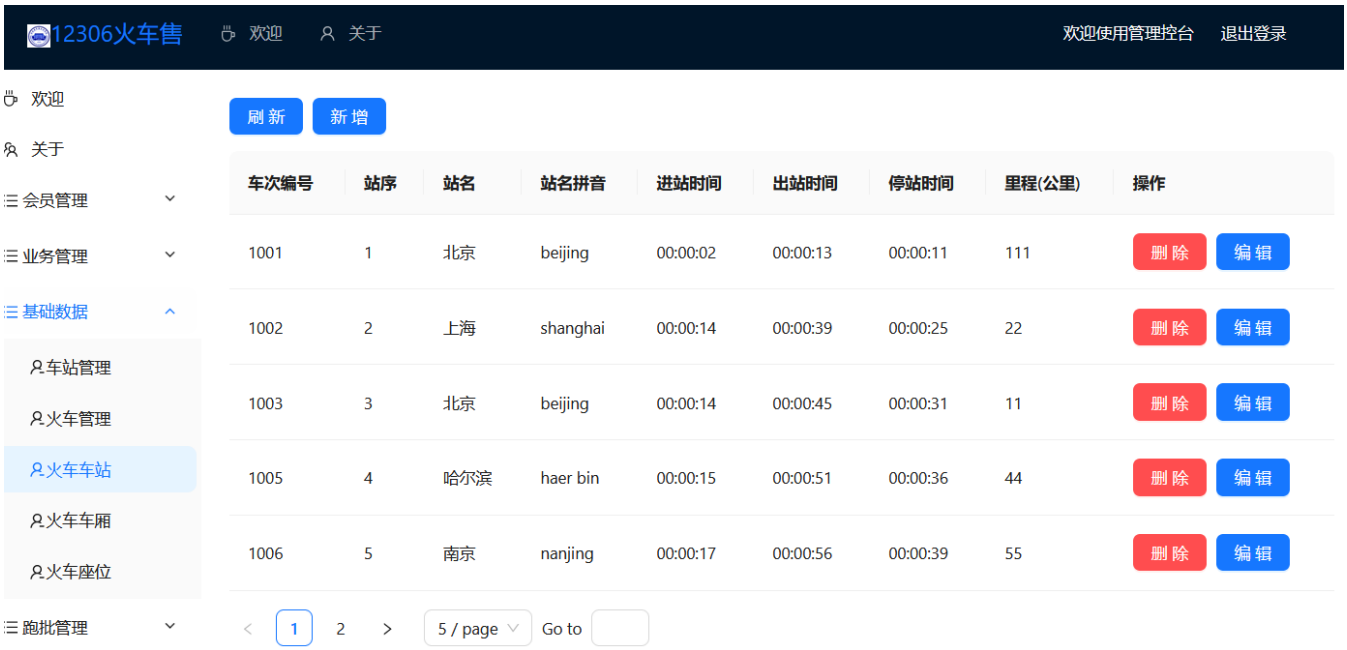
const onFinish = (values: any) => {
     if(formState.id){
      console.log(values);
      updateData(values);
     }else{
      saveData(values);
     }
     // 刷新列表
     router.push('/main/base/train');
}
const onFinishFailed = (errorInfo: any) => {
  console.log('Failed:', errorInfo);
};
//添加监听拼音
watch(()=>formState.start,()=>{
    if(Tool.isNotEmpty(formState.start)){
      formState.startPinyin = pinyin(formState.start, { toneType: 'none' }).replaceAll(' ','');
    }else{
      formState.startPinyin = '';
    }
})
watch(()=>formState.end,()=>{
    if(Tool.isNotEmpty(formState.end)){
      formState.endPinyin = pinyin(formState.end, { toneType: 'none' }).replaceAll(' ','');
    }else{
      formState.endPinyin = '';
    }
})
</script>
```

8.3 火车车站

8.3.1 数据表

```
CREATE TABLE `train_station` (
  `id` bigint NOT NULL COMMENT 'id',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `index` int NOT NULL COMMENT '站序',
  `name` varchar(20) NOT NULL COMMENT '站名',
  `name_pinyin` varchar(50) NOT NULL COMMENT '站名拼音',
  `in_time` time DEFAULT NULL COMMENT '进站时间',
  `out_time` time DEFAULT NULL COMMENT '出站时间',
  `stop_time` time DEFAULT NULL COMMENT '停站时长',
  `km` decimal(8,2) NOT NULL COMMENT '里程（公里）|从上一站到本站的距离',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `train_code_index_unique` (`train_code`,`index`),
  UNIQUE KEY `train_code_name_unique` (`train_code`,`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='火车车站';
```

8.3.2 显示




```
<template>
    <p>
      <a-space>
        <a-button type="primary" @click="refreshData">刷新</a-button>
        <a-button type="primary" @click="add">新增</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{column,record}">
         <template v-if="column.dataIndex==='operation'">
          <a-space>
            <a-popconfirm
              v-if="dataSource.length"
              title="是否删除?"
              cancelText="取消"
              okText="确定"
              @confirm="onDelete(record.id)"
              >
              <a-button type="primary" danger>删除</a-button>
            </a-popconfirm>
            <a-button type="primary" @click="edit(record)">编辑</a-button>
          </a-space>
         </template>
```

```
    </template>
  </a-table>
  <br>
  <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';


//路由对象
const router = useRouter();

//接口
interface TrainStation {
  id: number,
  trainCode: string,
  index: string,
  name: string,
  namePinyin: string,
  inTime: string,
  outTime: string,
  stopTime: string,
  km: string,
}

//数据列
const columns = [
{
  title: '车次编号',
  dataIndex: 'trainCode',
  key: 'trainCode',
},
{
  title: '站序',
  dataIndex: 'index',
  key: 'index',
},
{
  title: '站名',
  dataIndex: 'name',
  key: 'name',
},
{
  title: '站名拼音',
  dataIndex: 'namePinyin',
  key: 'namePinyin',
},
{
  title: '进站时间',
  dataIndex: 'inTime',
  key: 'inTime',
},
{
  title: '出站时间',
  dataIndex: 'outTime',
  key: 'outTime',
},
{
  title: '停站时间',
  dataIndex: 'stopTime',
  key: 'stopTime',
},
{
  title: '里程(公里)',
  dataIndex: 'km',
  key: 'km',
},
{
```

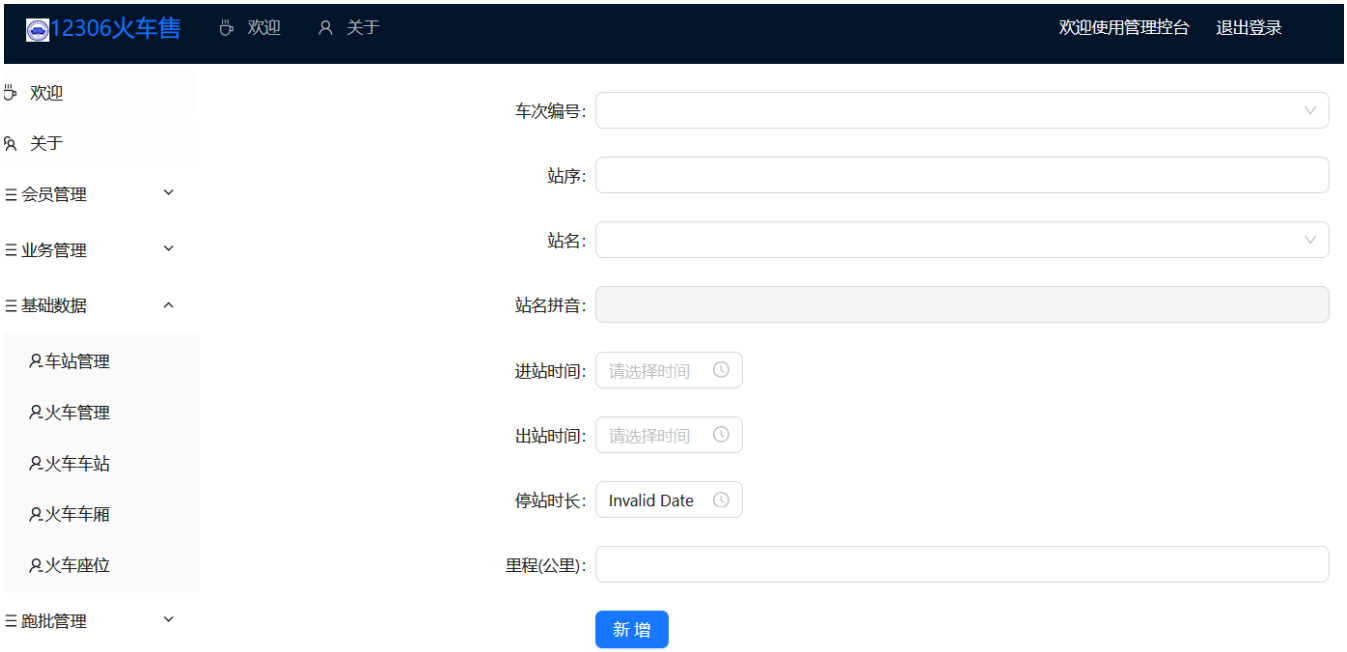
```
    title: '操作',
    dataIndex: 'operation'
  }
  ];
  //数据源
  const dataSource = ref<TrainStation[]>([]);

  //删除
  const onDelete = (id:number) => {
      axios({
        method: 'delete',
        url: `/trainBase/trainStation/delete/${id}`,
      }).then(()=>{
        showData();
        message.success('删除成功');
      })
  }
  //刷新
  const refreshData = () => {
      showData();
  }
  //新增
  const add = () => {
    router.push('/main/base/trainStationSave');
  }
  //编辑
  const edit = (record:TrainStation) => {
    router.push({
      path: '/main/base/trainStationSave',
      state: {
        params:JSON.stringify(record)
      }
    })
  }
  //--------------分页----------------
  const total = ref(0);
  //分页参数
  const queryParams=ref({
    current:1,
    size:5,
    type:null,
    name:''
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainBase/trainStation/page',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })
</script>
```

8.3.3 保存更新




```
<template>
  <a-form
    :model="formState"
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="id"
      name="id"
      hidden="true"
    >
      <a-input v-model:value="formState.id" />
    </a-form-item>
    <a-form-item
      label="车次编号"
      name="trainCode"
    >
      <TrainSelect v-model="formState.trainCode"></TrainSelect>
    </a-form-item>
    <a-form-item
      label="站序"
      name="index"
    >
    <a-input v-model:value="formState.index"/>
    </a-form-item>
    <a-form-item
      label="站名"
      name="name"
    >
      <StationSelect v-model="formState.name"></StationSelect>
    </a-form-item>
    <a-form-item
      label="站名拼音"
      name="namePinyin"
      >
      <a-input v-model:value="formState.namePinyin" disabled/>
    </a-form-item>
    <a-form-item
      label="进站时间"
      name="inTime"
    >
    <a-time-picker v-model:value="formState.inTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
```

```
    </a-form-item>
    <a-form-item
      label="出站时间"
      name="outTime"
    >
    <a-time-picker v-model:value="formState.outTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
    </a-form-item>
    <a-form-item
    label="停站时长"
    name="stopTime"
  >
  <a-time-picker v-model:value="formState.stopTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
  </a-form-item>
  <a-form-item
      label="里程(公里)"
      name="km"
    >
    <a-input v-model:value="formState.km" />
    </a-form-item>

    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.id ? '修改' : '新增' }}
      </a-button>
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted,watch } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';
import {pinyin} from 'pinyin-pro';
import TrainSelect from '@/components/TrainSelect.vue';
import StationSelect from '@/components/StationSelect.vue';
import dayjs from 'dayjs';

//路由对象
const router = useRouter();
 //接口
 interface FormState {
  id: string,
  trainCode: string,
  index: string,
  name: string,
  namePinyin: string,
  inTime: string,
  outTime: string,
  stopTime: string,
  km: string,
  }
//表单数据
const formState = reactive<FormState>({
  id: '',
  trainCode: '',
  index: '',
  name: '',
  namePinyin: '',
  inTime: '',
  outTime: '',
  stopTime: '',
  km: '',
});

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
```

```
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainBase/trainStation/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainBase/trainStation/update', values).then(() => {
     message.success('更新成功');
  })
}

const onFinish = (values: any) => {
  console.log('--------values------->>>',values);
  console.log('--------formState------->>>',formState);
     if(formState.id){
      console.log(values);
      updateData(values);
     }else{
      saveData(values);
     }
     // 刷新列表
     router.push('/main/base/trainStation');
}
const onFinishFailed = (errorInfo: any) => {
  console.log('Failed:', errorInfo);
};
//添加监听拼音
watch(()=>formState.name,()=>{
    if(Tool.isNotEmpty(formState.name)){
      formState.namePinyin = pinyin(formState.name, { toneType: 'none' }).replace(' ','');
    }else{
      formState.namePinyin = '';
    }
}, { immediate: true })

//计算进站停车时长
watch(()=>formState.inTime,()=>{
  let diff = dayjs(formState.outTime,'HH:mm:ss').diff(dayjs(formState.inTime,'HH:mm:ss'),'seconds');
  formState.stopTime = dayjs('00:00:00','HH:mm:ss').second(diff).format('HH:mm:ss');
}, { immediate: true })

//计算出站停车时长
watch(()=>formState.outTime,()=>{
  let diff = dayjs(formState.outTime,'HH:mm:ss').diff(dayjs(formState.inTime,'HH:mm:ss'),'seconds');
  formState.stopTime = dayjs('00:00:00','HH:mm:ss').second(diff).format('HH:mm:ss');
}, { immediate: true })

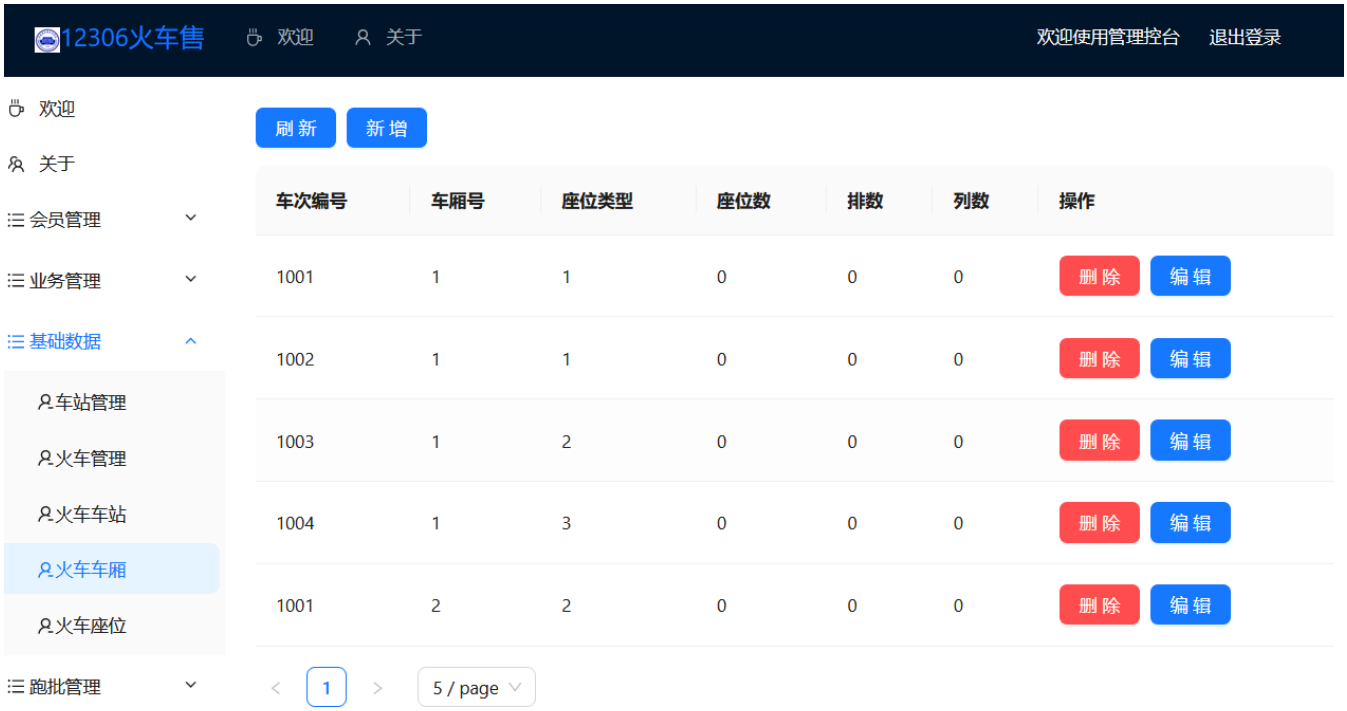
</script>
```

8.4 火车车厢

8.4.1 数据表

```sql
CREATE TABLE `train_carriage` (
  `id` bigint NOT NULL COMMENT 'id',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `index` int NOT NULL COMMENT '厢号',
  `seat_type` char(1) NOT NULL COMMENT '座位类型|枚举[SeatTypeEnum]',
  `seat_count` int NOT NULL COMMENT '座位数',
  `row_count` int NOT NULL COMMENT '排数',
  `col_count` int NOT NULL COMMENT '列数',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `train_code_index_unique` (`train_code`,`index`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='火车车厢';
```

8.4.2 显示




```vue
<template>
    <p>
      <a-space>
        <a-button type="primary" @click="refreshData">刷新</a-button>
        <a-button type="primary" @click="add">新增</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{column,record}">
         <template v-if="column.dataIndex==='operation'">
          <a-space>
            <a-popconfirm
              v-if="dataSource.length"
              title="是否删除?"
              cancelText="取消"
              okText="确定"
              @confirm="onDelete(record.id)"
              >
              <a-button type="primary" danger>删除</a-button>
            </a-popconfirm>
            <a-button type="primary" @click="edit(record)">编辑</a-button>
          </a-space>
         </template>
      </template>
    </a-table>
    <br>
    <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
```

```
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';


//路由对象
const router = useRouter();

//接口
interface TrainCarriage {
  id: number,
  trainCode: string,
  index: string,
  seatType: string,
  seatCount: string,
  rowCount: string,
  colCount: string,
}

//数据列
const columns = [
{
  title: '车次编号',
  dataIndex: 'trainCode',
  key: 'trainCode',
},
{
  title: '车厢号',
  dataIndex: 'index',
  key: 'index',
},
{
  title: '座位类型',
  dataIndex: 'seatType',
  key: 'seatType',
},
{
  title: '座位数',
  dataIndex: 'seatCount',
  key: 'seatCount',
},
{
  title: '排数',
  dataIndex: 'rowCount',
  key: 'rowCount',
},
{
  title: '列数',
  dataIndex: 'colCount',
  key: 'colCount',
},
{
  title: '操作',
  dataIndex: 'operation'
}
];
//数据源
const dataSource = ref<TrainCarriage[]>([]);

//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/trainBase/trainCarriage/delete/${id}`,
    }).then(()=>{
      showData();
      message.success('删除成功');
    })
```

```
  }
  //刷新
  const refreshData = () => {
      showData();
  }
  //新增
  const add = () => {
    router.push('/main/base/trainCarriageSave');
  }
  //编辑
  const edit = (record:TrainCarriage) => {
    router.push({
      path: '/main/base/trainCarriageSave',
      state: {
        params:JSON.stringify(record)
      }
    })
  }
  //--------------分页----------------
  const total = ref(0);
  //分页参数
  const queryParams=ref({
    current:1,
    size:5,
    type:null,
    name:''
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainBase/trainCarriage/page',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })
</script>
```

8.4.3 保存更新




```
<template>
  <a-form
    :model="formState"
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="id"
      name="id"
      hidden="true"
    >
      <a-input v-model:value="formState.id" />
    </a-form-item>
    <a-form-item
      label="车次编号"
      name="trainCode"
    >
      <TrainSelect v-model:value="formState.trainCode" />
    </a-form-item>
    <a-form-item
      label="车厢号"
      name="index"
    >
      <a-input v-model:value="formState.index" />
    </a-form-item>
    <a-form-item
      label="座位类型"
      name="seatType"
    >
    <a-select v-model:value="formState.seatType">
      <a-select-option v-for="item in SEAT_TYPE_ARRAY" :key="item.code" :value="item.code">
      {{item.desc}}
      </a-select-option>
    </a-select>
    </a-form-item>
    <a-form-item
      label="排数"
      name="rowCount"
    >
      <a-input-number v-model:value="formState.rowCount" />
    </a-form-item>
    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.id ? '修改' : '新增' }}
      </a-button>
```

```
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';
import TrainSelect from '@/components/TrainSelect.vue';
import {SEAT_TYPE_ARRAY} from '@/utils/enum.ts'

//路由对象
const router = useRouter();
 //接口
 interface FormState {
  id: string,
  trainCode: string,
  index: string,
  seatType: string,
  seatCount: number,
  rowCount: number,
  colCount: number,
  }
//表单数据
const formState = reactive<FormState>({
  id: "",
  trainCode: "",
  index: "",
  seatType: "",
  seatCount: 0,
  rowCount: 0,
  colCount: 0,
});

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainBase/trainCarriage/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainBase/trainCarriage/update', values).then(() => {
     message.success('更新成功');
  })
}

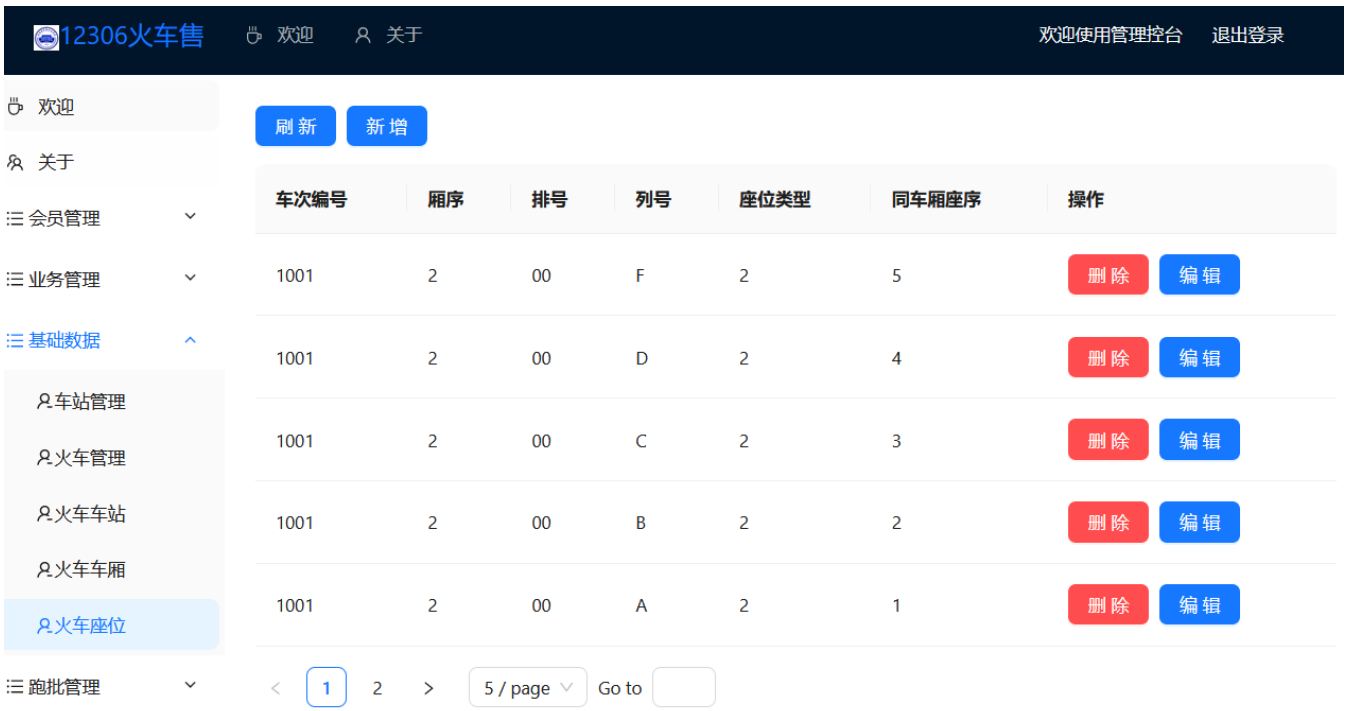
const onFinish = () => {
     if(formState.id){
      updateData(formState);
     }else{
      saveData(formState);
     }
     // 刷新列表
     router.push('/main/base/trainCarriage');
}
const onFinishFailed = (errorInfo: any) => {
  console.log('Failed:', errorInfo);
};
</script>
```

8.5 火车座位

8.5.1 数据表

```
CREATE TABLE `train_seat` (
  `id` bigint NOT NULL COMMENT 'id',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `carriage_index` int NOT NULL COMMENT '厢序',
  `row` char(2) NOT NULL COMMENT '排号|01, 02',
  `col` char(1) NOT NULL COMMENT '列号|枚举[SeatColEnum]',
  `seat_type` char(1) NOT NULL COMMENT '座位类型|枚举[SeatTypeEnum]',
  `carriage_seat_index` int NOT NULL COMMENT '同车厢座序',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='座位';
```

8.5.2 显示




```
<template>
    <p>
      <a-space>
        <a-button type="primary" @click="refreshData">刷新</a-button>
        <a-button type="primary" @click="add">新增</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{column,record}">
         <template v-if="column.dataIndex==='operation'">
          <a-space>
            <a-popconfirm
              v-if="dataSource.length"
              title="是否删除?"
              cancelText="取消"
              okText="确定"
              @confirm="onDelete(record.id)"
              >
              <a-button type="primary" danger>删除</a-button>
            </a-popconfirm>
            <a-button type="primary" @click="edit(record)">编辑</a-button>
          </a-space>
         </template>
      </template>
    </a-table>
```

```
  <br>
  <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';


//路由对象
const router = useRouter();

//接口
interface TrainSeat {
  id: number,
  trainCode: string,
  carriageIndex: string,
  row: string,
  col: string,
  seatType: string,
  carriageSeatIndex: string,
}

//数据列
const columns = [
{
  title: '车次编号',
  dataIndex: 'trainCode',
  key: 'trainCode',
},
{
  title: '厢序',
  dataIndex: 'carriageIndex',
  key: 'carriageIndex',
},
{
  title: '排号',
  dataIndex: 'row',
  key: 'row',
},
{
  title: '列号',
  dataIndex: 'col',
  key: 'col',
},
{
  title: '座位类型',
  dataIndex: 'seatType',
  key: 'seatType',
},
{
  title: '同车厢座序',
  dataIndex: 'carriageSeatIndex',
  key: 'carriageSeatIndex',
},
{
  title: '操作',
  dataIndex: 'operation'
}
];
//数据源
const dataSource = ref<TrainSeat[]>([]);

//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/trainBase/seat/delete/${id}`,
    }).then(()=>{
      showData();
```

```
        message.success('删除成功');
      })
  }
  //刷新
  const refreshData = () => {
      showData();
  }
  //新增
  const add = () => {
    router.push('/main/base/trainSeatSave');
  }
  //编辑
  const edit = (record:TrainSeat) => {
    router.push({
      path: '/main/base/trainSeatSave',
      state: {
        params:JSON.stringify(record)
      }
    })
  }
  //--------------分页----------------
  const total = ref(0);
  //分页参数
  const queryParams=ref({
    current:1,
    size:5,
    type:null,
    name:''
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainBase/seat/page',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })
</script>
```

8.5.3 自动生成座位

- 座位模块：service

```
public interface SeatService extends IService<TrainSeatEntity> {
    void genTrainSeat(String trainCode);
}

-----------------------------

@Transactional
@Override
public void genTrainSeat(String trainCode) {
    //生成数据前先按照车次编号删除对应数据，防止重复
    baseMapper.delete(new LambdaQueryWrapper<TrainSeatEntity>
().eq(TrainSeatEntity::getTrainCode,trainCode));
```

```
    //查找当前车次下的所有的车厢
    List<TrainCarriageEntity> carriageEntityList = trainCarriageMapper
        .selectList(new LambdaQueryWrapper<TrainCarriageEntity>()
                    .eq(TrainCarriageEntity::getTrainCode,trainCode));
    // 循环生成每个车厢的座位
    for (TrainCarriageEntity trainCarriage : carriageEntityList) {
        Integer rowCount = trainCarriage.getRowCount();//获取总行数
        String seatType = trainCarriage.getSeatType();//获取座位类
        int seatIndex = 1;
        // 根据车厢的座位类型，筛选出所有的列，比如车箱类型是一等座，则筛选出columnList={ACDF}
        List<SeatColEnum> colEnumList = SeatColEnum.getColsByType(seatType);
        // 循环行数
        for (int row = 0; row <= rowCount; row++) {
            // 循环列数
            for (SeatColEnum seatColEnum : colEnumList) {
                // 构造座位数据并保存数据库
                TrainSeatEntity trainSeat = new TrainSeatEntity();
                trainSeat.setTrainCode(trainCode);
                trainSeat.setCarriageIndex(trainCarriage.getIndex());
                //两位，前面补零
                trainSeat.setRow(StrUtil.fillBefore(String.valueOf(row),'0',2));
                trainSeat.setCol(seatColEnum.getCode());
                trainSeat.setSeatType(seatType);
                trainSeat.setCarriageSeatIndex(seatIndex++);
                trainSeat.setCreateTime(new Date());
                trainSeat.setUpdateTime(new Date());
                //保存
                baseMapper.insert(trainSeat);
            }
        }
    }
}
```

- 在train火车模块调用

```
@RestController
@RequestMapping("/train")
public class TrainController {
    @GetMapping("gen/{trainCode}")
    public R genTrain(@PathVariable String trainCode) {
        seatService.genTrainSeat(trainCode);
        return R.ok();
    }
}
```

九、每日火车数据

- 根据火车基础数据生成

9.1 需要完成的任务

- 增加每日车次数据维护：车次、车站、车厢、座位
- 增加定时任务：每天从车次基础数据生成每日车次数据

9.2 业务管理-每日车次

- 区别火车表：
    - 添加了一个date日期字段：表示每日
    - 添加了UNIQUE KEY date_code_unique (date , code)：表示同一天不能有相同的火车编号出现

9.2.1 数据表

```
CREATE TABLE `daily_train` (
  `id` bigint NOT NULL COMMENT 'id',
  `date` date NOT NULL COMMENT '日期',
  `code` varchar(20) NOT NULL COMMENT '车次编号',
  `type` char(1) NOT NULL COMMENT '车次类型|枚举[TrainTypeEnum]',
  `start` varchar(20) NOT NULL COMMENT '始发站',
  `start_pinyin` varchar(50) NOT NULL COMMENT '始发站拼音',
  `start_time` time NOT NULL COMMENT '出发时间',
  `end` varchar(20) NOT NULL COMMENT '终点站',
  `end_pinyin` varchar(50) NOT NULL COMMENT '终点站拼音',
  `end_time` time NOT NULL COMMENT '到站时间',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `date_code_unique` (`date`,`code`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='每日车次';
```

9.2.2 后端编写

- 注意：实体类中的date格式需要格式化
  - get请求：

    ```
    @DateTimeFormat(pattern = "yyyy-MM-dd")
    private Date date;
    ```

  - post请求：

    ```
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss")
    private Date date;
    ```

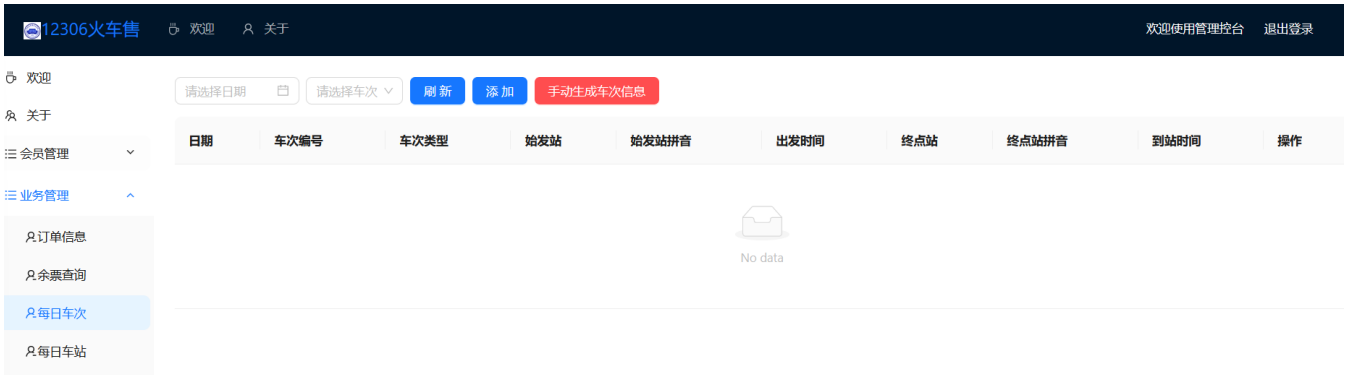
- 返回的日期格式可以在yaml配置文件编写

```
spring:
  application:
    name: daily-train
  jackson:
    date-format: YYYY-MM-dd
```

9.2.3 三层编写：分页查询

- 搭建train-daily模块

9.2.4 前端：DailyTrain.vue




```
<template>
    <p>
      <a-space>
        <a-date-picker v-model:value="queryParams.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期" />
```

```
      <TrainSelect v-model="queryParams.code" style="width:200px"></TrainSelect>
      <a-button type="primary" @click="refreshData">刷新</a-button>
      <a-button type="primary" @click="add">添加</a-button>
      <a-button type="primary" @click="genTrainCode" danger>手动生成车次信息</a-button>
    </a-space>
  </p>
  <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
    <template #bodyCell="{column,record}">
      <template v-if="column.dataIndex==='operation'">
       <a-space>
         <a-popconfirm
           v-if="dataSource.length"
           title="是否删除?"
           cancelText="取消"
           okText="确定"
           @confirm="onDelete(record.id)"
           >
           <a-button type="primary" danger>删除</a-button>
         </a-popconfirm>
         <a-button type="primary" @click="edit(record)">编辑</a-button>
       </a-space>
      </template>
    </template>
  </a-table>
  <br>
  <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
  <!-- 模态窗口 -->
  <a-modal v-model:visible="genDailyVisible" title="生成车次" @ok="handleGenDailyOk"
         :confirm-loading="genDailyLoading" ok-text="确认" cancel-text="取消">
  <a-form :model="formState" :label-col="{span: 4}" :wrapper-col="{ span: 20 }">
    <a-form-item label="日期">
      <a-date-picker v-model:value="formState.date" placeholder="请选择日期"/>
    </a-form-item>
  </a-form>
</a-modal>
</template>

<script setup lang="ts">
import { ref, onMounted, reactive } from 'vue';
import axios from "axios";
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import TrainSelect from '@/components/TrainSelect.vue';
import dayjs from 'dayjs';


//路由对象
const router = useRouter();

//接口
interface DailyTrain {
  id: number,
  date: string,
  code: string,
  type: string,
  start: string,
  startPinyin: string,
  startTime: string,
  end: string,
  endPinyin: string,
  endTime: string,
}

//数据列
const columns = [
{
  title: '日期',
  dataIndex: 'date',
  key: 'date',
},
{
  title: '车次编号',
```

```
    dataIndex: 'code',
    key: 'code',
  },
  {
    title: '车次类型',
    dataIndex: 'type',
    key: 'type',
  },
  {
    title: '始发站',
    dataIndex: 'start',
    key: 'start',
  },
  {
    title: '始发站拼音',
    dataIndex: 'startPinyin',
    key: 'start',
  },
  {
    title: '出发时间',
    dataIndex: 'startTime',
    key: 'startTime',
  },
  {
    title: '终点站',
    dataIndex: 'end',
    key: 'end',
  },
  {
    title: '终点站拼音',
    dataIndex: 'endPinyin',
    key: 'endPinyin',
  },
  {
    title: '到站时间',
    dataIndex: 'endTime',
    key: 'endTime',
  },
  {
    title: '操作',
    dataIndex: 'operation'
  }
];
//数据源
const dataSource = ref<DailyTrain[]>([]);

//--------------模态窗口------------------
const formState = reactive({
  date:''
})
const genDailyVisible = ref(false);
const genDailyLoading = ref(false);

const genTrainCode = () => {
  genDailyVisible.value=true;
}
const handleGenDailyOk = () => {
  //日期格式
  let date = dayjs(formState.date).format("YYYY-MM-DD");
  axios.get(`/daily-train/gen_daily/${date}`).then(()=>{
    message.success('生成成功');
    genDailyVisible.value=false;
  })
}

//----------------------------------------

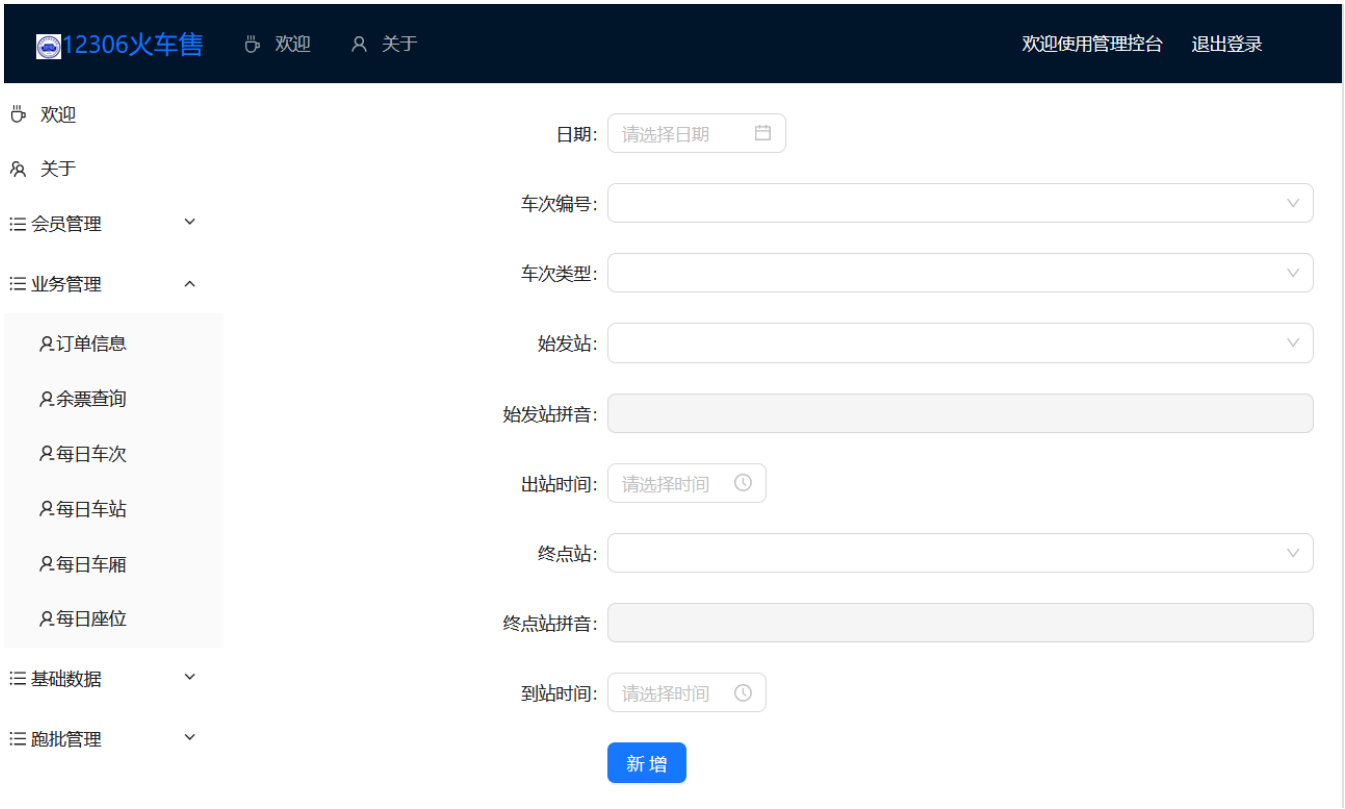
//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/trainDaily/dailyTrain/delete/${id}`,
```

```
        }).then(()=>{
          showData();
          message.success('删除成功');
        })
    }
    //刷新
    const refreshData = () => {
        showData();
    }
    //新增
    const add = () => {
      router.push('/main/daily/dailyTrainSave');
    }
    //编辑
    const edit = (record:DailyTrain) => {
      router.push({
        path: '/main/daily/dailyTrainSave',
        state: {
          params:JSON.stringify(record)
        }
      })
    }
    //--------------分页----------------
    const total = ref(0);
    //分页参数
    const queryParams=ref({
      current:1,
      size:5,
      code:null,
      date:null
    })

    //当前页码变化
    const onChange = (page:number,pageSize:number) => {
      queryParams.value.current = page
      queryParams.value.size = pageSize
      showData()
    };
    //加载数据
    const showData = () => {
      axios({
        method: 'get',
        url: '/trainDaily/dailyTrain/loadPage',
        params: queryParams.value
      }).then(({data})=>{
          total.value = data.data.total;
          dataSource.value = data.data.list;
      })
    }

    //页面加载数据
    onMounted(()=>{
      showData();
    })
</script>
```

9.2.5 前端：DailyTrainSave.vue保存更新




```
<template>
  <a-form
    :model="formState"
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="id"
      name="id"
      hidden="true"
    >
      <a-input v-model:value="formState.id" />
    </a-form-item>
    <a-form-item
      label="日期"
      name="date"
    >
    <a-date-picker v-model:value="formState.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期"/>
    </a-form-item>
    <a-form-item
      label="车次编号"
      name="code"
    >
     <TrainSelect v-model="formState.code" @change="onChangeCode"></TrainSelect>
    </a-form-item>
    <a-form-item
      label="车次类型"
      name="type"
    >
    <a-select v-model:value="formState.type">
      <a-select-option v-for="item in TRAIN_TYPE_ARRAY" :key="item.code" :value="item.code">
      {{item.desc}}
      </a-select-option>
    </a-select>
    </a-form-item>
    <a-form-item
      label="始发站"
      name="start"
    >
      <StationSelect v-model="formState.start"></StationSelect>
```

```
    </a-form-item>
    <a-form-item
      label="始发站拼音"
      name="startPinyin"
      >
      <a-input v-model:value="formState.startPinyin" disabled/>
    </a-form-item>
    <a-form-item
      label="出站时间"
      name="startTime"
    >
    <a-time-picker v-model:value="formState.startTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
    </a-form-item>
    <a-form-item
      label="终点站"
      name="end"
    >
    <StationSelect v-model="formState.end"></StationSelect>
    </a-form-item>
    <a-form-item
    label="终点站拼音"
    name="endPinyin"
  >
    <a-input v-model:value="formState.endPinyin" disabled/>
  </a-form-item>
  <a-form-item
      label="到站时间"
      name="endTime"
    >
    <a-time-picker v-model:value="formState.endTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
    </a-form-item>


    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.id ? '修改' : '新增' }}
      </a-button>
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted,watch } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';
import {pinyin} from 'pinyin-pro';
import StationSelect from '@/components/StationSelect.vue';
import TrainSelect from '@/components/TrainSelect.vue';
import {TRAIN_TYPE_ARRAY} from '@/utils/enum.ts'

//路由对象
const router = useRouter();
 //接口
 interface FormState {
    id: string,
    date: string,
    code: string,
    type: string,
    start: string,
    startPinyin: string,
    startTime: string,
    end: string,
    endPinyin: string,
    endTime: string,
  }
//表单数据
const formState = reactive<FormState>({
  id: '',
  date:'',
  code: '',
  type: '',
  start: '',
```

```
  startPinyin: '',
  startTime: '',
  end: '',
  endPinyin: '',
  endTime: '',
});

//根据车次添加相关信息
const onChangeCode=(train:FormState)=>{//组件的change事件可以获取整行数据
  let t = Tool.copy(train)
  delete t.id
  Object.assign(formState, t)
}

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainDaily/dailyTrain/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainDaily/dailyTrain/update', values).then(() => {
     message.success('更新成功');
  })
}

const onFinish = (values: any) => {
     if(formState.id){
      console.log(values);
      updateData(values);
     }else{
      saveData(values);
     }
     // 刷新列表
     router.push('/main/daily/dailyTrain');
}
const onFinishFailed = (errorInfo: any) => {
  console.log('Failed:', errorInfo);
};
//添加监听拼音
watch(()=>formState.start,()=>{
    if(Tool.isNotEmpty(formState.start)){
      formState.startPinyin = pinyin(formState.start, { toneType: 'none' }).replaceAll(' ','');
    }else{
      formState.startPinyin = '';
    }
})
watch(()=>formState.end,()=>{
    if(Tool.isNotEmpty(formState.end)){
      formState.endPinyin = pinyin(formState.end, { toneType: 'none' }).replaceAll(' ','');
    }else{
      formState.endPinyin = '';
    }
})
</script>
```

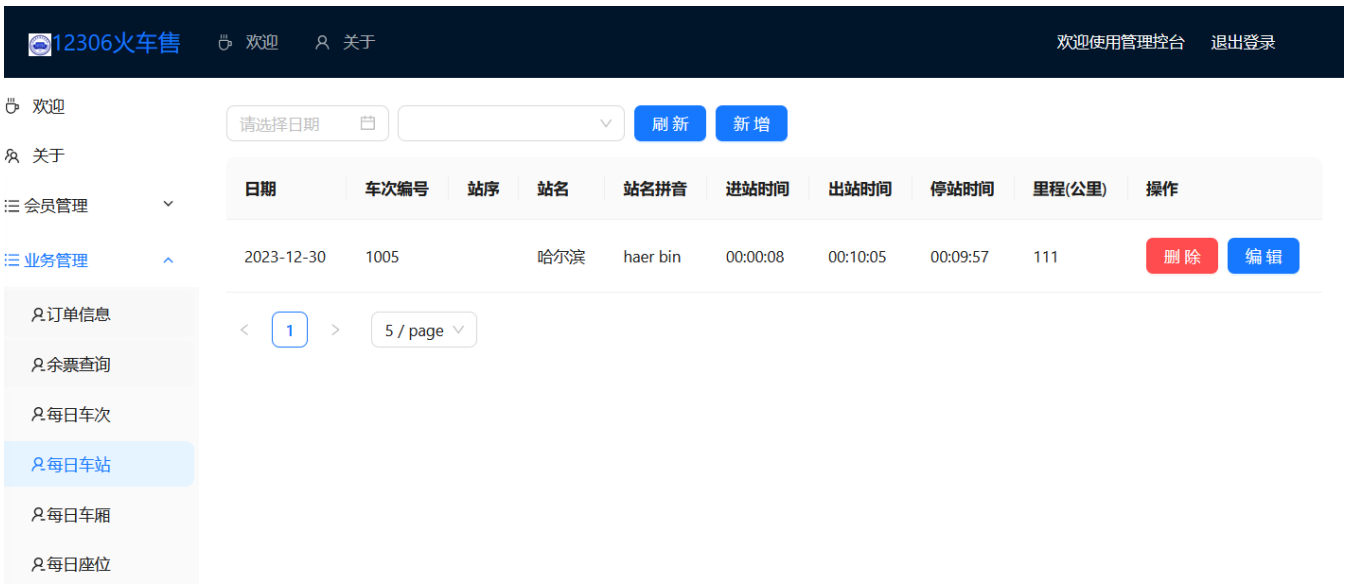
9.3 业务管理-每日车站

9.3.1 数据表

```
CREATE TABLE `daily_train_station` (
  `id` bigint NOT NULL COMMENT 'id',
  `date` date NOT NULL COMMENT '日期',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `index` int NOT NULL COMMENT '站序|第一站是0',
  `name` varchar(20) NOT NULL COMMENT '站名',
  `name_pinyin` varchar(50) NOT NULL COMMENT '站名拼音',
  `in_time` time DEFAULT NULL COMMENT '进站时间',
  `out_time` time DEFAULT NULL COMMENT '出站时间',
  `stop_time` time DEFAULT NULL COMMENT '停站时长',
  `km` decimal(8,2) NOT NULL COMMENT '里程（公里）|从上一站到本站的距离',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `date_train_code_index_unique` (`date`,`train_code`,`index`),
  UNIQUE KEY `date_train_code_name_unique` (`date`,`train_code`,`name`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='每日车站';
```

9.3.2 三层编写：分页查询

9.3.3 前端：DailyTrainStation.vue




```
<template>
  <p>
    <a-space>
      <a-date-picker v-model:value="queryParams.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期" />
      <TrainSelect v-model="queryParams.trainCode"></TrainSelect>
      <a-button type="primary" @click="refreshData">刷新</a-button>
      <a-button type="primary" @click="add">新增</a-button>
    </a-space>
  </p>
  <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
    <template #bodyCell="{column,record}">
       <template v-if="column.dataIndex==='operation'">
        <a-space>
          <a-popconfirm
            v-if="dataSource.length"
            title="是否删除?"
            cancelText="取消"
            okText="确定"
            @confirm="onDelete(record.id)"
            >
            <a-button type="primary" danger>删除</a-button>
          </a-popconfirm>
          <a-button type="primary" @click="edit(record)">编辑</a-button>
        </a-space>
```

```
        </template>
      </template>
    </a-table>
    <br>
    <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import TrainSelect from '@/components/TrainSelect.vue';


//路由对象
const router = useRouter();

//接口
interface DailyTrainStation {
  id: number,
  date:string,
  trainCode: string,
  indexa: string,
  name: string,
  namePinyin: string,
  inTime: string,
  outTime: string,
  stopTime: string,
  km: string,
}

//数据列
const columns = [
{
  title: '日期',
  dataIndex: 'date',
  key: 'date',
},
{
  title: '车次编号',
  dataIndex: 'trainCode',
  key: 'trainCode',
},
{
  title: '站序',
  dataIndex: 'indexa',
  key: 'indexa',
},
{
  title: '站名',
  dataIndex: 'name',
  key: 'name',
},
{
  title: '站名拼音',
  dataIndex: 'namePinyin',
  key: 'namePinyin',
},
{
  title: '进站时间',
  dataIndex: 'inTime',
  key: 'inTime',
},
{
  title: '出站时间',
  dataIndex: 'outTime',
  key: 'outTime',
},
{
  title: '停站时间',
  dataIndex: 'stopTime',
```

```
  key: 'stopTime',
},
{
  title: '里程(公里)',
  dataIndex: 'km',
  key: 'km',
},
{
  title: '操作',
  dataIndex: 'operation'
}
];
//数据源
const dataSource = ref<DailyTrainStation[]>([]);

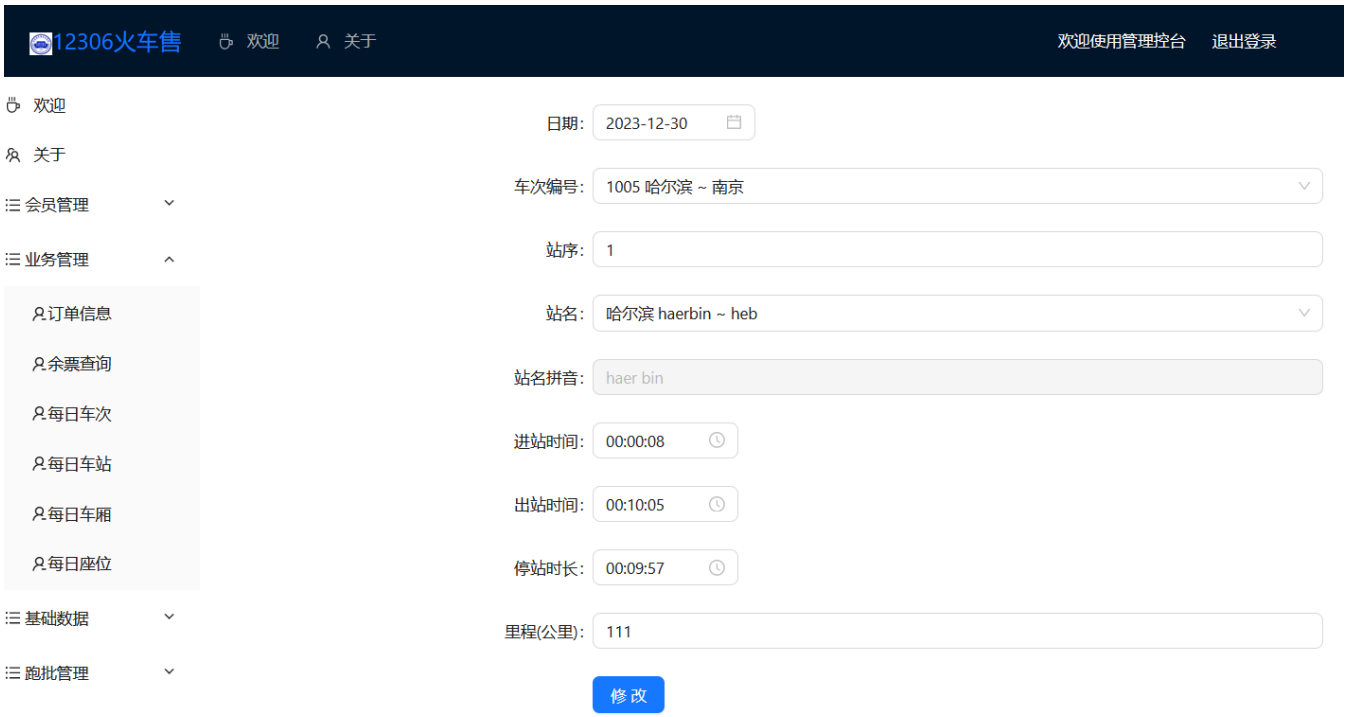
//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/trainDaily/dailyTrainStation/delete/${id}`,
    }).then(()=>{
      showData();
      message.success('删除成功');
    })
}
//刷新
const refreshData = () => {
    showData();
}
//新增
const add = () => {
  router.push('/main/daily/dailyTrainStationSave');
}
//编辑
const edit = (record:DailyTrainStation) => {
  router.push({
    path: '/main/daily/dailyTrainStationSave',
    state: {
      params:JSON.stringify(record)
    }
  })
}
//--------------分页----------------
const total = ref(0);
//分页参数
const queryParams=ref({
  current:1,
  size:5,
  date:null,
  trainCode:null
})

//当前页码变化
const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
//加载数据
const showData = () => {
  axios({
    method: 'get',
    url: '/trainDaily/dailyTrainStation/loadPage',
    params: queryParams.value
  }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
}

//页面加载数据
onMounted(()=>{
```

```
  showData();
})
</script>
```

9.3.4 前端：DailyTrainStationSave.vue




```
<template>
  <a-form
    :model="formState"
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="id"
      name="id"
      hidden="true"
    >
      <a-input v-model:value="formState.id" />
    </a-form-item>
    <a-form-item
      label="日期"
      name="date"
    >
    <a-date-picker v-model:value="formState.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期"/>
    </a-form-item>
    <a-form-item
      label="车次编号"
      name="trainCode"
    >
      <TrainSelect v-model="formState.trainCode"></TrainSelect>
    </a-form-item>
    <a-form-item
      label="站序"
      name="index"
    >
    <a-input v-model:value="formState.index"/>
    </a-form-item>
    <a-form-item
      label="站名"
      name="name"
    >
      <StationSelect v-model="formState.name"></StationSelect>
```

```
    </a-form-item>
    <a-form-item
      label="站名拼音"
      name="namePinyin"
      >
      <a-input v-model:value="formState.namePinyin" disabled/>
    </a-form-item>
    <a-form-item
      label="进站时间"
      name="inTime"
    >
    <a-time-picker v-model:value="formState.inTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
    </a-form-item>
    <a-form-item
      label="出站时间"
      name="outTime"
    >
    <a-time-picker v-model:value="formState.outTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
    </a-form-item>
    <a-form-item
    label="停站时长"
    name="stopTime"
  >
  <a-time-picker v-model:value="formState.stopTime" valueFormat="HH:mm:ss" placeholder="请选择时间" />
  </a-form-item>
  <a-form-item
      label="里程(公里)"
      name="km"
    >
    <a-input v-model:value="formState.km" />
    </a-form-item>

    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.id ? '修改' : '新增' }}
      </a-button>
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted,watch } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';
import {pinyin} from 'pinyin-pro';
import TrainSelect from '@/components/TrainSelect.vue';
import StationSelect from '@/components/StationSelect.vue';
import dayjs from 'dayjs';

//路由对象
const router = useRouter();
 //接口
 interface FormState {
  id: string,
  date:string,
  trainCode: string,
  index: string,
  name: string,
  namePinyin: string,
  inTime: string,
  outTime: string,
  stopTime: string,
  km: string,
  }
//表单数据
const formState = reactive<FormState>({
  id: '',
  date:'',
  trainCode: '',
  index: '',
  name: '',
  namePinyin: '',
```

```
  inTime: '',
  outTime: '',
  stopTime: '',
  km: '',
});

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainDaily/dailyTrainStation/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainDaily/dailyTrainStation/update', values).then(() => {
     message.success('更新成功');
  })
}

const onFinish = (values: any) => {
     if(formState.id){
      console.log(values);
      updateData(values);
     }else{
      saveData(values);
     }
     // 刷新列表
     router.push('/main/daily/dailyTrainStation');
}
const onFinishFailed = (errorInfo: any) => {
  console.log('Failed:', errorInfo);
};
//添加监听拼音
watch(()=>formState.name,()=>{
    if(Tool.isNotEmpty(formState.name)){
      formState.namePinyin = pinyin(formState.name, { toneType: 'none' }).replace(' ','');
    }else{
      formState.namePinyin = '';
    }
}, { immediate: true })

//计算进站停车时长
watch(()=>formState.inTime,()=>{
  let diff = dayjs(formState.outTime,'HH:mm:ss').diff(dayjs(formState.inTime,'HH:mm:ss'),'seconds');
  formState.stopTime = dayjs('00:00:00','HH:mm:ss').second(diff).format('HH:mm:ss');
}, { immediate: true })

//计算出站停车时长
watch(()=>formState.outTime,()=>{
  let diff = dayjs(formState.outTime,'HH:mm:ss').diff(dayjs(formState.inTime,'HH:mm:ss'),'seconds');
  formState.stopTime = dayjs('00:00:00','HH:mm:ss').second(diff).format('HH:mm:ss');
}, { immediate: true })

</script>
```

9.4 业务管理-每日车厢

9.4.1 数据表

```
CREATE TABLE `daily_train_carriage` (
  `id` bigint NOT NULL COMMENT 'id',
  `date` date NOT NULL COMMENT '日期',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `index` int NOT NULL COMMENT '箱序',
  `seat_type` char(1) NOT NULL COMMENT '座位类型|枚举[SeatTypeEnum]',
  `seat_count` int NOT NULL COMMENT '座位数',
  `row_count` int NOT NULL COMMENT '排数',
  `col_count` int NOT NULL COMMENT '列数',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `date_train_code_index_unique` (`date`,`train_code`,`index`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='每日车厢';
```

9.4.2 三层编写：分页

- service

```
public interface DailyTrainCarriageService extends IService<DailyTrainCarriageEntity> {

    Page<DailyTrainCarriageEntity> getDailyTrainCarriageList(Page page, DailyTrainCarriageEntity
dailyTrainCarriageEntity);

    void saveCarriage(DailyTrainCarriageEntity dailyTrainCarriageEntity);

    //按车次获取车厢
    List<DailyTrainCarriageEntity> loadByTrainCode(String trainCode);
}
```

- serviceImpl

```
package com.hxx.service.impl;

import cn.hutool.core.bean.BeanUtil;
import cn.hutool.core.util.ObjUtil;
import com.baomidou.mybatisplus.core.conditions.query.LambdaQueryWrapper;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.hxx.entity.DailyTrainCarriageEntity;
import com.hxx.entity.DailyTrainStationEntity;
import com.hxx.enums.SeatColEnum;
import com.hxx.mapper.DailyTrainCarriageMapper;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;

@Service
public class DailyTrainCarriageServiceImpl extends ServiceImpl<DailyTrainCarriageMapper,
DailyTrainCarriageEntity>
implements com.hxx.service.DailyTrainCarriageService{

    @Autowired
    private DailyTrainCarriageMapper dailyTrainCarriageMapper;
    @Override
    public Page<DailyTrainCarriageEntity> getDailyTrainCarriageList(Page page, DailyTrainCarriageEntity
dailyTrainCarriageEntity) {
        return dailyTrainCarriageMapper.selectPage(page, new
LambdaQueryWrapper<DailyTrainCarriageEntity>()
                .eq(dailyTrainCarriageEntity.getTrainCode() != null,
DailyTrainCarriageEntity::getTrainCode, dailyTrainCarriageEntity.getTrainCode())
                .eq(dailyTrainCarriageEntity.getDate() != null, DailyTrainCarriageEntity::getDate,
dailyTrainCarriageEntity.getDate())
        );
    }
```

```
    @Override
    public List<DailyTrainCarriageEntity> loadByTrainCode(String trainCode) {
        return dailyTrainCarriageMapper.selectList(
            new LambdaQueryWrapper<DailyTrainCarriageEntity>()
                .eq(DailyTrainCarriageEntity::getTrainCode, trainCode));
    }

    /**
     * 每天每车次的座位是唯一的
     * @param entity
     */
    @Override
    public void saveCarriage(DailyTrainCarriageEntity entity) {
        //计算列数和总座位数
        List<SeatColEnum> seatColEnums = SeatColEnum.getColsByType(entity.getSeatType());
        entity.setColCount(seatColEnums.size());
        entity.setSeatCount(entity.getColCount() * entity.getRowCount());
        //对拷
        DailyTrainCarriageEntity trainCarriageEntity = BeanUtil.copyProperties(entity,
DailyTrainCarriageEntity.class);
        //判断
        if (ObjUtil.isNull(trainCarriageEntity.getId())) {
            if (ObjUtil.isNotNull(checkUnique(entity.getTrainCode(), entity.getIndex().toString()))) {
                throw new RuntimeException("该车次编号已存在,不是唯一");
            }
            dailyTrainCarriageMapper.insert(trainCarriageEntity);
        }
    }

    //检查车次编号是否唯一
    private DailyTrainCarriageEntity checkUnique(String trainCode, String index) {
        return dailyTrainCarriageMapper.selectOne(new LambdaQueryWrapper<DailyTrainCarriageEntity>()
                .eq(DailyTrainCarriageEntity::getTrainCode, trainCode)
                .eq(DailyTrainCarriageEntity::getIndex, index));
    }
}
```

- controller

```
package com.hxx.controller;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.hxx.entity.DailyTrainCarriageEntity;
import com.hxx.service.DailyTrainCarriageService;
import com.hxx.util.R;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/dailyTrainCarriage")
public class DailyTrainCarriageController {
    @Autowired
    private DailyTrainCarriageService dailyTrainCarriageService;

    @GetMapping("loadPage")
    public R loadPage(Page page, DailyTrainCarriageEntity dailyTrainCarriageEntity) {
        dailyTrainCarriageService.getDailyTrainCarriageList(page, dailyTrainCarriageEntity);
        return R.ok().data("list", page.getRecords()).data("total", page.getTotal());
    }

    @PostMapping("save")
    public R save(@RequestBody DailyTrainCarriageEntity dailyTrainCarriageEntity) {
        dailyTrainCarriageService.saveCarriage(dailyTrainCarriageEntity);
        return R.ok();
    }

    @PutMapping("update")
    public R update(@RequestBody DailyTrainCarriageEntity dailyTrainCarriageEntity) {
        dailyTrainCarriageService.updateById(dailyTrainCarriageEntity);
        return R.ok();
```

```
    }

    @DeleteMapping("delete/{id}")
    public R delete(@PathVariable Long id) {
        dailyTrainCarriageService.removeById(id);
        return R.ok();
    }

    @GetMapping("load/{trainCode}")
    public R load(@PathVariable String trainCode) {
        return R.ok().data("list", dailyTrainCarriageService.loadByTrainCode(trainCode));
    }
}
```

9.4.3 前端：DailyTrainCarriage.vue




```
<template>
    <p>
      <a-space>
        <a-date-picker v-model:value="queryParams.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期" />
        <TrainSelect v-model="queryParams.code" style="width: 200px;"></TrainSelect>
        <a-button type="primary" @click="refreshData">刷新</a-button>
        <a-button type="primary" @click="add">添加</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{column,record}">
         <template v-if="column.dataIndex==='operation'">
          <a-space>
            <a-popconfirm
              v-if="dataSource.length"
              title="是否删除?"
              cancelText="取消"
              okText="确定"
              @confirm="onDelete(record.id)"
              >
              <a-button type="primary" danger>删除</a-button>
            </a-popconfirm>
            <a-button type="primary" @click="edit(record)">编辑</a-button>
          </a-space>
         </template>
      </template>
    </a-table>
    <br>
    <TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
  </template>

  <script setup lang="ts">
  import { ref, onMounted } from 'vue';
  import axios from "axios";
```

```
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import TrainSelect from '@/components/TrainSelect.vue';


//路由对象
const router = useRouter();

//接口
interface TrainCarriage {
  id: number,
  date: string,
  trainCode: string,
  index: string,
  seatType: string,
  seatCount: string,
  rowCount: string,
  colCount: string,
}

//数据列
const columns = [
{
  title: '日期',
  dataIndex: 'date',
  key: 'date',
},
{
  title: '车次编号',
  dataIndex: 'trainCode',
  key: 'trainCode',
},
{
  title: '车厢号',
  dataIndex: 'index',
  key: 'index',
},
{
  title: '座位类型',
  dataIndex: 'seatType',
  key: 'seatType',
},
{
  title: '座位数',
  dataIndex: 'seatCount',
  key: 'seatCount',
},
{
  title: '排数',
  dataIndex: 'rowCount',
  key: 'rowCount',
},
{
  title: '列数',
  dataIndex: 'colCount',
  key: 'colCount',
},
{
  title: '操作',
  dataIndex: 'operation'
}
];
//数据源
const dataSource = ref<TrainCarriage[]>([]);

//删除
const onDelete = (id:number) => {
    axios({
      method: 'delete',
      url: `/trainDaily/dailyTrainCarriage/delete/${id}`,
    }).then(()=>{
      showData();
```

```
        message.success('删除成功');
      })
  }
  //刷新
  const refreshData = () => {
      showData();
  }
  //新增
  const add = () => {
    router.push('/main/daily/DailyTrainCarriageSave');
  }
  //编辑
  const edit = (record:TrainCarriage) => {
    router.push({
      path: '/main/daily/DailyTrainCarriageSave',
      state: {
        params:JSON.stringify(record)
      }
    })
  }
  //--------------分页----------------
  const total = ref(0);
  //分页参数
  const queryParams=ref({
    current:1,
    size:5,
    date:null,
    code:''
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainDaily/dailyTrainCarriage/loadPage',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })
</script>
```

9.4.4 前端：DailyTrainCarriageSave.vue




```
<template>
  <a-form
    :model="formState"
    name="basic"
    :label-col="{ span: 8 }"
    :wrapper-col="{ span: 16 }"
    autocomplete="off"
    @finish="onFinish"
    @finishFailed="onFinishFailed"
  >
    <a-form-item
      label="id"
      name="id"
      hidden="true"
    >
      <a-input v-model:value="formState.id" />
    </a-form-item>
  <a-form-item
    label="日期"
    name="date"
  >
  <a-date-picker v-model:value="formState.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期"/>
  </a-form-item>
    <a-form-item
      label="车次编号"
      name="trainCode"
    >
      <TrainSelect v-model:value="formState.trainCode" />
    </a-form-item>
    <a-form-item
      label="车厢号"
      name="index"
    >
      <a-input v-model:value="formState.index" />
    </a-form-item>
    <a-form-item
      label="座位类型"
      name="seatType"
    >
    <a-select v-model:value="formState.seatType">
      <a-select-option v-for="item in SEAT_TYPE_ARRAY" :key="item.code" :value="item.code">
      {{item.desc}}
      </a-select-option>
    </a-select>
    </a-form-item>
    <a-form-item
      label="排数"
      name="rowCount"
    >
      <a-input-number v-model:value="formState.rowCount" />
    </a-form-item>
```

```
    <a-form-item :wrapper-col="{ offset: 8, span: 16 }">
      <a-button type="primary" html-type="submit">
        {{ formState.id ? '修改' : '新增' }}
      </a-button>
    </a-form-item>
  </a-form>
</template>
<script lang="ts" setup>
import { reactive,onMounted } from 'vue';
import { message } from 'ant-design-vue';
import { useRouter } from 'vue-router';
import axios from 'axios';
import TrainSelect from '@/components/TrainSelect.vue';
import {SEAT_TYPE_ARRAY} from '@/utils/enum.ts'

//路由对象
const router = useRouter();
 //接口
 interface FormState {
  id: string,
  date: string,
  trainCode: string,
  index: string,
  seatType: string,
  seatCount: number,
  rowCount: number,
  colCount: number,
  }
//表单数据
const formState = reactive<FormState>({
  id: "",
  date: "",
  trainCode: "",
  index: "",
  seatType: "",
  seatCount: 0,
  rowCount: 0,
  colCount: 0,
});

//获取编辑的数据
onMounted(() => {
  const record = history.state.params
  //判断
  if(record){
    //回填数据
    Object.assign(formState, JSON.parse(record))
  }
})

//新增
const saveData=(values:any)=>{
  axios.post('/trainDaily/dailyTrainCarriage/save', values).then(() => {
     message.success('保存成功');
  })
}
//修改
const updateData=(values:any)=>{
  axios.put('/trainDaily/dailyTrainCarriage/update', values).then(() => {
     message.success('更新成功');
  })
}

const onFinish = (values: any) => {
     if(formState.id){
      updateData(values);
     }else{
      saveData(values);
     }
     // 刷新列表
     router.push('/main/daily/dailyTrainCarriage');
}
```

```
const onFinishFailed = (errorInfo: any) => {
  console.log('Failed:', errorInfo);
};
</script>
```

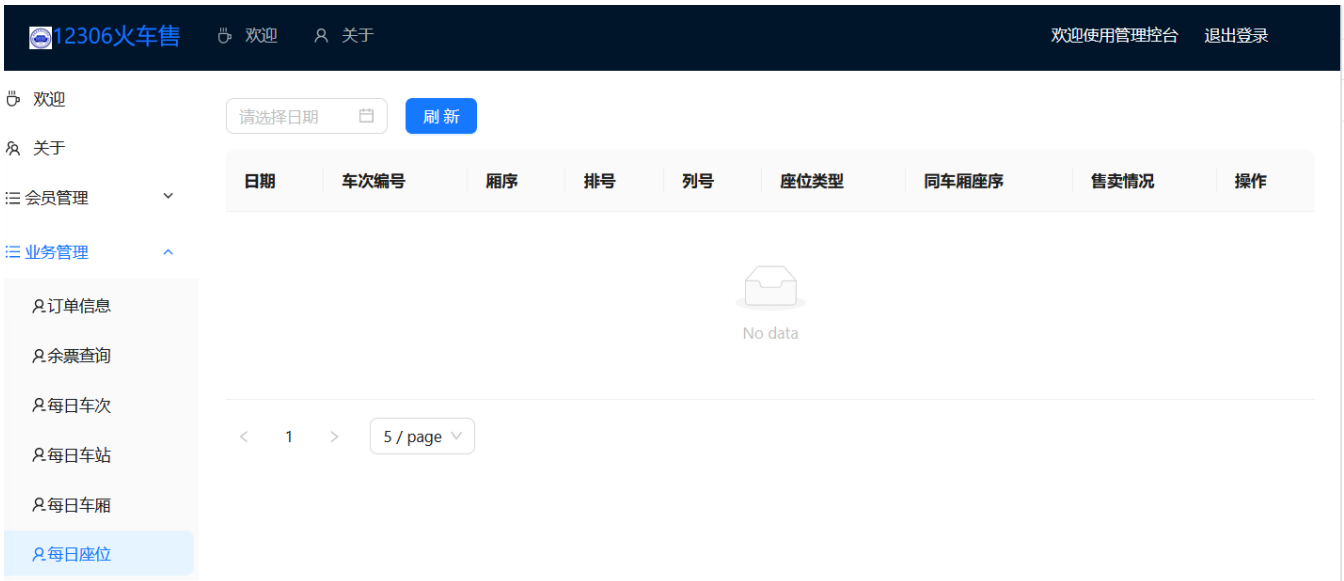
9.5 业务管理-每日座位

9.5.1 数据表

```
CREATE TABLE `daily_train_seat` (
  `id` bigint NOT NULL COMMENT 'id',
  `date` date NOT NULL COMMENT '日期',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `carriage_index` int NOT NULL COMMENT '箱序',
  `row` char(2) NOT NULL COMMENT '排号|01, 02',
  `col` char(1) NOT NULL COMMENT '列号|枚举[SeatColEnum]',
  `seat_type` char(1) NOT NULL COMMENT '座位类型|枚举[SeatTypeEnum]',
  `carriage_seat_index` int NOT NULL COMMENT '同车箱座序',
  `sell` varchar(50) NOT NULL COMMENT '售卖情况|将经过的车站用01拼接，0表示可卖，1表示已卖',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='每日座位';
```

9.5.2 三层分页

9.5.3 前端：DailyTrainSeat.vue




```
<template>
    <p>
      <a-space>
        <a-date-picker v-model:value="queryParams.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期" />
        <TrainSelect v-model="queryParams.code" ></TrainSelect>
        <a-button type="primary" @click="refreshData">刷新</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false" />
    <br><TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
  </template>

  <script setup lang="ts">
  import { ref, onMounted } from 'vue';
  import axios from "axios";

  //接口
```

```
interface TrainSeat {
  id: number,
  date: string,
  sell:string,
  trainCode: string,
  carriageIndex: string,
  row: string,
  col: string,
  seatType: string,
  carriageSeatIndex: string,
}

//数据列
const columns = [
{
  title: '日期',
  dataIndex: 'date',
  key: 'date',
},
{
  title: '车次编号',
  dataIndex: 'trainCode',
  key: 'trainCode',
},
{
  title: '厢序',
  dataIndex: 'carriageIndex',
  key: 'carriageIndex',
},
{
  title: '排号',
  dataIndex: 'row',
  key: 'row',
},
{
  title: '列号',
  dataIndex: 'col',
  key: 'col',
},
{
  title: '座位类型',
  dataIndex: 'seatType',
  key: 'seatType',
},
{
  title: '同车厢座序',
  dataIndex: 'carriageSeatIndex',
  key: 'carriageSeatIndex',
}
,
{
  title: '售卖情况',
  dataIndex: 'sell',
  key: 'sell',
},
{
  title: '操作',
  dataIndex: 'operation'
}
];
//数据源
const dataSource = ref<TrainSeat[]>([]);

//刷新
const refreshData = () => {
    showData();
}
//--------------分页----------------
const total = ref(0);
//分页参数
const queryParams=ref({
  current:1,
```

```
    size:5,
    date:null,
    code:null
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainDaily/dailyTrainSeat/loadPage',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })
</script>
```

十、跑批

10.1 定时任务

10.1.1 springboot自带的定时任务案例

- 启动项目自动调用

```
@Component
@EnableScheduling
public class SpringBootTest {

    private Logger logger = LoggerFactory.getLogger(SpringBootTest.class);
    @Scheduled(cron = "0/5 * * * * ?")
    public void test(){
        logger.info("定时任务执行,每隔5秒调用一次: {}", LocalDateTime.now());
    }
}
```



10.1.2 使用quartz完成任务调度

- pom依赖

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-quartz</artifactId>
</dependency>
```

- 编写任务类

```
public class SpringBootTest implements Job {
    private Logger logger = LoggerFactory.getLogger(SpringBootTest.class);
    @Override
    public void execute(JobExecutionContext jobExecutionContext) throws JobExecutionException {
        logger.info("任务调度：{}", LocalDateTime.now());
    }
}
```

- 编写任务调度类

```
@Configuration
public class QuartzConfig {

    /**
     * 定义一个Spring Batch作业的细节信息。
     *
     * @return JobDetail对象，包含了作业的名称、组名以及其他配置信息。
     *
     * 该方法使用JobBuilder构建一个新的作业细节对象。作业被命名为"job1"，属于"group1"组。
     * 通过调用storeDurably方法，配置作业在调度器中持久化存储，这意味着作业信息会在调度器重启后仍然可用。
     * 使用@Bean注解表明该方法返回的对象将被注册为Spring Bean，以便在应用程序中使用。
     */
    @Bean
    public JobDetail getJobDetail(){
        return JobBuilder.newJob(SpringBootTest.class)
                .withIdentity("job1", "group1")
                .storeDurably()
                .build();
    }


    /**
     * 创建一个Quartz触发器。
     *
     * 本方法旨在通过Spring的@Bean注解配置一个Quartz触发器，该触发器被设计为立即启动，并按照预设的CRON表达式周期性地触发作业。
     * 使用了CronScheduleBuilder来定义触发器的调度策略，即按照"0/5 * * * * ?"的CRON表达式每5秒触发一次。
     *
     * @return 返回构建完成的Quartz触发器实例。
     */
    @Bean
    public org.quartz.Trigger getTrigger(){
        // 创建一个新的触发器，并指定其适用于由getJobDetail()方法返回的作业。
        // 使用withIdentity为触发器指定一个唯一标识和组名。
        // 设置触发器立即启动。
        // 定义触发器的调度策略为基于CRON表达式的调度。
        // 最终构建并返回触发器实例。
        return TriggerBuilder.newTrigger()
                .forJob(getJobDetail())
                .withIdentity("trigger1", "group1")
                .startNow()
                .withSchedule(CronScheduleBuilder.cronSchedule("0/5 * * * * ?"))
                .build();
    }
}
```

10.1.3 任务调度并发设置

- 如果当前任务没有执行完不能开始下一次的任务
- 修改任务类：@DisallowConcurrentExecution

```
@DisallowConcurrentExecution
public class SpringBootTest implements Job {
    private Logger logger = LoggerFactory.getLogger(SpringBootTest.class);
    @Override
    public void execute(JobExecutionContext jobExecutionContext) throws JobExecutionException {
        logger.info("任务调度开始：{}", LocalDateTime.now());
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        }
        logger.info("任务调度结束：{}", LocalDateTime.now());
    }
}
```

10.2 定时生成每日车次数据

10.2.1 创建模块train-batch

- pom依赖

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-
v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>com.hxx</groupId>
        <artifactId>train12306</artifactId>
        <version>1.0-SNAPSHOT</version>
    </parent>
    <artifactId>train-batch</artifactId>
    <name>Archetype - train-batch</name>
    <url>http://maven.apache.org</url>

    <dependencies>
        <dependency>
            <groupId>com.hxx</groupId>
            <artifactId>train-common</artifactId>
            <version>1.0-SNAPSHOT</version>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-quartz</artifactId>
        </dependency>
    </dependencies>
</project>
```



- 配置文件

```yaml
server:
  port: 8005
spring:
  application:
    name: trainBatch
  cloud:
    nacos:
      config:
        server-addr: localhost:8848
        username: nacos
        password: nacos
  datasource:
    url: jdbc:mysql://127.0.0.1:3306/train?useUnicode=true&characterEncoding=utf-
8&useSSL=false&serverTimezone=Asia/Shanghai
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: root
    password: root
  jackson:
    date-format: YYYY-MM-dd
```

10.2.2 编写任务调度类

- 实体类: 任务请求

```java
package com.hxx.domain;

import lombok.Data;
import lombok.ToString;

@Data
@ToString
public class CronJobReq {
    //任务分组
    private String group;
    //任务名称
    private String name;
    //任务描述
    private String description;
    //cron表达式
    private String cronExpression;
}
```

- 实体类: 任务响应

```java
package com.hxx.domain;

import com.fasterxml.jackson.annotation.JsonFormat;
```

```
import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.Data;
import lombok.ToString;

import java.util.Date;

@JsonInclude(JsonInclude.Include.NON_NULL)//忽略为null的字段
@Data
@ToString
public class CronJobResp {
    //任务分组
    private String group;
    //任务名称
    private String name;
    //任务描述
    private String description;
    //cron表达式
    private String cronExpression;
    //状态
    private String state;
    //下次执行时间
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "GMT+8")
    private Date nextFireTime;
    //上次执行时间
    @JsonFormat(pattern = "yyyy-MM-dd HH:mm:ss", timezone = "GMT+8")
    private Date prevFireTime;
}
```

- 把任务调度数据集成数据库存储
  - 编写job工厂类

```
package com.hxx.config;

import org.springframework.beans.factory.config.AutowireCapableBeanFactory;
import org.springframework.scheduling.quartz.SpringBeanJobFactory;
import org.springframework.stereotype.Component;
import org.quartz.spi.TriggerFiredBundle;

@Component
public class MyJobFactory extends SpringBeanJobFactory {

    // 注入AutowireCapableBeanFactory
    @Autowired
    private AutowireCapableBeanFactory beanFactory;

    /**
     * 创建作业实例。
     * 重写了父类方法，以便在Spring上下文中初始化作业实例。
     *
     * @param bundle 触发器触发时携带的bundle，包含作业和触发器的相关信息。
     * @return 创建的作业实例。
     * @throws Exception 如果创建作业实例过程中出现异常。
     */
    // 重写createJobInstance方法
    @Override
    protected Object createJobInstance(TriggerFiredBundle bundle) throws Exception {
        // 调用父类方法创建作业实例。
        // 调用父类的方法
        Object jobInstance = super.createJobInstance(bundle);
        // 将作业实例注入到Spring上下文中，自动装配其依赖。
        // 将jobInstance添加到spring容器中
        beanFactory.autowireBean(jobInstance);
        return jobInstance;
    }
}
```

  - 任务调度配置类

```
package com.hxx.config;
```

```
import jakarta.annotation.Resource;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.quartz.SchedulerFactoryBean;

import javax.sql.DataSource;

@Configuration
public class SchedulerConfig {
    @Resource
    private MyJobFactory myJobFactory;

    /**
     * 配置Quartz调度器的工厂 bean。
     *
     * @param dataSource 用于Quartz调度器的数据库源，指定特定的数据库源以供Quartz使用。
     * @return SchedulerFactoryBean 实例，配置了自定义的工作单元工厂和数据源。
     *
     * @Bean 注解表明该方法将返回一个被Spring管理的bean实例。
     */
    @Bean
    public SchedulerFactoryBean schedulerFactoryBean(@Qualifier("dataSource")DataSource dataSource) {
        SchedulerFactoryBean schedulerFactoryBean = new SchedulerFactoryBean();
        // 设置自定义的工作单元工厂，以支持Spring管理的bean作为任务执行。
        schedulerFactoryBean.setJobFactory(myJobFactory);
        // 配置数据源，使Quartz能够使用数据库存储作业和触发器信息。
        schedulerFactoryBean.setDataSource(dataSource);
        // 设置调度器在启动时的延迟时间，单位为秒，这里设置为2秒后启动。
        schedulerFactoryBean.setStartupDelay(2);
        return schedulerFactoryBean;
    }

}
```

- 编写控制层

```
package com.hxx.controller;

import com.hxx.domain.CronJobReq;
import com.hxx.domain.CronJobResp;
import com.hxx.util.R;
import org.quartz.*;
import org.quartz.impl.matchers.GroupMatcher;
import org.quartz.impl.triggers.CronTriggerImpl;
import org.slf4j.Logger;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.scheduling.SchedulingException;
import org.springframework.scheduling.quartz.SchedulerFactoryBean;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

@RestController
public class JobController {

    private Logger logger = org.slf4j.LoggerFactory.getLogger(JobController.class);

    @Autowired
    private SchedulerFactoryBean schedulerFactoryBean;

    /**
     * 启动指定的定时任务。
     *
     * 通过接收CronJobReq对象作为请求体，从中获取任务名称和组别，
     * 然后触发对应的任务执行。此方法用于在接收到请求时，立即启动一个
```

```
     * 已经配置好的定时任务，而不需要等待下一次触发时间。
     *
     * @param cronJobReq 包含任务名称和组别的请求对象。
     * @return 返回一个表示操作成功的响应对象。
     * @throws SchedulerException 如果触发任务时发生错误。
     */
    /**
     * 启动任务
     */
    @RequestMapping("/run")
    public R run(@RequestBody CronJobReq cronJobReq) throws SchedulerException {
        logger.info("启动任务");
        // 从请求对象中获取任务名称和组别
        String jobReqName = cronJobReq.getName();
        String jobReqGroup = cronJobReq.getGroup();
        // 触发指定名称和组别的任务
        schedulerFactoryBean.getScheduler().triggerJob(JobKey.jobKey(jobReqName, jobReqGroup));
        // 返回操作成功的响应对象
        return R.ok();
    }

    /**
     * 创建一个定时任务。
     * 通过@RequestBody注解将CronJobReq对象绑定到方法参数上，使得可以从HTTP请求体中直接获取到定时任务的配置信息。
     *
     * @param cronJobReq 包含定时任务详细信息的请求对象，如任务名称、分组、CRON表达式和描述。
     * @return 返回一个表示操作结果的对象，此处为R.ok()，表示操作成功。
     * @throws SchedulerException 如果调度器操作失败，则抛出此异常。
     */
    /**
     * 创建定时任务
     */
    @RequestMapping("/add")
    public R add(@RequestBody CronJobReq cronJobReq) throws SchedulerException {
        // 从请求对象中提取定时任务的名称、分组、CRON表达式和描述信息
        // 从请求对象中获取任务名称和组别
        String jobReqName = cronJobReq.getName();
        String jobReqGroup = cronJobReq.getGroup();
        String cronExpression = cronJobReq.getCronExpression();
        String description = cronJobReq.getDescription();
        // 记录创建定时任务的日志信息
        logger.info("创建定时任务:{},{},{},{}", jobReqName, jobReqGroup, cronExpression, description);

        try {
            // 获取调度器实例
            var scheduler = schedulerFactoryBean.getScheduler();
            // 启动调度器
            scheduler.start();
            // 根据任务名称和分组创建JobDetail对象，指定任务执行的类通过Class.forName动态获取
            JobDetail jobDetail = JobBuilder.newJob((Class<? extends Job>) Class.forName(jobReqName))
                    .withIdentity(jobReqName, jobReqGroup).build();
            // 根据CRON表达式创建CronScheduleBuilder
            CronScheduleBuilder scheduleBuilder = CronScheduleBuilder.cronSchedule(cronExpression);
            // 根据任务名称、分组、调度策略和描述创建Trigger对象
            CronTrigger trigger = TriggerBuilder.newTrigger().withIdentity(jobReqName, jobReqGroup)
                    .withSchedule(scheduleBuilder).withDescription(description).build();
            // 将JobDetail和Trigger注册到调度器中
            scheduler.scheduleJob(jobDetail, trigger);
        } catch (ClassNotFoundException e) {
            // 如果任务执行类找不到，则打印堆栈跟踪
            e.printStackTrace();
        } catch (Exception e) {
            // 其他异常处理
            e.printStackTrace();
        }
        // 返回操作成功的响应
        return R.ok();
    }

    /**
     * 暂停任务
     */
```

```
    @RequestMapping("/pause")
    public R pause(@RequestBody CronJobReq cronJobReq) throws SchedulerException {
        // 从请求对象中获取任务名称和组别
        String jobReqName = cronJobReq.getName();
        String jobReqGroup = cronJobReq.getGroup();
        logger.info("暂停任务:{},{}", jobReqName, jobReqGroup);
        // 暂停指定名称和组别的任务
        schedulerFactoryBean.getScheduler().pauseJob(JobKey.jobKey(jobReqName, jobReqGroup));
        logger.info("暂停任务成功");
        // 返回操作成功的响应对象
        return R.ok();
    }

    /**
     * 恢复任务
     */
    @RequestMapping("/resume")
    public R resume(@RequestBody CronJobReq cronJobReq) throws SchedulerException {
        // 从请求对象中获取任务名称和组别
        String jobReqName = cronJobReq.getName();
        String jobReqGroup = cronJobReq.getGroup();
        logger.info("恢复任务:{},{}", jobReqName, jobReqGroup);
        // 恢复指定名称和组别的任务
        schedulerFactoryBean.getScheduler().resumeJob(JobKey.jobKey(jobReqName, jobReqGroup));
        logger.info("恢复任务成功");
        return R.ok();
    }

    /**
     * 更新任务调度
     * 通过接收CronJobReq对象，更新已存在的任务的调度信息，包括cron表达式和描述。
     * 如果任务不存在，则抛出SchedulerException。
     *
     * @param cronJobReq 包含任务名称、组别、cron表达式和描述的请求对象。
     * @return 返回一个表示操作结果的对象。
     * @throws SchedulerException 如果任务不存在或者调度器操作失败，则抛出此异常。
     */
    /**
     * 更新任务
     */
    @RequestMapping("/reschedule")
    public R reschedule(@RequestBody CronJobReq cronJobReq) throws SchedulerException {
        // 从请求对象中提取任务名称、组别、cron表达式和描述
        // 从请求对象中获取任务名称和组别
        String jobReqName = cronJobReq.getName();
        String jobReqGroup = cronJobReq.getGroup();
        String cronExpression = cronJobReq.getCronExpression();
        String description = cronJobReq.getDescription();
        // 记录更新任务的信息
        logger.info("更新任务:{},{},{}", jobReqName, jobReqGroup, cronExpression);
        // 获取调度器实例
        // 获取调度器实例
        var scheduler = schedulerFactoryBean.getScheduler();
        // 根据任务名称和组别创建TriggerKey
        TriggerKey triggerKey = TriggerKey.triggerKey(jobReqName, jobReqGroup);
        try {
            // 创建基于cron表达式的调度构建器
            CronScheduleBuilder scheduleBuilder = CronScheduleBuilder.cronSchedule(cronExpression);
            // 根据TriggerKey获取现有的CronTrigger
            CronTriggerImpl trigger = (CronTriggerImpl) scheduler.getTrigger(triggerKey);
            // 如果触发器不存在，则抛出异常
            if (trigger == null) {
                throw new SchedulingException("获取定时任务失败");
            }
            // 设置触发器的开始时间
            trigger.setStartTime(new Date());
            // 更新触发器的描述和调度信息
            CronTrigger cronTrigger = trigger;
            cronTrigger =
cronTrigger.getTriggerBuilder().withIdentity(triggerKey).withDescription(description)
                    .withSchedule(scheduleBuilder).build();
            // 重新调度任务
```

```
            scheduler.rescheduleJob(triggerKey, cronTrigger);
            // 记录更新成功的日志
            logger.info("更新任务成功");
        } catch (Exception e) {
            // 打印异常堆栈跟踪
            e.printStackTrace();
        }
        // 返回操作成功的响应
        return R.ok();
    }

    /**
     * 删除任务
     */
    @RequestMapping("/delete")
    public R delete(@RequestBody CronJobReq cronJobReq) throws SchedulerException {
        // 从请求对象中获取任务名称和组别
        String jobReqName = cronJobReq.getName();
        String jobReqGroup = cronJobReq.getGroup();
        logger.info("删除任务:{},{}", jobReqName, jobReqGroup);
        // 删除指定名称和组别的任务
        schedulerFactoryBean.getScheduler().deleteJob(JobKey.jobKey(jobReqName, jobReqGroup));
        logger.info("删除任务成功");
        return R.ok();
    }

    /**
     * 获取任务列表
     */
    @RequestMapping("/query")
    public List<CronJobResp> all() throws SchedulerException {
        logger.info("获取任务列表");
        //获取任务列表
        List<CronJobResp> cronJobRespList = new ArrayList<>();
        //获取任务调度对象
        Scheduler scheduler = schedulerFactoryBean.getScheduler();
        //遍历任务组
        for (String groupName : scheduler.getJobGroupNames()) {
        //遍历任务
            for (JobKey jobKey : scheduler.getJobKeys(GroupMatcher.jobGroupEquals(groupName))) {
                //封装任务信息
                CronJobResp cronJobResp = new CronJobResp();
                //任务组
                cronJobResp.setGroup(groupName);
                //任务名称
                cronJobResp.setName(jobKey.getName());
                //获取触发器集合
                List<? extends Trigger> triggers = scheduler.getTriggersOfJob(jobKey);
                //获取触发器
                CronTrigger trigger = (CronTrigger)triggers.get(0);
                //下次执行时间
                cronJobResp.setNextFireTime(trigger.getNextFireTime());
                //上次执行时间
                cronJobResp.setPrevFireTime(trigger.getPreviousFireTime());
                //cron表达式
                cronJobResp.setCronExpression(trigger.getCronExpression());
                //描述
                cronJobResp.setDescription(trigger.getDescription());
                //状态名称
                Trigger.TriggerState triggerState = scheduler.getTriggerState(trigger.getKey());
                cronJobResp.setState(triggerState.name());
                //添加到集合中
                cronJobRespList.add(cronJobResp);
            }
        }
        return cronJobRespList;
    }
}
```

10.3 feign远程调用生成数据

10.3.1 pom依赖

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-loadbalancer</artifactId>
</dependency>
```

- 主配置类

```
package com.hxx;

import org.mybatis.spring.annotation.MapperScan;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
import org.springframework.cloud.openfeign.EnableFeignClients;
import org.springframework.core.env.ConfigurableEnvironment;

@SpringBootApplication
@EnableDiscoveryClient
@EnableFeignClients
public class TrainBatchApplication {

    public static void main(String[] args) {
        SpringApplication application = new SpringApplication(TrainBatchApplication.class);
        ConfigurableEnvironment environment = application.run(args).getEnvironment();
        System.out.println("启动成功：" + environment.getProperty("server.port"));
    }
}
```

10.3.2 在train-daily模块中添加远程调用train模块中的所有车次信息

- 获取所有的车次数据生成对应的每日数据

```
@FeignClient("trainBase")
public interface TrainFeign {
    @GetMapping("/train/all")
    List<TrainEntity> trainAll();
}
```

- service业务编写

```
public interface DailyTrainService extends IService<DailyTrainEntity> {

    Page<DailyTrainEntity> getDailyTrainList(Page page, DailyTrainEntity dailyTrainEntity);

    ///生成指定日期所有车次的数据
    void genDaily(Date date);

    //生成指定日期指定车次的数据
    void genDailyTrain(Date date, TrainEntity trainEntity);
}
```

- service业务实现

```
@Service
public class DailyTrainServiceImpl extends ServiceImpl<DailyTrainMapper, DailyTrainEntity>
implements com.hxx.service.DailyTrainService{

    @Autowired
    private DailyTrainMapper dailyTrainMapper;
```

```
    @Autowired
    private TrainFeign trainFeign;


    /**
     * 生成每日训练报告
     * 本方法旨在根据指定的日期生成每日的训练报告。它首先从外部服务获取所有的训练实体列表，
     * 然后针对每个训练实体生成相应的每日训练报告。
     *
     * @param date 生成报告的日期
     * @throws RuntimeException 如果没有可用的训练数据，则抛出运行时异常
     */
    @Override
    public void genDaily(Date date) {
        // 从外部服务获取所有训练实体
        List<TrainEntity> trainList = trainFeign.trainAll();

        // 检查是否存在训练实体，如果不存在则抛出异常
        if (CollUtil.isEmpty(trainList)) {
            throw new RuntimeException("没有基础数据，查询结束");
        }

        // 遍历每个训练实体，生成每日训练报告
        for (TrainEntity train : trainList) {
            genDailyTrain(date,train);
        }
    }


    @Override
    public void genDailyTrain(Date date, TrainEntity trainEntity) {
        //删除该车次已有的数据(按照车次和日期)
        dailyTrainMapper.delete(new LambdaQueryWrapper<DailyTrainEntity>()
                .eq(DailyTrainEntity::getDate,date)
                .eq(DailyTrainEntity::getCode,trainEntity.getCode())
        );
        //生成该车次的数据
        DailyTrainEntity dailyTrainEntity = BeanUtil.copyProperties(trainEntity,
DailyTrainEntity.class);
        //重新设置id，因为每日都生成数据，所以不能使用train自带的id，会重复
        dailyTrainEntity.setId(SnowUtil.getSnowflakeId());
        dailyTrainEntity.setDate(date);
        dailyTrainMapper.insert(dailyTrainEntity);
    }
}
```

- controller控制层

```
@RestController
@RequestMapping("/dailyTrain")
public class DailyTrainController {
    @Autowired
    private DailyTrainService dailyTrainService;

    @GetMapping("gen_daily/{date}")
    public R genDaily(@PathVariable @DateTimeFormat(pattern = "yyyy-MM-dd")Date date) {
        dailyTrainService.genDaily(date);
        return R.ok();
    }
}
```

10.3.3 在train-batch模块

- 编写远程feign接口

```java
@FeignClient("trainDaily")
public interface TrainDailyFeign {

    @GetMapping("/dailyTrain/gen_daily/{date}")
    public R genDaily(@PathVariable @DateTimeFormat(pattern = "yyyy-MM-dd") Date date);
}
```

- 编写任务类

```java
@DisallowConcurrentExecution
public class DailyTrainJob implements Job {

    @Resource
    private TrainDailyFeign trainDailyFeign;

    private static final Logger log = org.slf4j.LoggerFactory.getLogger(DailyTrainJob.class);
    @Override
    public void execute(JobExecutionContext jobExecutionContext) throws JobExecutionException {
        log.info("生成15天后车次数据开始");
        Date date = new Date();
        //15天偏移量
        DateTime dateTime = DateUtil.offsetDay(date, 15);
        trainDailyFeign.genDaily(dateTime.toJdkDate());
        log.info("生成15天后车次数据结束");
    }
}
```

10.4 编写前端BatchJob.vue

- 任务管理



- 添加



10.4 生成每日车站

10.4.1 在train-station模块

- 添加一个按照车次获取车站信息的方法

```
@RestController
@RequestMapping("/trainStation")
public class TrainStationController {
    @Autowired
    private TrainStationService trainStationService;

    @GetMapping("/trainCode")
    public List<TrainStationEntity> trainStation(@RequestParam("trainCode") String trainCode){
        QueryWrapper<TrainStationEntity> wrapper = new QueryWrapper<>();
        wrapper.eq("train_code",trainCode);
        return trainStationService.list(wrapper);
    }
}
```

10.4.2 train-daily模块feign调用

```
@FeignClient("trainBase")
public interface TrainFeign {
    @GetMapping("/train/all")
    List<TrainEntity> trainAll();

    @GetMapping("/trainStation/trainCode")
    public List<TrainStationEntity> trainCodeStation(@RequestParam("trainCode") String trainCode);
}
```

- service

```
@Service
public class DailyTrainStationServiceImpl extends ServiceImpl<DailyTrainStationMapper,
DailyTrainStationEntity>
implements com.hxx.service.DailyTrainStationService{
    @Autowired
    private DailyTrainStationMapper dailyTrainStationMapper;

    @Autowired
    private TrainFeign trainFeign;

    @Override
    public void genDaily(Date date, String trainCode) {
        // 删除某日某车次的车站数据
        dailyTrainStationMapper.delete(new LambdaQueryWrapper<DailyTrainStationEntity>()
                .eq(DailyTrainStationEntity::getDate,date)
                .eq(DailyTrainStationEntity::getTrainCode,trainCode)
        );
        // 查询所有的车站数据
        List<TrainStationEntity> stationList = trainFeign.trainCodeStation(trainCode);
        if (CollUtil.isEmpty(stationList)) {
            throw new RuntimeException("未查询到车站数据");
        }
        stationList.forEach(trainStationEntity -> {
            DailyTrainStationEntity dailyTrainStationEntity =
BeanUtil.copyProperties(trainStationEntity, DailyTrainStationEntity.class);
            dailyTrainStationEntity.setDate(date);
            dailyTrainStationEntity.setId(SnowUtil.getSnowflakeId());
            dailyTrainStationMapper.insert(dailyTrainStationEntity);
        });
    }
}
```

- controller

```java
@RestController
@RequestMapping("/dailyTrainStation")
public class DailyTrainStationController {

    @Autowired
    private DailyTrainStationService dailyTrainStationService;

    @GetMapping("genTrainStation")
    public void genTrainStation(@RequestParam @DateTimeFormat(pattern = "yyyy-MM-dd") Date date,
                                @RequestParam("trainCode") String trainCode) {
        dailyTrainStationService.genDaily(date, trainCode);
    }
}
```

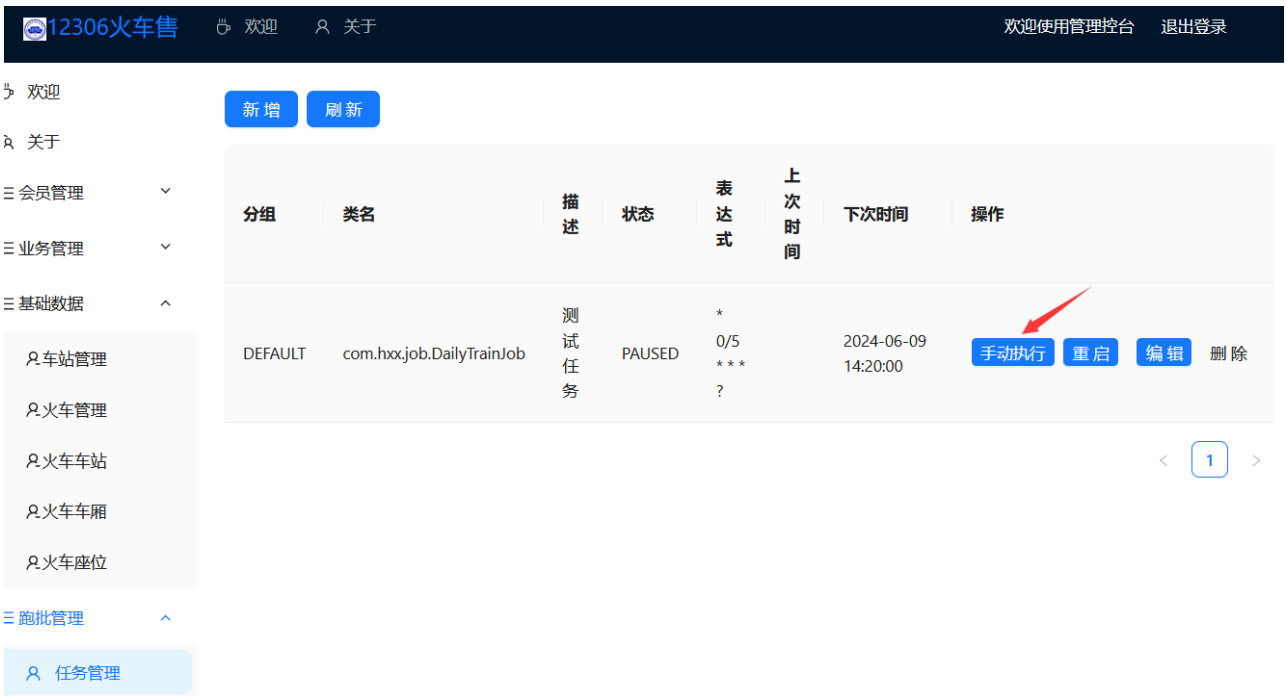
10.4.3 在DailyTrain调用

- 在生成每日车次方法生成每日车站数据

```java
public class DailyTrainServiceImpl extends ServiceImpl<DailyTrainMapper, DailyTrainEntity>

    @Override  1 usage
    public void genDailyTrain(Date date, TrainEntity trainEntity) {
        //删除该车次已有的数据(按照车次和日期)
        dailyTrainMapper.delete(new LambdaQueryWrapper<DailyTrainEntity>()
                .eq(DailyTrainEntity::getDate,date)
                .eq(DailyTrainEntity::getCode,trainEntity.getCode())
        );
        //生成该车次的数据
        DailyTrainEntity dailyTrainEntity = BeanUtil.copyProperties(trainEntity, DailyTrainEntity.class);
        //重新设置id，因为每日都生成数据，所以不能使用train自带的id，会重复
        dailyTrainEntity.setId(SnowUtil.getSnowflakeId());
        dailyTrainEntity.setDate(date);
        dailyTrainMapper.insert(dailyTrainEntity);
        //调用对应的车次生成车站数据
        dailyTrainStationService.genDaily(date,trainEntity.getCode());
    }
```

- 可以在**任务管理**测试



10.5 生成每日车厢

10.5.1 在train-carriage中添加方法

- 根据车次获取所有车厢信息

```
@RestController
@RequestMapping("/trainCarriage")
public class TrainCarriageController {
    @Autowired
    private TrainCarriageSerivce trainCarriageService;

    @GetMapping("findAll")
    public List<TrainCarriageEntity> findAll(@RequestParam String trainCode) {
        return trainCarriageService
                .list(new LambdaQueryWrapper<TrainCarriageEntity>()
                        .eq(TrainCarriageEntity::getTrainCode, trainCode));
    }
}
```

10.5.2 train-daily模块

- 添加远程feign接口

```
@FeignClient("trainBase")
public interface TrainFeign {
    @GetMapping("/train/all")
    List<TrainEntity> trainAll();

    @GetMapping("/trainStation/trainCode")
    public List<TrainStationEntity> trainCodeStation(@RequestParam("trainCode") String trainCode);

    @GetMapping("/trainCarriage/findAll")
    List<TrainCarriageEntity> findAll(@RequestParam String trainCode);
}
```

- DailyTrainCarriageService添加方法

```
@Service
public class DailyTrainCarriageServiceImpl extends ServiceImpl<DailyTrainCarriageMapper,
DailyTrainCarriageEntity>
implements com.hxx.service.DailyTrainCarriageService{

    @Autowired
    private DailyTrainCarriageMapper dailyTrainCarriageMapper;

    @Autowired
    private TrainFeign trainFeign;

    @Override
    public void genDaily(Date date, String trainCode) {
        //删除
        dailyTrainCarriageMapper.delete(new LambdaQueryWrapper<DailyTrainCarriageEntity>()
                .eq(DailyTrainCarriageEntity::getTrainCode, trainCode)
                .eq(DailyTrainCarriageEntity::getDate, date));
        //获取车厢
        List<TrainCarriageEntity> trainCarriageEntities = trainFeign.findAll(trainCode);
        //判断
        if (ObjUtil.isNull(trainCarriageEntities)) {
            throw new RuntimeException("该车次不存在车厢");
        }
        for (TrainCarriageEntity trainCarriageEntity : trainCarriageEntities) {
            //对拷
            DailyTrainCarriageEntity dailyTrainCarriageEntity =
BeanUtil.copyProperties(trainCarriageEntity, DailyTrainCarriageEntity.class);
            //设置日期
            dailyTrainCarriageEntity.setDate(date);
            dailyTrainCarriageEntity.setId(SnowUtil.getSnowflakeId());
            dailyTrainCarriageEntity.setTrainCode(trainCode);
            dailyTrainCarriageMapper.insert(dailyTrainCarriageEntity);
```

```
        }
    }
}
```

- controller

```
@RestController
@RequestMapping("/dailyTrainCarriage")
public class DailyTrainCarriageController {
    @Autowired
    private DailyTrainCarriageService dailyTrainCarriageService;

    @GetMapping("/genDailyCarriage")
    public void genDailyCarriage(@RequestParam @DateTimeFormat(pattern = "yyyy-MM-dd") Date date,
                                 @RequestParam("trainCode") String trainCode) {
        dailyTrainCarriageService.genDaily(date, trainCode);
    }
}
```

- service：同时生成车次、车站、车厢数据
  - 在DailyTrainService业务编写



- 测试跑批

10.6 生成每日座位

10.6.1 模块train-base

- 根据车次获取所有的座位信息

```
@RestController
@RequestMapping("/seat")
public class SeatController {
    @Autowired
    private SeatService seatService;

    @GetMapping("/load/{trainCode}")
    public List<TrainSeatEntity> load(@PathVariable String trainCode) {
        return seatService.list(new QueryWrapper<TrainSeatEntity>()
                .eq("train_code", trainCode));
    }
}
```

10.6.2 daily-train-seat远程调用feigin

```
@FeignClient("trainBase")
public interface TrainFeign {
    @GetMapping("/train/all")
    List<TrainEntity> trainAll();

    @GetMapping("/trainStation/trainCode")
    public List<TrainStationEntity> trainCodeStation(@RequestParam("trainCode") String trainCode);

    @GetMapping("/trainCarriage/findAll")
    List<TrainCarriageEntity> findAll(@RequestParam String trainCode);

    @GetMapping("/seat/load/{trainCode}")
    public List<TrainSeatEntity> load(@PathVariable String trainCode);
}
```

- service

```
public interface DailyTrainSeatService extends IService<DailyTrainSeatEntity> {

    Page<DailyTrainSeatEntity> getDailyTrainSeatList(Page page, DailyTrainSeatEntity
dailyTrainSeatEntity);

    /**
     *
     * @param date  日期
     * @param trainCode 车次
     * @param sell 售卖情况，默认=0
     */
    void genDaily(Date date, String trainCode,String sell);
}
```

- serviceImpl

```
@Service
public class DailyTrainSeatServiceImpl extends ServiceImpl<DailyTrainSeatMapper, DailyTrainSeatEntity>
implements com.hxx.service.DailyTrainSeatService{
    @Autowired
    private DailyTrainSeatMapper dailyTrainSeatMapper;

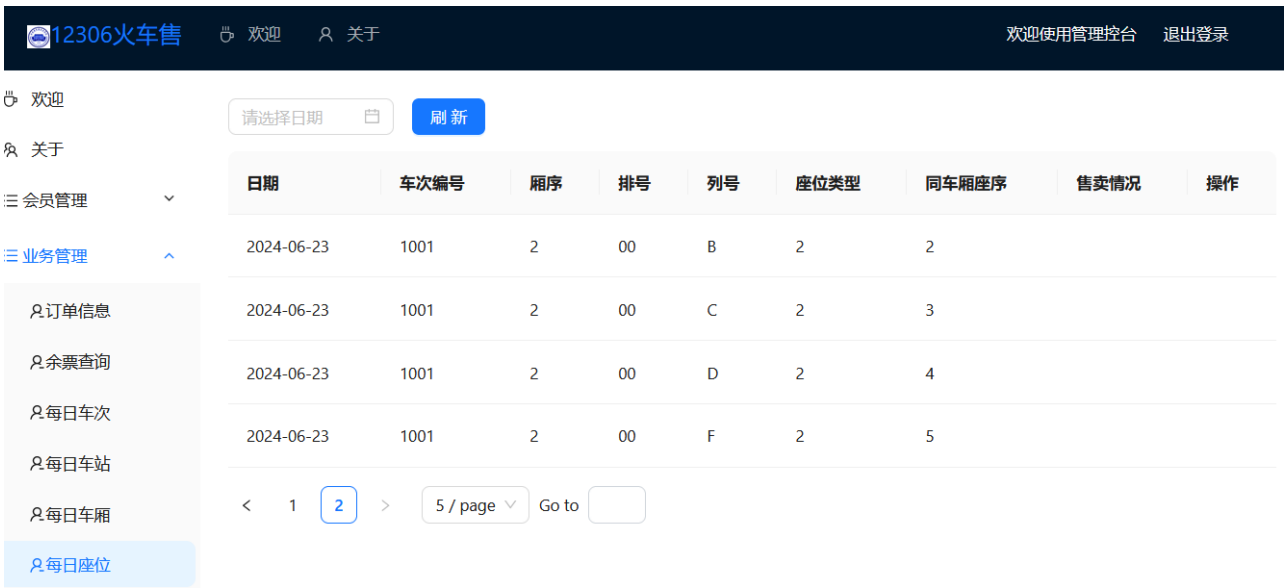
    @Autowired
    private TrainFeign trainFeign;

    @Override
    public void genDaily(Date date, String trainCode, String sell) {
        //按照车次和日期删除数据
        dailyTrainSeatMapper.delete(new LambdaQueryWrapper<DailyTrainSeatEntity>()
                .eq(DailyTrainSeatEntity::getTrainCode,trainCode)
                .eq(DailyTrainSeatEntity::getDate,date)
        );
        //获取车次的所有座位
        List<TrainSeatEntity> seatList = trainFeign.load(trainCode);
        if (ObjUtil.isNull(seatList)) {
            throw new RuntimeException("没有座位数据，任务结束");
        }
        for (TrainSeatEntity seat : seatList) {
            DailyTrainSeatEntity dailyTrainSeatEntity =
                            BeanUtil.copyProperties(seat,DailyTrainSeatEntity.class);
            dailyTrainSeatEntity.setId(SnowUtil.getSnowflakeId());
            dailyTrainSeatEntity.setDate(date);
            dailyTrainSeatEntity.setTrainCode(trainCode);
            dailyTrainSeatEntity.setSell(sell);
            dailyTrainSeatMapper.insert(dailyTrainSeatEntity);
        }
    }
}
```

- DailyTrainServiceImpl编写同时生成车次、车站、车厢、座位

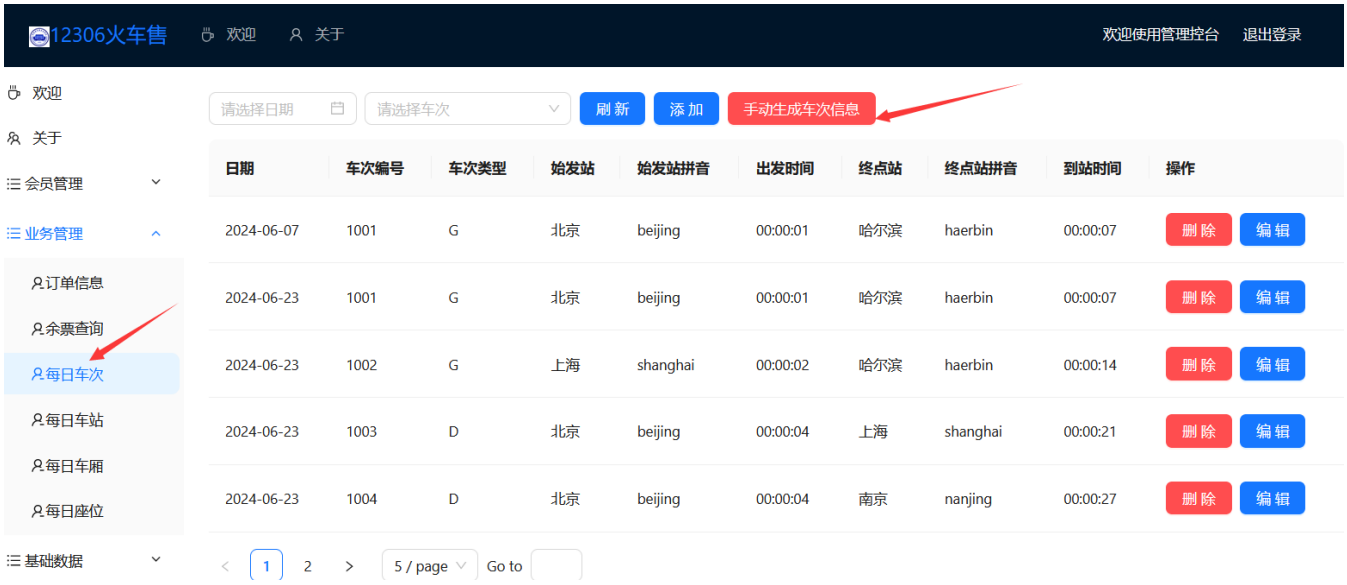
```
@Override
public void genDailyTrain(Date date, TrainEntity trainEntity) {
    //删除该车次已有的数据(按照车次和日期)
    dailyTrainMapper.delete(new LambdaQueryWrapper<DailyTrainEntity>()
            .eq(DailyTrainEntity::getDate,date)
            .eq(DailyTrainEntity::getCode,trainEntity.getCode())
    );
    //生成该车次的数据
    DailyTrainEntity dailyTrainEntity = BeanUtil.copyProperties(trainEntity, DailyTrainEntity.class);
    //重新设置id，因为每日都生成数据，所以不能使用train自带的id，会重复
    dailyTrainEntity.setId(SnowUtil.getSnowflakeId());
    dailyTrainEntity.setDate(date);
    //生成每日车次
    dailyTrainMapper.insert(dailyTrainEntity);
    //调用对应的车次生成车站数据
    dailyTrainStationService.genDaily(date,trainEntity.getCode());
    // 调用车次车厢生成车次车厢数据
    dailyTrainCarriageService.genDaily(date,trainEntity.getCode());
    //根据车次获取车站的数量
    int stationCount = trainFeign.trainCodeStation(trainEntity.getCode()).size();
    String sell = StrUtil.fillBefore("", '0', stationCount - 1);
    //生成每日座位
    dailyTrainSeatService.genDaily(date,trainEntity.getCode(),sell);
}
```

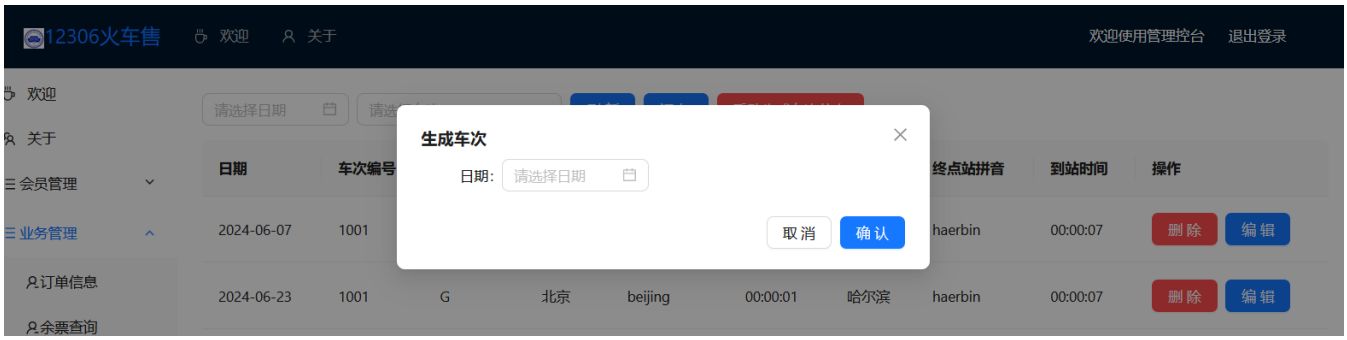
- 测试



10.7 手动生成数据

- 指定日期后生成数据





- 前端：DailyTrain.vue

```
//生成车次信息
  const handleGenDailyOk = () => {
    //日期格式
    let date = dayjs(formState.date).format("YYYY-MM-DD");
    axios.get(`/trainDaily/dailyTrain/gen_daily/${date}`).then(()=>{
      message.success('生成成功');
      genDailyVisible.value=false;
    })
  }
```

- 后端：

```
@RestController
@RequestMapping("/dailyTrain")
public class DailyTrainController {
    @Autowired
    private DailyTrainService dailyTrainService;

    @GetMapping("gen_daily/{date}")
    public R genDaily(@PathVariable @DateTimeFormat(pattern = "yyyy-MM-dd")Date date) {
        dailyTrainService.genDaily(date);
        return R.ok();
    }
}
```

十一、预售票



11.1 余票查询

11.1.1 数据表

```
CREATE TABLE `daily_train_ticket` (
  `id` bigint NOT NULL COMMENT 'id',
  `date` date NOT NULL COMMENT '日期',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `start` varchar(20) NOT NULL COMMENT '出发站',
  `start_pinyin` varchar(50) NOT NULL COMMENT '出发站拼音',
  `start_time` time NOT NULL COMMENT '出发时间',
  `start_index` int NOT NULL COMMENT '出发站序|本站是整个车次的第几站',
  `end` varchar(20) NOT NULL COMMENT '到达站',
  `end_pinyin` varchar(50) NOT NULL COMMENT '到达站拼音',
  `end_time` time NOT NULL COMMENT '到站时间',
  `end_index` int NOT NULL COMMENT '到站站序|本站是整个车次的第几站',
  `ydz` int NOT NULL COMMENT '一等座余票',
  `ydz_price` decimal(8,2) NOT NULL COMMENT '一等座票价',
  `edz` int NOT NULL COMMENT '二等座余票',
  `edz_price` decimal(8,2) NOT NULL COMMENT '二等座票价',
  `rw` int NOT NULL COMMENT '软卧余票',
  `rw_price` decimal(8,2) NOT NULL COMMENT '软卧票价',
  `yw` int NOT NULL COMMENT '硬卧余票',
  `yw_price` decimal(8,2) NOT NULL COMMENT '硬卧票价',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  UNIQUE KEY `date_train_code_start_end_unique` (`date`,`train_code`,`start`,`end`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='余票信息';
```

11.1.2 编写三层（train-daily模块）

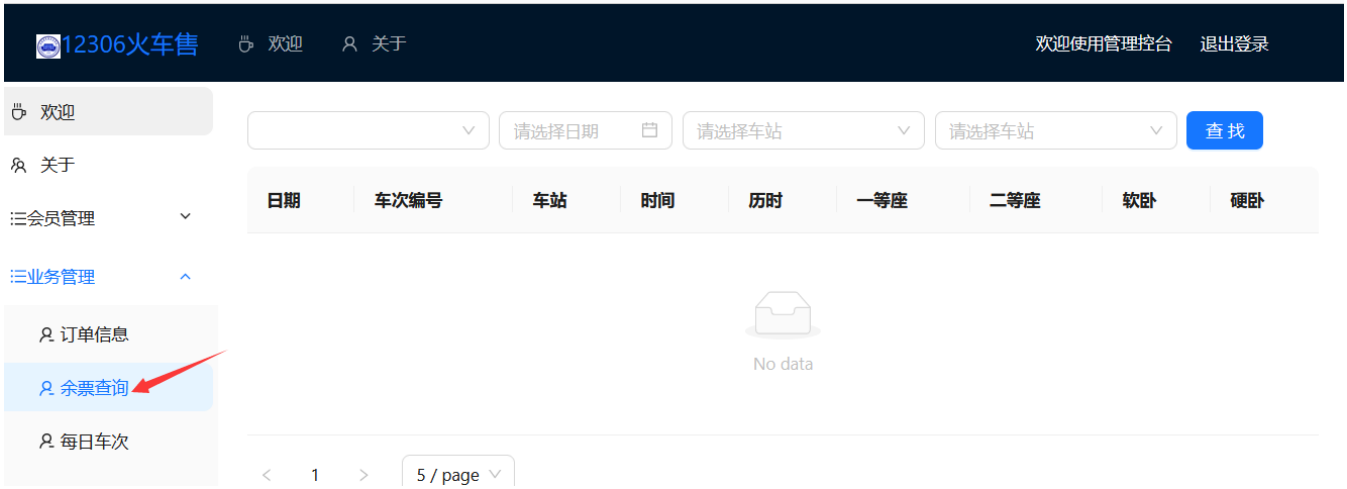
```
@Service
public class DailyTrainTicketServiceImpl extends ServiceImpl<DailyTrainTicketMapper, DailyTrainTicketEntity>
implements DailyTrainTicketService {

    @Autowired
    private DailyTrainTicketMapper dailyTrainTicketMapper;

    @Override
    public Page<DailyTrainTicketEntity> loadPage(Page page, DailyTrainTicketEntity dailyTrainTicketEntity) {
        return dailyTrainTicketMapper.selectPage(page,new LambdaQueryWrapper<DailyTrainTicketEntity>()
                .eq(dailyTrainTicketEntity.getTrainCode()!=null,DailyTrainTicketEntity::getTrainCode,
                    dailyTrainTicketEntity.getTrainCode())
                .eq(dailyTrainTicketEntity.getDate()!=null,DailyTrainTicketEntity::getDate,
```

```
                dailyTrainTicketEntity.getDate())
            .eq(dailyTrainTicketEntity.getStart()!=null,DailyTrainTicketEntity::getStart,
                dailyTrainTicketEntity.getStart())
            .eq(dailyTrainTicketEntity.getEnd()!=null,DailyTrainTicketEntity::getEnd,
                dailyTrainTicketEntity.getEnd())
        );
    }
}
```

11.1.3 前端编写




```
<template>
    <p>
      <a-space>
        <train-select-view v-model="queryParams.trainCode" width="200px"></train-select-view>
        <a-date-picker v-model:value="queryParams.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期"></a-
date-picker>
        <station-select-view v-model="queryParams.start" width="200px"></station-select-view>
        <station-select-view v-model="queryParams.end" width="200px"></station-select-view>
        <a-button type="primary" @click="refreshData()">查找</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{ column, record }">
        <template v-if="column.dataIndex === 'operation'">
        </template>
        <template v-else-if="column.dataIndex === 'station'">
          {{record.start}}<br/>
          {{record.end}}
        </template>
        <template v-else-if="column.dataIndex === 'time'">
          {{record.startTime}}<br/>
          {{record.endTime}}
        </template>
        <template v-else-if="column.dataIndex === 'duration'">
          {{calDuration(record.startTime, record.endTime)}}<br/>
          <div v-if="record.startTime.replaceAll(':', '') >= record.endTime.replaceAll(':', '')">
            次日到达
          </div>
          <div v-else>
            当日到达
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'ydz'">
          <div v-if="record.ydz >= 0">
            {{record.ydz}}<br/>
            {{record.ydzPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'edz'">
```

```
          <div v-if="record.edz >= 0">
            {{record.edz}}<br/>
            {{record.edzPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'rw'">
          <div v-if="record.rw >= 0">
            {{record.rw}}<br/>
            {{record.rwPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'yw'">
          <div v-if="record.yw >= 0">
            {{record.yw}}<br/>
            {{record.ywPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
      </template>
    </a-table>
    <br><TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
  </template>

  <script setup lang="ts">
  import { ref, onMounted } from 'vue';
  import axios from "axios";
  import TrainSelectView from "@/components/TrainSelect.vue";
  import StationSelectView from "@/components/StationSelect.vue";
  import dayjs from "dayjs";

    const columns = [
    {
    title: '日期',
    dataIndex: 'date',
    key: 'date',
    },
    {
    title: '车次编号',
    dataIndex: 'trainCode',
    key: 'trainCode',
    },
    {
    title: '车站',
    dataIndex: 'station',
    },
    {
    title: '时间',
    dataIndex: 'time',
    },
    {
    title: '历时',
    dataIndex: 'duration',
    },
    {
    title: '一等座',
    dataIndex: 'ydz',
    key: 'ydz',
    },
    {
    title: '二等座',
    dataIndex: 'edz',
    key: 'edz',
    },
    {
```

```
    title: '软卧',
    dataIndex: 'rw',
    key: 'rw',
    },
    {
    title: '硬卧',
    dataIndex: 'yw',
    key: 'yw',
    },
    ];
   //接口
   interface DailyTrainDicket {
    id: number,
    date: string,
    trainCode: string,
    start: string,
    startPinyin: string,
    startTime: string,
    startIndex: string,
    end: string,
    endPinyin: string,
    endTime: string,
    endIndex: string,
    ydz: string,
    ydzPrice: number,
    edz: string,
    edzPrice: number,
    rw: string,
    rwPrice: number,
    yw: string,
    ywPrice: number,
  }
  //数据源
  const dataSource = ref<DailyTrainDicket[]>([]);

  //刷新
  const refreshData = () => {
      showData();
  }

  const calDuration = (startTime:string, endTime:string) => {
      let diff = dayjs(endTime, 'HH:mm:ss').diff(dayjs(startTime, 'HH:mm:ss'), 'seconds');
      return dayjs('00:00:00', 'HH:mm:ss').second(diff).format('HH:mm:ss');
  };

 //--------------分页----------------
  const total = ref(0);
  //分页参数
  const queryParams=ref({
    current:1,
    size:5,
    trainCode:null,
    date:null,
    start:null,
    end:null,
  })

  //当前页码变化
  const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
  //加载数据
  const showData = () => {
    axios({
      method: 'get',
      url: '/trainDaily/dailyTrainTicket/loadPage',
      params: queryParams.value
    }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
```

```
  })
}

//页面加载数据
onMounted(()=>{
  showData();
})
</script>
```

11.1.4 生成余票数据

- DailyTrainSeatServiceImpl添加获取座位数量的方法

```
@Service
public class DailyTrainSeatServiceImpl extends ServiceImpl<DailyTrainSeatMapper, DailyTrainSeatEntity>
implements com.hxx.service.DailyTrainSeatService{
    @Autowired
    private DailyTrainSeatMapper dailyTrainSeatMapper;

    @Autowired
    private TrainFeign trainFeign;

    /**
     * 根据日期、车次代码和座位类型获取指定列车座位数量。
     *
     * @param date 乘车日期，用于查询特定日期的列车座位信息。
     * @param trainCode 车次代码，用于查询特定车次的列车座位信息。
     * @param seatType 座位类型，可选参数，用于查询特定类型的座位信息。如果为null，则不筛选座位类型。
     * @return 返回指定日期、车次和座位类型下的座位数量。如果不存在匹配的座位信息，则返回0。
     */
    @Override
    public Integer getSeatCount(Date date, String trainCode, String seatType) {
        // 使用LambdaQueryWrapper构建查询条件，查询指定日期、车次和（可选的）座位类型的座位数量
        return dailyTrainSeatMapper.selectList(new LambdaQueryWrapper<DailyTrainSeatEntity>()
                .eq(DailyTrainSeatEntity::getTrainCode, trainCode) // 筛选车次代码匹配的座位信息
                .eq(DailyTrainSeatEntity::getDate, date) // 筛选日期匹配的座位信息
                .eq(seatType != null, DailyTrainSeatEntity::getSeatType, seatType)
        // 如果seatType不为null，则筛选座位类型匹配的座位信息
        ).size();
    }
}
```

- DailyTrainTicketServiceImpl生成余票数据

```
@Service
public class DailyTrainTicketServiceImpl extends ServiceImpl<DailyTrainTicketMapper,
DailyTrainTicketEntity>
implements DailyTrainTicketService {

    @Autowired
    private DailyTrainTicketMapper dailyTrainTicketMapper;

    @Autowired
    private DailyTrainSeatService dailyTrainSeatService;

    @Autowired
    private TrainFeign trainFeign;

    @Override
    @Transactional
    public void genDaily(DailyTrainEntity dailyTrainEntity, Date date, String trainCode) {
        // 删除某日某车次的车票数据
        dailyTrainTicketMapper.delete(new LambdaQueryWrapper<DailyTrainTicketEntity>()
                .eq(DailyTrainTicketEntity::getDate,date)
                .eq(DailyTrainTicketEntity::getTrainCode,trainCode)
        );
        //查询某车次的所有车站数据
        List<TrainStationEntity> stationList = trainFeign.trainCodeStation(trainCode);
        if (CollUtil.isEmpty(stationList)) {
```

```
            throw new RuntimeException("该车次没有车站数据");
        }
        //循环生成该车次的所有车票数据:这是一个嵌套循环
        for (int i = 0; i < stationList.size(); i++) {
            // 开始站
            TrainStationEntity startStation = stationList.get(i);
            BigDecimal sunKM = BigDecimal.ZERO;
            for (int j = i + 1; j < stationList.size(); j++) {
                // 终点站
                TrainStationEntity endStation = stationList.get(j);
                sunKM = sunKM.add(endStation.getKm());
                // 生成车票数据
                DailyTrainTicketEntity dailyTrainTicketEntity = new DailyTrainTicketEntity();
                dailyTrainTicketEntity.setDate(date);
                dailyTrainTicketEntity.setTrainCode(trainCode);
                dailyTrainTicketEntity.setStart(startStation.getName());
                dailyTrainTicketEntity.setStartPinyin(startStation.getNamePinyin());
                dailyTrainTicketEntity.setStartIndex(startStation.getIndex());
                dailyTrainTicketEntity.setStartTime(startStation.getOutTime());
                dailyTrainTicketEntity.setEnd(endStation.getName());
                dailyTrainTicketEntity.setEndPinyin(endStation.getNamePinyin());
                dailyTrainTicketEntity.setEndIndex(endStation.getIndex());
                dailyTrainTicketEntity.setEndTime(endStation.getInTime());
                //获取座位数量
                //一等座
                Integer ydz = dailyTrainSeatService.getSeatCount(date,trainCode,
SeatTypeEnum.YDZ.getCode());
                //二等座
                Integer edz = dailyTrainSeatService.getSeatCount(date,trainCode,
SeatTypeEnum.EDZ.getCode());
                //硬卧
                Integer yw = dailyTrainSeatService.getSeatCount(date,trainCode,
SeatTypeEnum.YW.getCode());
                //软卧
                Integer rw = dailyTrainSeatService.getSeatCount(date,trainCode,
SeatTypeEnum.RW.getCode());

                //添加信息数据
                dailyTrainTicketEntity.setYdz(ydz);
                dailyTrainTicketEntity.setEdz(edz);
                dailyTrainTicketEntity.setYw(yw);
                dailyTrainTicketEntity.setRw(rw);

                //获取车票价格:票价=里程和*座位票价格*车次类型系数
                String trainType = dailyTrainEntity.getType();
                //获取车票系数
                BigDecimal priceRate = EnumUtil.getFieldBy(TrainTypeEnum::getPriceRate,
TrainTypeEnum::getCode, trainType);
                //一等座
                BigDecimal ydzPrice =
sunKM.multiply(SeatTypeEnum.YDZ.getPrice()).multiply(priceRate).setScale(2, RoundingMode.HALF_UP);
                //二等座
                BigDecimal edzPrice =
sunKM.multiply(SeatTypeEnum.EDZ.getPrice()).multiply(priceRate).setScale(2, RoundingMode.HALF_UP);
                //硬卧
                BigDecimal ywPrice =
sunKM.multiply(SeatTypeEnum.YW.getPrice()).multiply(priceRate).setScale(2, RoundingMode.HALF_UP);
                //软卧
                BigDecimal rwPrice =
sunKM.multiply(SeatTypeEnum.RW.getPrice()).multiply(priceRate).setScale(2, RoundingMode.HALF_UP);

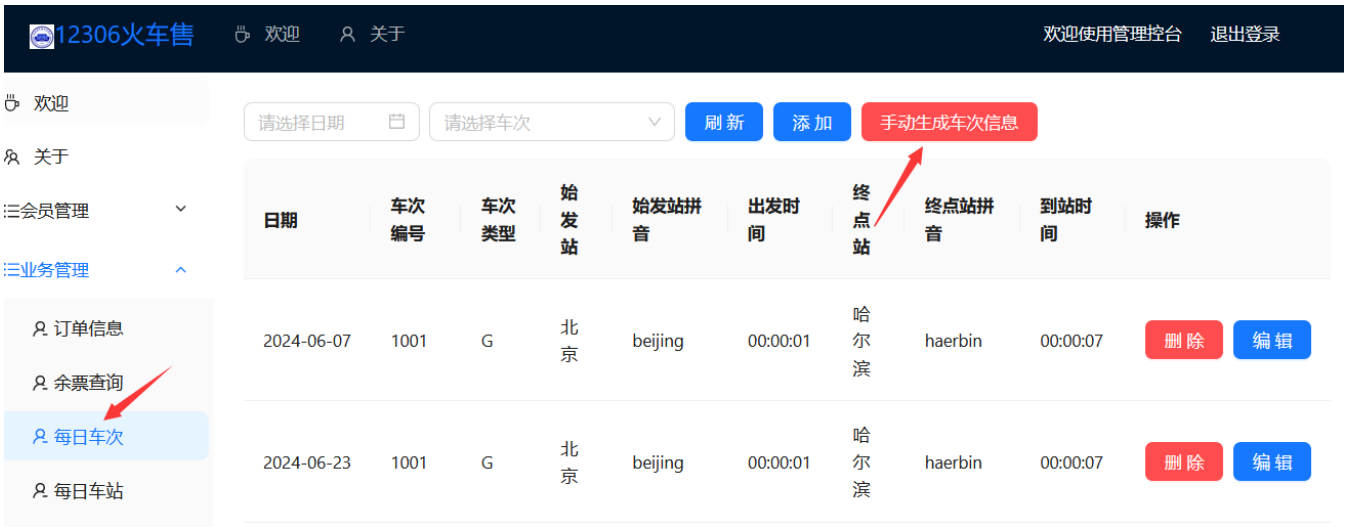
                //添加信息数据
                dailyTrainTicketEntity.setYdzPrice(ydzPrice);
                dailyTrainTicketEntity.setEdzPrice(edzPrice);
                dailyTrainTicketEntity.setYwPrice(ywPrice);
                dailyTrainTicketEntity.setRwPrice(rwPrice);
                dailyTrainTicketMapper.insert(dailyTrainTicketEntity);
            }
        }


    }
```

```
}
```

11.1.5 DailyTrainServiceImpl中添加生成余票信息

```
DailyTrainTicketService.java | DailyTrainTicketServiceImpl.java | DailyTrainServiceImpl.java | DailyTrainSeatService.java | DailyTrainSeatServiceImpl.java | TrainSeatEntity.java | TrainFeign.java
21  public class DailyTrainServiceImpl extends ServiceImpl<DailyTrainMapper, DailyTrainEntity>
76      public void genDailyTrain(Date date, TrainEntity trainEntity) {
89          //调用对应的车次生成车站数据
90          dailyTrainStationService.genDaily(date,trainEntity.getCode());
91          // 调用车次车厢生成车次车厢数据
92          dailyTrainCarriageService.genDaily(date,trainEntity.getCode());
93          //根据车次获取车站的数量
94          int stationCount = trainFeign.trainCodeStation(trainEntity.getCode()).size();
95          String sell = StrUtil.fillBefore( str: "", filledChar: '0', len: stationCount - 1);
96          //生成每日座位
97          dailyTrainSeatService.genDaily(date,trainEntity.getCode(),sell);
98          //生成每日车票
99          dailyTrainTicketService.genDaily(dailyTrainEntity,date,trainEntity.getCode());
100     }
101 }
```



- 测试



11.2 会员管理

- 在会员管理中添加余票查询
  - 复制DailyTrainDicket.vue到会员管理模块即可
  - 查询条件只保留：日期、始发站、终点站
  - 表格中去掉日期列




```
<template>
    <p>
      <a-space>
        <!-- <train-select-view v-model="queryParams.trainCode" width="200px"></train-select-view> -->
        <a-date-picker v-model:value="queryParams.date" valueFormat="YYYY-MM-DD" placeholder="请选择日期"></a-
date-picker>
        <station-select-view v-model="queryParams.start" width="200px"></station-select-view>
        <station-select-view v-model="queryParams.end" width="200px"></station-select-view>
        <a-button type="primary" @click="refreshData()">查找</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{ column, record }">
        <template v-if="column.dataIndex === 'operation'">
          <a-button type="primary" @click="order(record)">预定</a-button>
        </template>
        <template v-else-if="column.dataIndex === 'station'">
          {{record.start}}<br/>
          {{record.end}}
        </template>
        <template v-else-if="column.dataIndex === 'time'">
          {{record.startTime}}<br/>
          {{record.endTime}}
        </template>
        <template v-else-if="column.dataIndex === 'duration'">
          {{calDuration(record.startTime, record.endTime)}}<br/>
          <div v-if="record.startTime.replaceAll(':', '') >= record.endTime.replaceAll(':', '')">
            次日到达
          </div>
          <div v-else>
            当日到达
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'ydz'">
          <div v-if="record.ydz >= 0">
            {{record.ydz}}<br/>
            {{record.ydzPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'edz'">
          <div v-if="record.edz >= 0">
            {{record.edz}}<br/>
            {{record.edzPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'rw'">
          <div v-if="record.rw >= 0">
            {{record.rw}}<br/>
```

```
            {{record.rwPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
        <template v-else-if="column.dataIndex === 'yw'">
          <div v-if="record.yw >= 0">
            {{record.yw}}<br/>
            {{record.ywPrice}}￥
          </div>
          <div v-else>
            --
          </div>
        </template>
      </template>
    </a-table>
    <br><TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import StationSelectView from "@/components/StationSelect.vue";
import dayjs from "dayjs";
import { useRouter } from 'vue-router';


    const router = useRouter();
    const columns = [
    {
    title: '车次编号',
    dataIndex: 'trainCode',
    key: 'trainCode',
    },
    {
    title: '车站',
    dataIndex: 'station',
    },
    {
    title: '时间',
    dataIndex: 'time',
    },
    {
    title: '历时',
    dataIndex: 'duration',
    },
    {
    title: '一等座',
    dataIndex: 'ydz',
    key: 'ydz',
    },
    {
    title: '二等座',
    dataIndex: 'edz',
    key: 'edz',
    },
    {
    title: '软卧',
    dataIndex: 'rw',
    key: 'rw',
    },
    {
    title: '硬卧',
    dataIndex: 'yw',
    key: 'yw',
    },
    {
      title:'操作',
      dataIndex:'operation'
    }
    ];
```

```
 //接口
 interface DailyTrainDicket {
  id: number,
  date: string,
  trainCode: string,
  start: string,
  startPinyin: string,
  startTime: string,
  startIndex: string,
  end: string,
  endPinyin: string,
  endTime: string,
  endIndex: string,
  ydz: string,
  ydzPrice: number,
  edz: string,
  edzPrice: number,
  rw: string,
  rwPrice: number,
  yw: string,
  ywPrice: number,
}
//数据源
const dataSource = ref<DailyTrainDicket[]>([]);

    //刷新
  const refreshData = () => {
      showData();
  }

  //预定
  const order=(record:DailyTrainDicket)=>{
    //把数据存储在sessionStorage中
    sessionStorage.setItem('order',JSON.stringify(record))
    router.push({path:'/main/trainOrder'})
  }

  //时间差
  const calDuration = (startTime:string, endTime:string) => {
      let diff = dayjs(endTime, 'HH:mm:ss').diff(dayjs(startTime, 'HH:mm:ss'), 'seconds');
      return dayjs('00:00:00', 'HH:mm:ss').second(diff).format('HH:mm:ss');
  };

  //--------------分页----------------
const total = ref(0);
//分页参数
const queryParams=ref({
  current:1,
  size:5,
  trainCode:null,
  date:null,
  start:null,
  end:null,
})


//当前页码变化
const onChange = (page:number,pageSize:number) => {
  queryParams.value.current = page
  queryParams.value.size = pageSize
  showData()
};
//加载数据
const showData = () => {
  axios({
    method: 'get',
    url: '/trainDaily/dailyTrainTicket/loadPage',
    params: queryParams.value
  }).then(({data})=>{
      total.value = data.data.total;
      dataSource.value = data.data.list;
  })
```

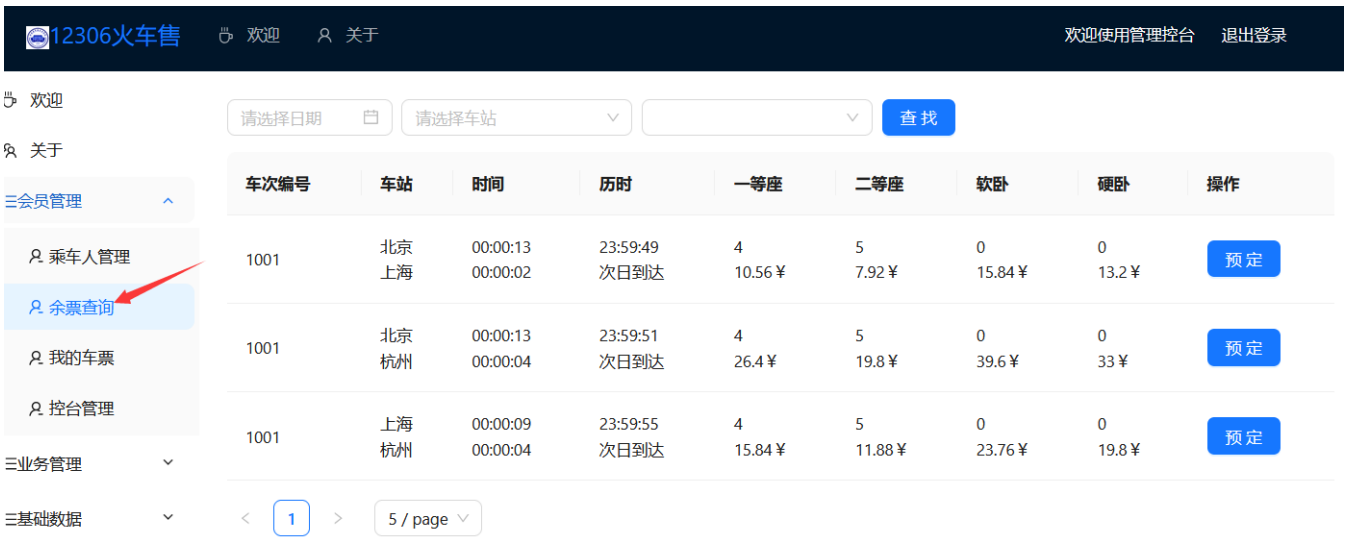
```
  }

  //页面加载数据
  onMounted(()=>{
    showData();
  })
  </script>
```

11.3 预定车票

11.3.1 预定

```
<template v-if="column.dataIndex === 'operation'">
    <a-button type="primary" @click="order(record)">预定</a-button>
</template>
----------------------------------------
{
    title:'操作',
    dataIndex:'operation'
}
----------------------------------------
//预定:跳转到order页面
const order=(record:DailyTrainDicket)=>{
    //把数据存储在sessionStorage中
    sessionStorage.setItem('order',JSON.stringify(record))
    router.push({path:'/main/trainOrder'})
}
```



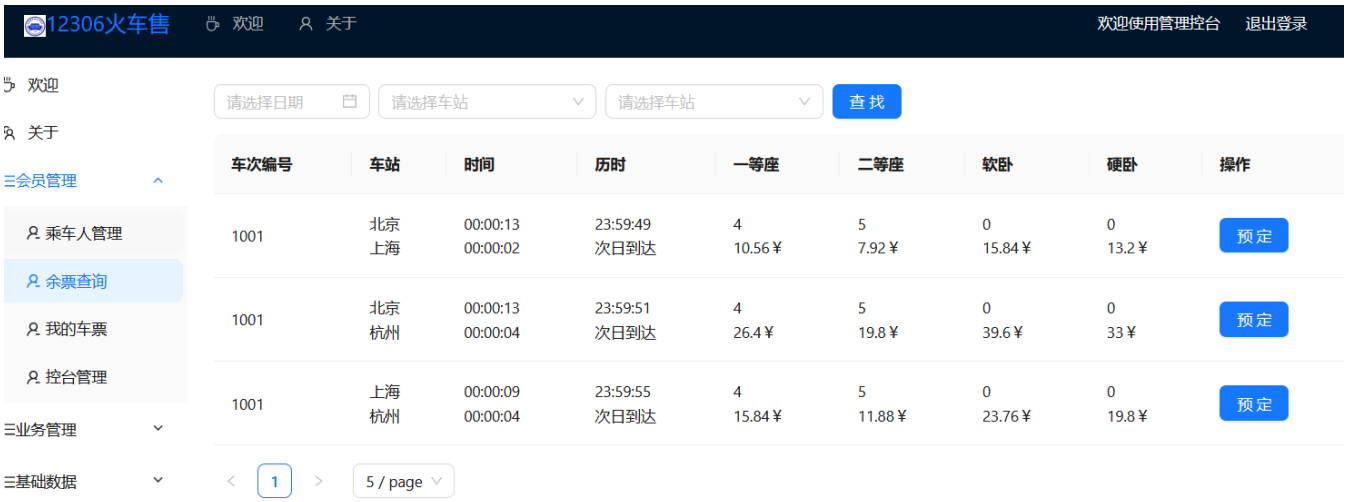
11.3.2 后端member模块

- 按照memberId获取购票乘客信息：PassengerController
- 当前登录的乘客可以给关联的乘客购买车票(例如：家庭成人)

```
@RestController
@RequestMapping("/passenger")
public class PassengerController {
    @Autowired
    private PassengerService passengerService;

    @GetMapping("/findByMemberId/{memberId}")
    public List<PassengerEntity> getPassengerByMemberId(@PathVariable Long memberId){
        return passengerService.list(new QueryWrapper<PassengerEntity>().eq("member_id",memberId));
    }
}
```

11.3.3 前端：车票预定TrainOrder.vue






```
<template>
  <div class="order-train">
    <span class="order-train-main">{{dailyTrainTicket.date}}</span> 
    <span class="order-train-main">{{dailyTrainTicket.trainCode}}</span>次 
    <span class="order-train-main">{{dailyTrainTicket.start}}</span>站
    <span class="order-train-main">({{dailyTrainTicket.startTime}})</span> 
    <span class="order-train-main">——</span> 
    <span class="order-train-main">{{dailyTrainTicket.end}}</span>站
    <span class="order-train-main">({{dailyTrainTicket.endTime}})</span> 

    <div class="order-train-ticket">
      <span v-for="item in seatTypes" :key="item.type">
        <span>{{item.desc}}</span>：
        <span class="order-train-ticket-main">{{item.price}}￥</span> 
        <span class="order-train-ticket-main">{{item.count}}</span> 张票
             
      </span>
    </div>
  </div>
  <!-------------- 分割线 ------------------->
  <a-divider></a-divider>
  <b>勾选购票的旅客:</b>  
  <a-checkbox-group v-model:value="passengerChecks" :options="passengerOptions" />
  <div class="order-tickets">
    <a-row class="order-tickets-header" v-if="tickets.length > 0">
      <a-col :span="2">乘客</a-col>
      <a-col :span="6">身份证</a-col>
      <a-col :span="4">票种</a-col>
      <a-col :span="4">座位类型</a-col>
    </a-row>
    <a-row class="order-tickets-row" v-for="ticket in tickets" :key="ticket.passengerId">
      <a-col :span="2">{{ticket.passengerName}}</a-col>
      <a-col :span="6">{{ticket.passengerIdCard}}</a-col>
      <a-col :span="4">
        <a-select v-model:value="ticket.passengerType" style="width: 100%">
          <a-select-option v-for="item in PASSENGER_TYPE_ARRAY" :key="item.code" :value="item.code">
            {{item.desc}}
```

```
          </a-select-option>
        </a-select>
      </a-col>
      <a-col :span="4">
        <a-select v-model:value="ticket.seatTypeCode" style="width: 100px">
          <a-select-option v-for="item in seatTypes" :key="item.code" :value="item.code">
            {{item.desc}}
          </a-select-option>
        </a-select>
      </a-col>
    </a-row>
  </div>
  <div v-if="tickets.length > 0">
    <a-button type="primary" size="large" @click="finishCheckPassenger">提交订单</a-button>
  </div>

  <a-modal v-model:visible="visible" title="请核对以下信息"
           style="top: 50px; width: 800px"
           ok-text="确认" cancel-text="取消"
           @ok="handleOk">
    <div class="order-tickets">
      <a-row class="order-tickets-header" v-if="tickets.length > 0">
        <a-col :span="3">乘客</a-col>
        <a-col :span="15">身份证</a-col>
        <a-col :span="3">票种</a-col>
        <a-col :span="3">座位类型</a-col>
      </a-row>
      <a-row class="order-tickets-row" v-for="ticket in tickets" :key="ticket.passengerId">
        <a-col :span="3">{{ticket.passengerName}}</a-col>
        <a-col :span="15">{{ticket.passengerIdCard}}</a-col>
        <a-col :span="3">
          <span v-for="item in PASSENGER_TYPE_ARRAY" :key="item.code">
            <span v-if="item.code === ticket.passengerType">
              {{item.desc}}
            </span>
          </span>
        </a-col>
        <a-col :span="3">
          <span v-for="item in seatTypes" :key="item.code">
            <span v-if="item.code === ticket.seatTypeCode">
              {{item.desc}}
            </span>
          </span>
        </a-col>
      </a-row>
      <!------------- 分割线 ------------------->
      <div v-if="chooseSeatType === 0" style="color: red;">
        您购买的车票不支持选座
        <div>12306规则：只有全部是一等座或全部是二等座才支持选座</div>
        <div>12306规则：余票小于一定数量时，不允许选座（20张票为例）</div>
      </div>
      <div v-else style="text-align: center">
        <a-switch class="choose-seat-item" v-for="item in seatColArray" :key="item.code"
                  v-model:checked="chooseSeatObj[item.code + '1']" :checked-children="item.desc" :un-
checked-children="item.desc" />
        <div v-if="tickets.length > 1">
          <a-switch class="choose-seat-item" v-for="item in seatColArray" :key="item.code"
                    v-model:checked="chooseSeatObj[item.code + '2']" :checked-children="item.desc" :un-
checked-children="item.desc" />
        </div>
        <div style="color: #999999">提示：您可以选择{{tickets.length}}个座位</div>
      </div>
      <br>
    </div>
  </a-modal>
  </template>

  <script setup lang="ts">
    import {SEAT_TYPE} from '@/utils/enum.ts'
    import {PASSENGER_TYPE_ARRAY,SEAT_COL_ARRAY} from '@/utils/enum.ts'
    import { message } from 'ant-design-vue';
    import {ref,watch,onMounted,computed} from 'vue'
```

```
    import axios from 'axios';

    //获取
    const param = sessionStorage.getItem('order')
    //参数对象
    let dailyTrainTicket:any =  {};
    //判断
    if(param){
      dailyTrainTicket = JSON.parse(param)
    }
    //座位类型
    const seatTypes:any = [];
    //遍历座位类型
    for (let KEY in SEAT_TYPE) {
        let key = KEY.toLowerCase();
        if (dailyTrainTicket[key] >= 0) {
            seatTypes.push({//添加到数组
                type: KEY,
                code: SEAT_TYPE[KEY]["code"],
                desc: SEAT_TYPE[KEY]["desc"],
                count: dailyTrainTicket[key],
                price: dailyTrainTicket[key + 'Price'],
            })
        }
    }
    //--------------购票人及数据列表-------------------
    const passengers:any = ref([]);
    const passengerOptions:any = ref([]);
    const passengerChecks = ref([]);
    const tickets:any = ref([]);
    const visible = ref(false);

    // 勾选或去掉某个乘客时，在购票列表中加上或去掉一张表
    watch(() => passengerChecks.value, (newVal, oldVal)=>{
      console.log("勾选乘客发生变化", newVal, oldVal)
      // 每次有变化时，把购票列表清空，重新构造列表
      tickets.value = [];
      // 遍历新勾选的乘客，构造购票列表
      passengerChecks.value.forEach((item:any) => tickets.value.push({
        passengerId: item.id ,
        passengerType: item.type,
        seatTypeCode: seatTypes[0].code,
        passengerName: item.name,
        passengerIdCard: item.idCard
      }))
    }, {immediate: true});

    // 查询乘客
    const handleQueryPassenger = () => {
      axios.get("/trainMember/passenger/findByMemberId/"+sessionStorage.getItem("memberId")).then((response)
=> {
        let data = response.data;
        if (data) {
          passengers.value = data;
          passengers.value.forEach((item:any) => passengerOptions.value.push({
            label: item.name,
            value: item
          }))
        } else {
          message.error( data.message);
        }
      });
    };

    //----------------选座购票-------------------------------
    // 购票列表，用于界面展示，并传递到后端接口，用来描述：哪个乘客购买什么座位的票（seat: "C1"表示座位类型）
    // {
    //   passengerId: 123,
    //   passengerType: "1",
    //   passengerName: "张三",
    //   passengerIdCard: "12323132132",
    //   seatTypeCode: "1",
```

```
//   seat: "C1"
// }
// 座位类型
// 0：不支持选座；1：选一等座；2：选二等座
const chooseSeatType:any = ref(0);
// 根据选择的座位类型，计算出对应的列，比如要选的是一等座，就筛选出ACDF，要选的是二等座，就筛选出ABCDF
const seatColArray = computed(() => {
  return SEAT_COL_ARRAY.filter(item => item.type === chooseSeatType.value);
});
// 选择的座位
// {
//   A1: false, C1: true, D1: false, F1: false,
//   A2: false, C2: false, D2: true, F2: false
// }
//选择的座位
const chooseSeatObj:any = ref({});
//初始化的座位
watch(() => seatColArray.value, () => {
  //清空之前的数据
  chooseSeatObj.value = {};
  // 初始化两排座位，都是未选中
  for (let i = 1; i <= 2; i++) {
    seatColArray.value.forEach((item) => {
      chooseSeatObj.value[item.code + i] = false;
    })
  }
  console.log("初始化两排座位，都是未选中：", chooseSeatObj.value);
}, {immediate: true});

// 点击提交订单
const finishCheckPassenger = () => {
  console.log("购票列表：", tickets.value);

  if (tickets.value.length > 5) {
    message.error('最多只能购买5张车票');
    return;
  }

  // 校验余票是否充足，购票列表中的每个座位类型，都去车次座位余票信息中，看余票是否充足
  // 前端校验不一定准，但前端校验可以减轻后端很多压力
  // 注意：这段只是校验，必须copy出seatTypesTemp变量来扣减，用原始的seatTypes去扣减，会影响真实的库存
  let seatTypesTemp = Tool.copy(seatTypes);
  for (let i = 0; i < tickets.value.length; i++) {
    let ticket = tickets.value[i];
    for (let j = 0; j < seatTypesTemp.length; j++) {
      let seatType = seatTypesTemp[j];
      // 同类型座位余票-1，这里扣减的是临时copy出来的库存，不是真正的库存，只是为了校验
      if (ticket.seatTypeCode === seatType.code) {
        seatType.count--;
        if (seatType.count < 0) {
          message.error(seatType.desc + '余票不足');
          return;
        }
      }
    }
  }
  console.log("前端余票校验通过");

  // 判断是否支持选座，只有纯一等座和纯二等座支持选座
  // 先筛选出购票列表中的所有座位类型，比如四张表：[1, 1, 2, 2]
  let ticketSeatTypeCodes = [];
  for (let i = 0; i < tickets.value.length; i++) {
    let ticket = tickets.value[i];
    ticketSeatTypeCodes.push(ticket.seatTypeCode);
  }
  // 为购票列表中的所有座位类型去重：[1, 2]
  const ticketSeatTypeCodesSet = Array.from(new Set(ticketSeatTypeCodes));
  console.log("选好的座位类型：", ticketSeatTypeCodesSet);
  if (ticketSeatTypeCodesSet.length !== 1) {
    console.log("选了多种座位，不支持选座");
    chooseSeatType.value = 0;
  } else {
```

```
      // ticketSeatTypeCodesSet.length === 1，即只选择了一种座位（不是一个座位，是一种座位）
      if (ticketSeatTypeCodesSet[0] === SEAT_TYPE.YDZ.code) {
        console.log("一等座选座");
        chooseSeatType.value = SEAT_TYPE.YDZ.code;
      } else if (ticketSeatTypeCodesSet[0] === SEAT_TYPE.EDZ.code) {
        console.log("二等座选座");
        chooseSeatType.value = SEAT_TYPE.EDZ.code;
      } else {
        console.log("不是一等座或二等座，不支持选座");
        chooseSeatType.value = 0;
      }

      // 余票小于20张时，不允许选座，否则选座成功率不高，影响出票
      if (chooseSeatType.value !== 0) {
        for (let i = 0; i < seatTypes.length; i++) {
          let seatType = seatTypes[i];
          // 找到同类型座位
          if (ticketSeatTypeCodesSet[0] === seatType.code) {
            // 判断余票，小于20张就不支持选座
            if (seatType.count < 20) {
              console.log("余票小于20张就不支持选座")
              chooseSeatType.value = 0;
              break;
            }
          }
        }
      }
    }
     // 弹出确认界面
     visible.value = true;
};
// 点击确认
const handleOk = () => {
  // 设置每张票的座位
  // 先清空购票列表的座位，有可能之前选了并设置座位了，但选座数不对被拦截了，又重新选一遍
  for (let i = 0; i < tickets.value.length; i++) {
    tickets.value[i].seat = null;
  }
  let i = -1;
  // 要么不选座位，要么所选座位应该等于购票数，即i === (tickets.value.length - 1)
  for (let key in chooseSeatObj.value) {
    if (chooseSeatObj.value[key]) {
      i++;
      if (i > tickets.value.length - 1) {
        message.error('所选座位数大于购票数');
        return;
      }
      tickets.value[i].seat = key;
    }
  }
  if (i > -1 && i < (tickets.value.length - 1)) {
    message.error('所选座位数小于购票数');
    return;
  }

  console.log("最终购票：", tickets.value);
  console.log("车次信息：", sessionStorage.getItem("order"));

  //车次信息
  const che:any = JSON.parse(sessionStorage.getItem("order"));

  axios({
    method:'post',
    url:'/trainMember/confirmOrder/saveConfirmOrder',
    data:{
      tickets:JSON.stringify(tickets.value),
      memberId:1001,
      date:che.date,
      trainCode:che.trainCode,
      start:che.start,
      end:che.end,
      status:'S',
```

```
          dailyTrainTicketId:che.id,

        }
      }).then((response) => {
        let data = response.data;
        if (data.success) {
          message.success("下单成功！");
          visible.value = false;
        } else {
          message.error(data.message);
        }
      });
    }
    //----------------------加载数据--------------------------
    onMounted(() => {
      handleQueryPassenger();
    });
  </script>

  <style>
  .order-train .order-train-main {
    font-size: 18px;
    font-weight: bold;
  }
  .order-train .order-train-ticket {
    margin-top: 15px;
  }
  .order-train .order-train-ticket .order-train-ticket-main {
    color: red;
    font-size: 18px;
  }

  .order-tickets {
    margin: 10px 0;
  }
  .order-tickets .ant-col {
    padding: 5px 10px;
  }
  .order-tickets .order-tickets-header {
    background-color: cornflowerblue;
    border: solid 1px cornflowerblue;
    color: white;
    font-size: 16px;
    padding: 5px 0;
  }
  .order-tickets .order-tickets-row {
    border: solid 1px cornflowerblue;
    border-top: none;
    vertical-align: middle;
    line-height: 30px;
  }

  .order-tickets .choose-seat-item {
    margin: 5px 5px;
  }
  </style>
```

11.3.4 选座购票

12306规则：
1、只有全部是一等座或全部是二等座才支持选座
2、余票小于一定数量时，不允许选择（例如10张为例）



11.3.5 后端：预定

- 数据表

```
CREATE TABLE `confirm_order` (
  `id` bigint NOT NULL COMMENT 'id',
  `member_id` bigint NOT NULL COMMENT '会员id',
  `date` date NOT NULL COMMENT '日期',
  `train_code` varchar(20) NOT NULL COMMENT '车次编号',
  `start` varchar(20) NOT NULL COMMENT '出发站',
  `end` varchar(20) NOT NULL COMMENT '到达站',
  `daily_train_ticket_id` bigint NOT NULL COMMENT '余票ID',
  `tickets` json NOT NULL COMMENT '车票',
  `status` char(1) NOT NULL COMMENT '订单状态|枚举[ConfirmOrderStatusEnum]',
  `create_time` datetime(3) DEFAULT NULL COMMENT '新增时间',
  `update_time` datetime(3) DEFAULT NULL COMMENT '修改时间',
  PRIMARY KEY (`id`),
  KEY `date_train_code_index` (`date`,`train_code`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci COMMENT='确认订单';
```

- 在common模块中添加订票枚举类

```
package com.hxx.enums;

public enum ConfirmOrderStatusEnum {

    INIT("I", "初始"),
    PENDING("P", "处理中"),
    SUCCESS("S", "成功"),
    FAILURE("F", "失败"),
    EMPTY("E", "无票"),
    CANCEL("C", "取消");

    private String code;

    private String desc;

    ConfirmOrderStatusEnum(String code, String desc) {
        this.code = code;
        this.desc = desc;
    }

    @Override    public String toString() {
        return "ConfirmOrderStatusEnum{" +
                "code='" + code + '\'' +
                ", desc='" + desc + '\'' +
                "} " + super.toString();
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
```

```
    }

    public void setDesc(String desc) {
        this.desc = desc;
    }

    public String getDesc() {
        return desc;
    }
}
```

- 在train-daily模块实现预定车票
  - service

```
@Service
public class ConfirmOrderServiceImpl extends ServiceImpl<ConfirmOrderMapper, ConfirmOrderEntity>
implements ConfirmOrderService {
    @Autowired
    private ConfirmOrderMapper confirmOrderMapper;
    @Override
    public Page<ConfirmOrderEntity> getConfirmOrderList(Page page, ConfirmOrderEntity
confirmOrderEntity) {
        return confirmOrderMapper.selectPage(page, new LambdaQueryWrapper<ConfirmOrderEntity>()
                .eq(confirmOrderEntity.getMemberId() != null, ConfirmOrderEntity::getMemberId,
                                    confirmOrderEntity.getMemberId())
                .eq(confirmOrderEntity.getStatus() != null, ConfirmOrderEntity::getStatus,
                                    confirmOrderEntity.getStatus())
                .eq(confirmOrderEntity.getTrainCode() != null, ConfirmOrderEntity::getTrainCode,
                                    confirmOrderEntity.getTrainCode())
                .eq(confirmOrderEntity.getDate() != null, ConfirmOrderEntity::getDate,
                                    confirmOrderEntity.getDate())
        );
    }
}
```

  - controller

```
package com.hxx.controller;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.hxx.entity.ConfirmOrderEntity;
import com.hxx.entity.DailyTrainTicketEntity;
import com.hxx.service.ConfirmOrderService;
import com.hxx.service.DailyTrainTicketService;
import com.hxx.util.R;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/confirmOrder")
public class ConfirmOrderController {
    @Autowired
    private ConfirmOrderService confirmOrderService;

    @Autowired
    private DailyTrainTicketService dailyTrainTicketService;

    /**
     * 查询订单
     */
    @GetMapping("confirmOrder")
    public R confirmOrder(Page page, ConfirmOrderEntity confirmOrderEntity){
        confirmOrderService.getConfirmOrderList(page,confirmOrderEntity);
        return R.ok().data("list",page.getRecords()).data("total",page.getTotal());
```

```
    }

    /**
     * 保存订单信息
     */
    @PostMapping("saveConfirmOrder")
    @Transactional
    public R save(@RequestBody ConfirmOrderEntity confirmOrderEntity){
        confirmOrderService.save(confirmOrderEntity);
        //减少余票
        DailyTrainTicketEntity dailyTrainTicketEntity =
            dailyTrainTicketService.getById(confirmOrderEntity.getDailyTrainTicketId());
        String tickets = confirmOrderEntity.getTickets();
        List tis = JSON.parseArray(tickets);
        for (Object ti : tis) {
            JSONObject jsonObject = JSON.parseObject(ti.toString());
            switch (jsonObject.get("seatTypeCode").toString()){
                case "1":
                    dailyTrainTicketEntity.setYdz(dailyTrainTicketEntity.getYdz()-1);
                    break;
                case "2":
                    dailyTrainTicketEntity.setEdz(dailyTrainTicketEntity.getEdz()-1);
                    break;
                case "3":
                    dailyTrainTicketEntity.setRw(dailyTrainTicketEntity.getRw()-1);
                case "4":
                    dailyTrainTicketEntity.setYw(dailyTrainTicketEntity.getYw()-1);
                }
                dailyTrainTicketService.updateById(dailyTrainTicketEntity);
        }
        return R.ok();
    }
}
```

11.3.6 前端

- 完成保存车票订单：TrainOrder.vue
- 添加数据到configrm_order表，同时减掉daily_train_ticket的余票数



11.3.7 订单信息




```
<template>
  <p>
    <a-space>
      <a-button type="primary" @click="handleQuery()">刷新</a-button>
    </a-space>
  </p>
  <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
    <template #bodyCell="{ column, record }">
      <template v-if="column.dataIndex === 'tickets'">
        <span v-for="item in SEAT_TYPE_ARRAY" :key="item.code">
          <span v-if="item.code === JSON.parse(record.tickets)[0].seatTypeCode">
            {{ item.desc }}<br>{{ JSON.parse(record.tickets).passengerName }}
          </span>
        </span>
      </template>
      <template v-else-if="column.dataIndex === 'status'">
        <span v-for="item in CONFIRM_ORDER_STATUS_ARRAY" :key="item.code">
          <span v-if="item.code === record.status">
            {{item.desc}}
          </span>
        </span>
      </template>
    </template>
  </a-table>
  <br><TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
</template>

<script setup lang="ts">
import { ref, onMounted } from 'vue';
import axios from "axios";
import {CONFIRM_ORDER_STATUS_ARRAY, SEAT_TYPE_ARRAY} from '@/utils/enum.ts'

interface confirmOrder {
      id: undefined,
      memberId: undefined,
      date: undefined,
      trainCode: undefined,
      start: undefined,
      end: undefined,
      dailyTrainTicketId: undefined,
      tickets: undefined,
      status: undefined,
      createTime: undefined,
      updateTime: undefined,
}
const columns = [
      {
      title: '日期',
      dataIndex: 'date',
      key: 'date',
      },
      {
      title: '车次编号',
      dataIndex: 'trainCode',
```

```
      key: 'trainCode',
      },
      {
      title: '出发站',
      dataIndex: 'start',
      key: 'start',
      },
      {
      title: '到达站',
      dataIndex: 'end',
      key: 'end',
      },
      {
      title: '车票',
      dataIndex: 'tickets',
      key: 'tickets',
      },
      {
      title: '订单状态',
      dataIndex: 'status',
      key: 'status',
      },
];
//数据源
const dataSource = ref<confirmOrder[]>([]);
//--------------分页----------------
const total = ref(0);
//分页参数
const queryParams=ref({
  current:1,
  size:5,
  //memberId:sessionStorage.getItem('mid'),
})

//当前页码变化
const onChange = (page:number,pageSize:number) => {
    queryParams.value.current = page
    queryParams.value.size = pageSize
    showData()
  };
//加载数据
const showData = () => {
  axios({
    method: 'get',
    url: '/trainDaily/confirmOrder/confirmOrder',
    params: queryParams.value
  }).then(({data})=>{
        total.value = data.data.total;
        dataSource.value = data.data.list;
    })
}

//-------------------分页结束--------------------
//-------------------查询开始--------------------
const handleQuery=()=>{
  showData();
}

//页面加载数据
onMounted(()=>{
  showData();
})
</script>
```

11.3.8 我的车票

- 根据登录会员id获取车票




```
<template>
    <p>
      <a-space>
        <a-button type="primary" @click="handleQuery()">刷新</a-button>
      </a-space>
    </p>
    <a-table :dataSource="dataSource"  :columns="columns" :pagination="false">
      <template #bodyCell="{ column, record }">
        <template v-if="column.dataIndex === 'tickets'">
          <span v-for="item in SEAT_TYPE" :key="item.code">
            <span v-if="item.code === JSON.parse(record.tickets).seatTypeCode">
              {{ item.desc }}<br>{{ JSON.parse(record.tickets).passengerName }}
            </span>
          </span>
        </template>
        <template v-else-if="column.dataIndex === 'status'">
          <span v-for="item in CONFIRM_ORDER_STATUS_ARRAY" :key="item.code">
            <span v-if="item.code === record.status">
              {{item.desc}}
            </span>
          </span>
        </template>
      </template>
    </a-table>
    <br><TrainPagination :total="total" :pageSizeOptions="['5','7','9']" @change="onChange"/>
  </template>

  <script setup lang="ts">
  import { ref, onMounted } from 'vue';
  import axios from "axios";
  import {CONFIRM_ORDER_STATUS_ARRAY,SEAT_TYPE} from '@/utils/enum.ts'

  interface confirmOrder {
        id: undefined,
        memberId: undefined,
        date: undefined,
        trainCode: undefined,
        start: undefined,
        end: undefined,
        dailyTrainTicketId: undefined,
        tickets: undefined,
        status: undefined,
        createTime: undefined,
        updateTime: undefined,
  }
  const columns = [
        {
        title: '日期',
        dataIndex: 'date',
        key: 'date',
        },
        {
```

```
        title: '车次编号',
        dataIndex: 'trainCode',
        key: 'trainCode',
        },
        {
        title: '出发站',
        dataIndex: 'start',
        key: 'start',
        },
        {
        title: '到达站',
        dataIndex: 'end',
        key: 'end',
        },
        {
        title: '车票',
        dataIndex: 'tickets',
        key: 'tickets',
        },
        {
        title: '订单状态',
        dataIndex: 'status',
        key: 'status',
        },
];
//数据源
const dataSource = ref<confirmOrder[]>([]);
 //--------------分页----------------
 const total = ref(0);
 //分页参数
 const queryParams=ref({
   current:1,
   size:5,
   memberId:sessionStorage.getItem('memberId'),
 })

 //当前页码变化
 const onChange = (page:number,pageSize:number) => {
   queryParams.value.current = page
   queryParams.value.size = pageSize
   showData()
 };
 //加载数据
 const showData = () => {
   axios({
     method: 'get',
     url: '/trainDaily/confirmOrder/confirmOrder',
     params: queryParams.value
   }).then(({data})=>{
       total.value = data.data.total;
       dataSource.value = data.data.list;
   })
 }

 //--------------------分页结束--------------------
 //--------------------查询开始--------------------
 const handleQuery=()=>{
   console.log('------queryParams------>>',queryParams.value);
   console.log('------------>>',sessionStorage.getItem('memberId'));
   showData();
 }

 //页面加载数据
 onMounted(()=>{
   showData();
 })
 </script>
```

十二、高并发设置

12.1 使用同步锁synchronized

- 多节点时执行不能完成超卖的问题（单机可以保证不超卖，但是，执行效率很低）,只能锁住当前单机的线程方法
- 特点：
    - 锁定的对象必须是同一个实例，否则无法实现同步。
    - 过度使用同步可能会导致性能问题，因为线程在等待锁时会处于阻塞状态。
    - 死锁是使用同步时需要注意的一个问题，当两个或多个线程互相等待对方释放锁时，就会发生死锁。

```
/**
 * 保存订单信息
 */
@PostMapping("saveConfirmOrder")
@Transactional
@Synchronized
public R save(@RequestBody ConfirmOrderEntity confirmOrderEntity){
    confirmOrderService.save(confirmOrderEntity);
    //减少余票
    DailyTrainTicketEntity dailyTrainTicketEntity =
        dailyTrainTicketService.getById(confirmOrderEntity.getDailyTrainTicketId());
    String tickets = confirmOrderEntity.getTickets();
    List tis = JSON.parseArray(tickets);
    for (Object ti : tis) {
        JSONObject jsonObject = JSON.parseObject(ti.toString());
        switch (jsonObject.get("seatTypeCode").toString()){
            case "1":
                dailyTrainTicketEntity.setYdz(dailyTrainTicketEntity.getYdz()-1);
                break;
            case "2":
                dailyTrainTicketEntity.setEdz(dailyTrainTicketEntity.getEdz()-1);
                break;
            case "3":
                dailyTrainTicketEntity.setRw(dailyTrainTicketEntity.getRw()-1);
            case "4":
                dailyTrainTicketEntity.setYw(dailyTrainTicketEntity.getYw()-1);
            }
            dailyTrainTicketService.updateById(dailyTrainTicketEntity);
    }
    return R.ok();
}
```

12.2 synchronized区别redis 分布式锁

- `synchronized` 关键字和 Redis 分布式锁是两种不同的同步机制.

12.2.1 实现机制

- synchronized 是 Java 语言内置的同步机制，它依赖于 Java 虚拟机（JVM）来实现线程间的同步。当一个线程访问同步代码块或同步方法时，它会自动获取对象的内置锁（也称为监视器锁或互斥锁），其他线程尝试访问同一对象的同步代码块或方法时会被阻塞，直到锁被释放。
- Redis 分布式锁是通过 Redis 服务器实现的，它依赖于 Redis 的数据结构和命令来实现。通常使用 SETNX（SET if Not eXists）命令来设置一个键值对，如果键不存在则设置成功并返回 1，表示获取了锁。其他客户端在尝试获取锁时，如果键已存在，则会失败并返回 0，表示锁已被占用。

12.2.2 使用场景

- synchronized 适用于单个 Java 应用程序内部的线程同步，适用于同步方法或代码块，确保同一时间只有一个线程可以执行特定的代码段。
- Redis 分布式锁适用于跨多个进程或多个机器的分布式系统中的线程同步，它允许多个进程或服务实例在共享资源上进行互斥访问。

12.2.3 性能和可伸缩性

- synchronized 由于是基于 JVM 的内置机制，其性能开销相对较小，但它的可伸缩性受限于单个 JVM 实例。
- Redis 分布式锁的性能开销相对较大，因为它涉及到网络通信和远程数据操作。然而，由于 Redis 的高性能和可伸缩性，分布式锁可以很好地扩展到多个服务器实例。

12.2.4 容错性和可靠性

- synchronized 锁的可靠性依赖于单个 JVM 实例的稳定性。如果 JVM 崩溃，锁可能会丢失。
- Redis 分布式锁的可靠性更高，因为 Redis 通常配置为持久化数据，即使服务器崩溃，锁信息也可以通过持久化数据恢复。

12.2.5 死锁和超时处理

- synchronized 本身不提供超时机制，如果一个线程在同步块中发生死锁，其他线程将无限期等待。
- Redis 分布式锁通常会设置一个超时时间（通过 expire参数设置），如果锁持有者在超时时间内没有释放锁，锁将自动释放，避免死锁。

12.2.6 如何选择

- 对于单个应用内部的线程同步，synchronized 是一个简单有效的选择。
- 对于分布式环境中的线程同步，尤其是需要跨多个服务实例的场景，Redis 分布式锁提供了更好的解决方案。

12.3 使用redis分布式锁

12.3.1 pom依赖

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

12.3.2 redis锁的使用

```
setnx区别setIfAbsent:

1、在Redis中，SETNX命令和SET命令的IfAbsent选项都用于设置键值对。
2、SETNX是一个原子命令，它只在键不存在时，才对键进行设置操作。
3、而SET命令的IfAbsent选项则是在已存在键的情况下不进行任何操作，而是只在键不存在时才进行设置。

4、在Spring框架中，StringRedisTemplate或RedisTemplate的opsForValue方法提供了setIfAbsent方法，它的功能与SETNX命令相似，都是只在键不存在时才进行设置。
```

```
@PostMapping("saveTest")
@Transactional
public R saveTest(@RequestBody ConfirmOrderEntity confirmOrderEntity){
    //设置key为订单日期+车次：同一天的同一个车次订单只能提交一次
    String key = confirmOrderEntity.getDate()+"-"+confirmOrderEntity.getTrainCode();
    //不存在就设置否则不添加
    Boolean setIfAbsent = stringRedisTemplate.opsForValue()
                                        .setIfAbsent(key, "1", 5, TimeUnit.SECONDS);
    if(setIfAbsent){
        System.out.println("获取了锁");
    }else{
        System.out.println("没有抢到锁");
        throw new RuntimeException("请稍候...");
    }
    //释放锁
    stringRedisTemplate.delete(key);
    return R.ok();
}
```

- 分布式锁能解决超卖的问题，但是，会剩余车票
    - 主要的原因就是没有办法确定执行需要的时间，也就没有办法设置超时时间

- 因为，某个线程异常或操作超时，而时间到了，所以，就释放锁了
- 另一个获取到锁后操作，但是，上一个线程没有操作完，也就是在同时执行抢票也会超卖

12.4 redis看门狗锁

- 特点：可以自动完成超时时间的管理，可以理解为一个守护线程，在操作没有完成前不会释放锁。

12.4.1 使用redisson完成看门狗锁

- redisson工具自带看门狗锁
- pom依赖

```
<dependency>
    <groupId>org.redisson</groupId>
    <artifactId>redisson-spring-boot-starter</artifactId>
    <version>3.21.0</version>
</dependency>
```

```
package com.hxx.controller;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.hxx.entity.ConfirmOrderEntity;
import com.hxx.entity.DailyTrainTicketEntity;
import com.hxx.service.ConfirmOrderService;
import com.hxx.service.DailyTrainTicketService;
import com.hxx.util.R;
import lombok.Synchronized;
import org.redisson.api.RLock;
import org.redisson.api.RedissonClient;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.redis.core.StringRedisTemplate;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.concurrent.TimeUnit;

@RestController
@RequestMapping("/confirmOrder")
public class ConfirmOrderController {
    @Autowired
    private ConfirmOrderService confirmOrderService;

    @Autowired
    private DailyTrainTicketService dailyTrainTicketService;

    @Autowired
    private StringRedisTemplate stringRedisTemplate;

    @Autowired
    private RedissonClient redissonClient;

    /**
     * 保存订单信息
     */
    @PostMapping("saveConfirmOrder")
    @Transactional
    public R save(@RequestBody ConfirmOrderEntity confirmOrderEntity){
        //设置key为订单日期+车次：同一天的同一个车次订单只能提交一次
        String key = confirmOrderEntity.getDate()+"-"+confirmOrderEntity.getTrainCode();
        RLock lock = null;
        try {
            /*
            //不带看门狗lock.tryLock(10,10,TimeUnit.SECONDS)
            waitTime：等待时间（就是等待获取锁的时间）
            leaseTime：超时时间（获取锁后的超时时间）
```

```
            TimeUnit：时间单位
            */
            //使用看门狗
            lock = redissonClient.getLock(key);
           ////0表示不设置超时时间，看门狗会监控执行后自动释放锁
            if (lock.tryLock(0,TimeUnit.SECONDS)){
                System.out.println("获取锁成功");
                //保存订单
                confirmOrderService.save(confirmOrderEntity);
                //减少余票
                DailyTrainTicketEntity dailyTrainTicketEntity =
                       dailyTrainTicketService.getById(confirmOrderEntity.getDailyTrainTicketId());
                String tickets = confirmOrderEntity.getTickets();
                List tis = JSON.parseArray(tickets);
                for (Object ti : tis) {
                    JSONObject jsonObject = JSON.parseObject(ti.toString());
                    switch (jsonObject.get("seatTypeCode").toString()){
                        case "1":
                            dailyTrainTicketEntity.setYdz(dailyTrainTicketEntity.getYdz()-1);
                            break;
                        case "2":
                            dailyTrainTicketEntity.setEdz(dailyTrainTicketEntity.getEdz()-1);
                            break;
                        case "3":
                            dailyTrainTicketEntity.setRw(dailyTrainTicketEntity.getRw()-1);
                        case "4":
                            dailyTrainTicketEntity.setYw(dailyTrainTicketEntity.getYw()-1);
                    }
                    //更新余票
                    dailyTrainTicketService.updateById(dailyTrainTicketEntity);
                }
            }
        }catch (Exception e){
            return R.error().msg("订单提交失败");
        }finally {
            //必须是当前线程才可以释放
            if (lock!=null && lock.isHeldByCurrentThread()){
                //释放锁
                lock.unlock();
            }
        }
        return R.ok();
    }
}
```

12.5 Redis红锁

12.5.1 红锁(因为redis的图标是红色，所以称为红锁)

- Redis红锁（Redlock）是一种分布式锁算法，由 Redis 的创造者 Salvatore Sanfilippo（也被称为 antirez）提出。它旨在在分布式系统中提供一个更安全、更可靠的锁机制，以防止在单点故障或网络分区的情况下出现数据不一致的问题。

12.5.2 红锁算法的基本步骤

- 红锁算法的核心思想是通过多个独立的 Redis 实例来实现锁的获取和释放，从而提高锁的可靠性。

1、获取当前时间：客户端记录获取锁之前的时间戳。

2、尝试获取锁：客户端尝试在多个 Redis 实例上依次获取锁，每个实例的获取操作都是独立的。为了提高效率，客户端可以并行地向所有实例发送锁请求

3、计算有效时间：客户端计算出锁的有效时间，这个时间是锁的默认过期时间减去步骤1中记录的时间戳。如果客户端在所有实例上都成功获取了锁，它会检查这个有效时间是否大于0。如果有效时间小于或等于0，则认为锁获取失败。

4、检查锁的有效性：客户端需要检查在步骤3中获取的锁是否仍然有效。这一步是必要的，因为可能在客户端获取锁的过程中，锁已经过期了。客户端需要确保它持有的锁是有效的。

5、锁的释放：如果客户端成功获取了锁，并且锁是有效的，那么它就可以执行需要同步的代码。当客户端完成任务后，它需要在所有实例上释放锁。

12.5.3 提高锁的可靠性的方式

- 容错性：即使部分 Redis 实例宕机或网络分区，只要大多数实例仍然可用，客户端仍然可以获取和释放锁。
- 避免单点故障：通过在多个独立的 Redis 实例上获取锁，红锁算法避免了单个 Redis 实例故障导致的锁失效问题。
- 避免死锁：红锁算法通过设置锁的有效时间来避免死锁，如果客户端在有效时间内无法完成任务，锁将自动释放。

12.5.4 不推荐使用

- 红锁算法虽然提高了分布式锁的可靠性，但实现起来相对复杂，需要客户端和服务器端的紧密配合。
- 红锁算法并不是 Redis 官方提供的功能，而是由社区提出的一种算法，因此在使用时需要仔细考虑其适用场景和潜在风险。

12.5.5 伪代码

```
RedissonRedLock redissonRedLock = new RedissonRedLock(redissonClient.getLock("节点1lock"),
                                                      redissonClient.getLock("节点2lock")...);
boolean tryLock = redissonRedLock.tryLock(10, 10, TimeUnit.SECONDS);
```

十三、RocketMQ信息队列

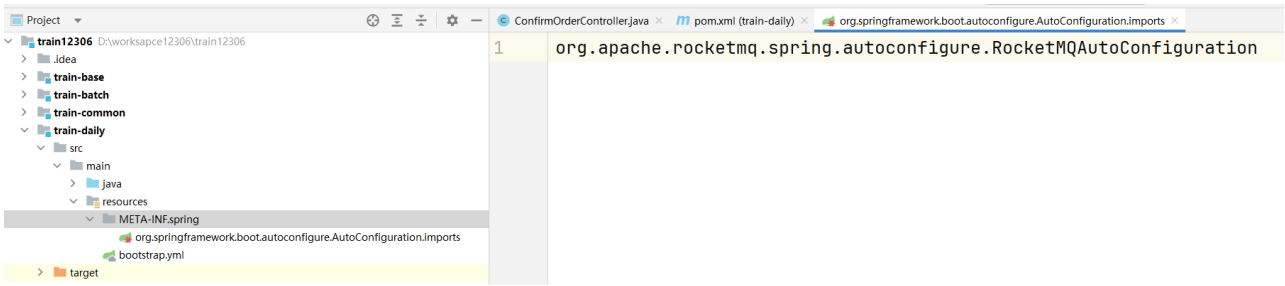
13.1 消息队列购票：train-daily模块

13.1.1 pom依赖

```
<dependency>
    <groupId>org.apache.rocketmq</groupId>
    <artifactId>rocketmq-spring-boot-starter</artifactId>
    <version>2.0.2</version>
</dependency>
```

- springboot3必须配置自动文件，否则注入rocketMQ对象为null

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports/
org.apache.rocketmq.spring.autoconfigure.RocketMQAutoConfiguration
```



- yaml配置文件内容

```
server:
  port: 8003
spring:
  application:
    name: trainDaily
  cloud:
    nacos:
      config:
        server-addr: localhost:8848
        username: nacos
        password: nacos
  datasource:
    url: jdbc:mysql://127.0.0.1:3306/train?useUnicode=true&characterEncoding=utf-
8&useSSL=false&serverTimezone=Asia/Shanghai
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: root
    password: root
  jackson:
```

```
    date-format: YYYY-MM-dd
  rocketmq:
    name-server: http://localhost:9876
    producer:
      group: confirmOrder
```

13.2 发送订票mq消息

```
package com.hxx.mq;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONArray;
import com.alibaba.fastjson.JSONObject;
import com.hxx.entity.ConfirmOrderEntity;
import com.hxx.enums.SeatTypeEnum;
import com.hxx.mapper.ConfirmOrderMapper;
import com.hxx.service.ConfirmOrderService;
import com.hxx.util.SnowUtil;
import jakarta.annotation.Resource;
import org.apache.rocketmq.spring.core.RocketMQTemplate;
import org.slf4j.Logger;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class BeforeConfirmOrderService {

    private static final Logger logger = org.slf4j.LoggerFactory.getLogger(BeforeConfirmOrderService.class);

    @Autowired
    private ConfirmOrderMapper confirmOrderMapper;

    @Autowired
    private ConfirmOrderService confirmOrderService;

    @Resource
    private RocketMQTemplate rocketMQTemplate;

    public void beforeConfirmOrder(ConfirmOrderEntity confirmOrderEntity) {
        //保存订单信息：确保购票无论是否能抢到锁
        JSONArray objects = JSON.parseArray(confirmOrderEntity.getTickets());
        for (Object object : objects) {
            JSONObject jsonObject = JSON.parseObject(object.toString());
            //座位类型
            Object seatTypeCode = jsonObject.get("seatTypeCode");
            //根据枚举类获取对应的座位类型
            SeatTypeEnum seatType = SeatTypeEnum.getEnumByCode(seatTypeCode.toString());
            // 保存订单信息
            confirmOrderEntity.setTickets(jsonObject.toJSONString());//重新设置车票信息
            confirmOrderEntity.setId(SnowUtil.getSnowflakeId());//重新设置id防止读取缓存id重复
            confirmOrderService.save(confirmOrderEntity);
            //发送消息
            logger.info("发送消息：{}", confirmOrderEntity);
            rocketMQTemplate.convertAndSend("confirmOrder", confirmOrderEntity);
            logger.info("发送消息成功：{}", confirmOrderEntity);
        }
    }
}
```

13.3 执行订单

```
@RestController
@RequestMapping("/confirmOrder")
public class ConfirmOrderController {

    @Autowired
    private ConfirmOrderService confirmOrderService;

    @Autowired
```

```
    private BeforeConfirmOrderService beforeConfirmOrderService;
    /**
     * 查询订单
     */
    @GetMapping("confirmOrder")
    public R confirmOrder(Page page, ConfirmOrderEntity confirmOrderEntity){
        confirmOrderService.getConfirmOrderList(page,confirmOrderEntity);
        return R.ok().data("list",page.getRecords()).data("total",page.getTotal());
    }

    /**
     * 保存订单信息
     */
    @PostMapping("saveConfirmOrder")
    @Transactional
    public R save(@RequestBody ConfirmOrderEntity confirmOrderEntity){
        //预处理订单
        beforeConfirmOrderService.beforeConfirmOrder(confirmOrderEntity);
        //提交订单
        return R.ok();
    }
}
```

13.4 监听消息MQ

```
package com.hxx.mq;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONArray;
import com.alibaba.fastjson.JSONObject;
import com.hxx.entity.ConfirmOrderEntity;
import com.hxx.entity.DailyTrainTicketEntity;
import com.hxx.enums.SeatTypeEnum;
import com.hxx.service.DailyTrainTicketService;
import org.apache.rocketmq.common.message.MessageExt;
import org.apache.rocketmq.spring.annotation.RocketMQMessageListener;
import org.apache.rocketmq.spring.core.RocketMQListener;
import org.redisson.api.RLock;
import org.redisson.api.RedissonClient;
import org.slf4j.Logger;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.concurrent.TimeUnit;

@Service
@RocketMQMessageListener(topic = "confirmOrder", consumerGroup = "confirmOrder")
public class ConfirmOrderConsumer implements RocketMQListener<MessageExt> {
    private static final Logger logger = org.slf4j.LoggerFactory.getLogger(ConfirmOrderConsumer.class);
    @Autowired
    private RedissonClient redissonClient;
    @Autowired
    private DailyTrainTicketService dailyTrainTicketService;
    @Override
    public void onMessage(MessageExt messageExt) {
        byte[] body = messageExt.getBody();
        logger.info("------------------------------------------------>>>>接收到消息,byte[]：{}", new
String(body));
        // 处理消息
        ConfirmOrderEntity confirmOrderEntity = JSON.parseObject(body, ConfirmOrderEntity.class);
        logger.info("接收到消息,confirmOrderEntity：{}",confirmOrderEntity);
        //设置key为订单日期+车次：同一天的同一个车次订单只能提交一次
        String key = confirmOrderEntity.getDate()+"-"+confirmOrderEntity.getTrainCode();
        //加锁
        RLock lock = null;
        try {
            //使用看门狗
            lock = redissonClient.getLock(key);
            //0表示不设置超时时间，看门狗会监控执行后自动释放锁
```

```
            if (lock.tryLock(0, TimeUnit.SECONDS)) {//带看门狗
                DailyTrainTicketEntity dailyTrainTicketEntity =
                        dailyTrainTicketService.getById(confirmOrderEntity.getDailyTrainTicketId());
                //根据tickets车票数量生成订单
                JSONObject jsonObject = JSON.parseObject(confirmOrderEntity.getTickets());
                switch (jsonObject.get("seatTypeCode").toString()) {
                    case "1":
                        dailyTrainTicketEntity.setYdz(dailyTrainTicketEntity.getYdz() - 1);
                        break;
                    case "2":
                        dailyTrainTicketEntity.setEdz(dailyTrainTicketEntity.getEdz() - 1);
                        break;
                    case "3":
                        dailyTrainTicketEntity.setRw(dailyTrainTicketEntity.getRw() - 1);
                    case "4":
                        dailyTrainTicketEntity.setYw(dailyTrainTicketEntity.getYw() - 1);
                }
                dailyTrainTicketService.updateById(dailyTrainTicketEntity);
            }
        }catch (Exception e){
            e.printStackTrace();
        }finally {
            //必须是当前线程才可以释放
            if (lock!=null && lock.isHeldByCurrentThread()){
                //释放锁
                lock.unlock();
            }
        }

    }
}
```

- 测试预定票

扩展：自定义组件

一、分页组件

```
<template>
  <a-pagination
    :total="total"
    :current='current'
    :pageSize='pageSize'
    :pageSizeOptions="pageSizeOptions"
    show-size-changer
    :show-quick-jumper="showQuickJumper"
    :show-sizer="showSizer"
    @change="handleChange"
  />
</template>

<script setup lang="ts">
import { PropType,ref } from 'vue';

//参数属性
defineProps({
  total: {
    type: Number as PropType<number>,
    required: true,
  },
  pageSize: {
    type: Number as PropType<number>,
    required: false,
  },
  current: {
    type: Number as PropType<number>,
```

```
    default: 1,
    required: false,
  },
  pageSizeOptions: {
    type: String as PropType<string>,
    default: ['5','10','20'],
  },
  showSizer: {
    type: Boolean as PropType<boolean>,
    default: true,
  },
  showQuickJumper: {
    type: Boolean as PropType<boolean>,
    default: true,
  },
  onChange: {
    type: Function as PropType<(page: number, pageSize: number) => void>,
    default: () => {},
  }
})


const current = ref<number>(1)
const pageSize = ref<number>(5)

const emit = defineEmits(['change'])

//当前页码改变
const handleChange = (_current: number,_pageSize:number) => {
  current.value = _current
  pageSize.value = _pageSize
  emit('change', _current, _pageSize)
}

</script>
```

使用组件

```
<a-table :dataSource="data" :columns="columns" :pagination="false"></a-table>
<TrainPagination :total="total" :pageSizeOptions="['5','7']" @change="onChange"></TrainPagination>

<script>
    //分页参数
    const queryParams=ref({
       current:1,
       size:5,
       type:null,
      name:''
    })
    //事件
    const onChange = (page:number,pageSize:number) => {
        queryParams.value.current = page
        queryParams.value.size = pageSize
        console.log('---------------->>>',page,pageSize);
        showData()
      };
      //加载数据
      const showData = () => {
        axios({
          method: 'get',
          url: '/member/passenger/page',
          params: queryParams.value
        }).then(res=>{
            let result = res.data;
            total.value = result.data.total;
            data.value = result.data.list;
        })
      }
</script>
```

二、车站下拉框

```
<template>
    <a-select v-model:value="name" show-search allowClear
        :filterOption="filterNameOption"
        @change="onChange" placeholder="请选择车站"
        :style="'width: ' + localWidth">
        <a-select-option v-for="item in stations" :key="item.name"
        :value="item.name" :label="item.name + item.namePinyin + item.namePy">
        {{item.name}} {{item.namePinyin}} ~ {{item.namePy}}
        </a-select-option>
    </a-select>
  </template>
  <script lang="ts" setup>
  import { ref, onMounted,watch,defineProps } from 'vue'
  import axios from 'axios'
  import {notification} from 'ant-design-vue'

   // 定义属性
   const props = defineProps({
        modelValue:{type:String,default:''},
        width: {type:String,default:'100%'}
   })
   // 名称
   const name = ref<string>()
   // 车站
   const stations = ref<any[]>([])
   //下列宽度
   const localWidth = ref<any>(props.width)
   //使用工具判断
   if(Tool.isEmpty(props.width)){
        localWidth.value = '100%'
   }
   //监听watch，动态获取数据
   watch(()=>props.modelValue,()=>{
        name.value = props.modelValue
   }, {immediate: true})

   //获取所有车站数据
   const getStations = async ()=>{
    await axios.get('/trainBase/station/all').then(res=>{
        let data = res.data
        if(data.success){
            stations.value = data.data.list
        }else{
            notification.error({message:data.msg})
        }
    })
   }
   //过滤车站数据
   const filterNameOption = (inputValue:string, option:any) => {
        return option.label.toLowerCase().indexOf(inputValue.toLocaleLowerCase()) > -1
   }

   // 定义事件
   const emit = defineEmits(['update:modelValue','change']);

   //响应给父组件
   const onChange = (value:string)=>{
        emit('update:modelValue',value)
        let station = stations.value.filter(item=>item.code === value)[0]
        if(Tool.isEmpty(station)){
            station={}
        }
        emit('change',station)
   }
   //加载数据
   onMounted(()=>{
        getStations()
    })
  </script>
```

三、车次下拉框

```
<template>
    <a-select v-model:value="trainCode" show-search allowClear
        :filterOption="filterTrainCodeOption"
        @change="onChange" placeholder="请选择车次"
        :style="'width: ' + localWidth">
        <a-select-option v-for="item in trains" :key="item.code"
            :value="item.code" :label="item.code + item.start + item.end">
            {{item.code}} {{item.start}} ~ {{item.end}}
        </a-select-option>
    </a-select>
</template>
<script lang="ts" setup>
import { ref, onMounted,watch,defineProps } from 'vue'
import axios from 'axios'
import {notification} from 'ant-design-vue'

    // 定义属性
    const props = defineProps({
        modelValue:{type:String,default:''},
        width: {type:String,default:'100%'}
    })
    // 名称
    const trainCode = ref<string>()
    // 车站
    const trains = ref<any[]>([])
    //下列宽度
    const localWidth = ref<any>(props.width)
    //使用工具判断
    if(Tool.isEmpty(props.width)){
        localWidth.value = '100%'
    }
    //监听watch，动态获取数据
    watch(()=>props.modelValue,()=>{
     trainCode.value = props.modelValue
    }, {immediate: true})

    //获取所有车次数据
    const getAllTrain = async ()=>{
     await axios.get('/trainBase/train/all').then(res=>{
        let data = res.data
        if(data.success){
            trains.value = data.data.list
        }else{
            notification.error({message:data.msg})
        }
     })
    }
    //过滤车站数据
    const filterTrainCodeOption = (inputValue:string, option:any) => {
        return option.label.toLowerCase().indexOf(inputValue.toLocaleLowerCase()) > -1
    }

    // 定义事件
    const emit = defineEmits(['update:modelValue','change']);

    //响应给父组件
    const onChange = (value:string)=>{
        emit('update:modelValue',value)
        let trainName = trains.value.filter(item=>item.code === value)[0]
        if(Tool.isEmpty(trainName)){
            trainName={}
        }
        emit('change',trainName)
    }
    //加载数据
    onMounted(()=>{
     getAllTrain()
```

```
  })
</script>
```

四、左侧菜单添加权限管理

```
<template>
  <a-layout-sider width="200" style="background: #fff">
  <a-menu
    mode="inline"
    :selectedKeys="selectedKeys"
    :style="{ height: '100%', borderRight: 0 }">
    <template v-for="item in menuData" :key="item.key">
      <a-sub-menu v-if="item.children" :key="item.key" :hidden="item.title==='会员管理'?false:!perm">
        <template #title>
          <span><UnorderedListOutlined />{{ item.title }}</span>
        </template>
        <a-menu-item
          v-for="subItem in item.children"
          :key="subItem.key"
        >
        <router-link :to="subItem.key">
          <UserDeleteOutlined /> {{ subItem.title }}
        </router-link>
        </a-menu-item>
      </a-sub-menu>
      <a-menu-item v-else :key="item.id" :hidden="false">
        <div v-if="item.title==='欢迎'">
          <router-link :to="item.key">
             <coffee-outlined />   {{ item.title }}
          </router-link>
        </div>
        <div v-else>
          <router-link :to="item.key">
              <TeamOutlined />   {{ item.title }}
          </router-link>
        </div>
      </a-menu-item>
    </template>
  </a-menu>
</a-layout-sider>
</template>

<script setup lang="ts">
import {onMounted, ref, watch} from 'vue';
import router from "@/router";

//菜单选中
const selectedKeys = ref<string[]>([]);

//权限参数
let perm = ref<boolean>(true);

const menuData = ref([
  {
    key: '/main/welcome',
    id: '/main/welcome',
    title: '欢迎',
    authority: perm.value,
  },
  {
    key: '/main/about',
    id: '/main/about',
    title: '关于',
    authority: perm.value,
  },
  {
    key: 'member',
    id: 'member',
    title: '会员管理',
    authority: perm.value,
```

```
    children: [
      { key: '/main/passenger', title: '乘车人管理', authority: 'show' },
      { key: '/main/trainDicker', title: '余票查询', authority: 'show' },
      { key: '/main/myConfirmOrder', title: '我的车票', authority: 'show' },
      { key: '/main/trainDicker', title: '控台管理', authority: 'show' },
    ],
  },
  {
    key: 'serviceData',
    id: 'serviceData',
    title: '业务管理',
    authority: 'serviceData',
    children: [
      { key: '/daily/confirmOrder', title: '订单信息', authority: '/daily/confirmOrder' },
      { key: '/daily/dailyTrainDicket', title: '余票查询', authority: '/daily/dailyTrainDicket' },
      { key: '/main/daily/dailyTrain', title: '每日车次', authority: '/main/daily/dailyTrain' },
      { key: '/main/daily/dailyTrainStation', title: '每日车站', authority: '/main/daily/dailyTrainStation'
},
      { key: '/main/daily/dailyTrainCarriage', title: '每日车厢', authority: '/main/daily/dailyTrainCarriage'
},
      { key: '/main/daily/dailyTrainSeat', title: '每日座位', authority: '/main/daily/dailyTrainSeat' },
    ],
  },
  {
    key: 'baseData',
    id: 'baseData',
    title: '基础数据',
    authority: 'baseData',
    children: [
      { key: '/main/base/station', title: '车站管理', authority: '/main/base/station' },
      { key: '/main/base/train', title: '火车管理', authority: '/main/base/train' },
      { key: '/main/base/trainStation', title: '火车车站', authority: '/main/base/trainStation' },
      { key: '/main/base/trainCarriage', title: '火车车厢', authority: '/main/base/trainCarriage' },
      { key: '/main/base/trainSeat', title: '火车座位', authority: '/main/base/trainSeat' },
    ],
  },
  {
    key: 'batchData',
    id: 'batchData',
    title: '跑批管理',
    authority: 'batchData',
    children: [
      { key: '/main/batch/batchJob', title: '任务管理', authority: '/main/batch/batchJob' },
    ],
  },
]);

//监控路由变化
watch(() => router.currentRoute.value.path, (newValue) => {
  selectedKeys.value = [];
  selectedKeys.value.push(newValue);
}, {immediate: true});

//权限判断
function checkPermission(authority:any) {
  return authority === '1';
}

//加载后调用
onMounted(() => {
  perm.value = checkPermission(sessionStorage.getItem('memberId'));
})
</script>
```